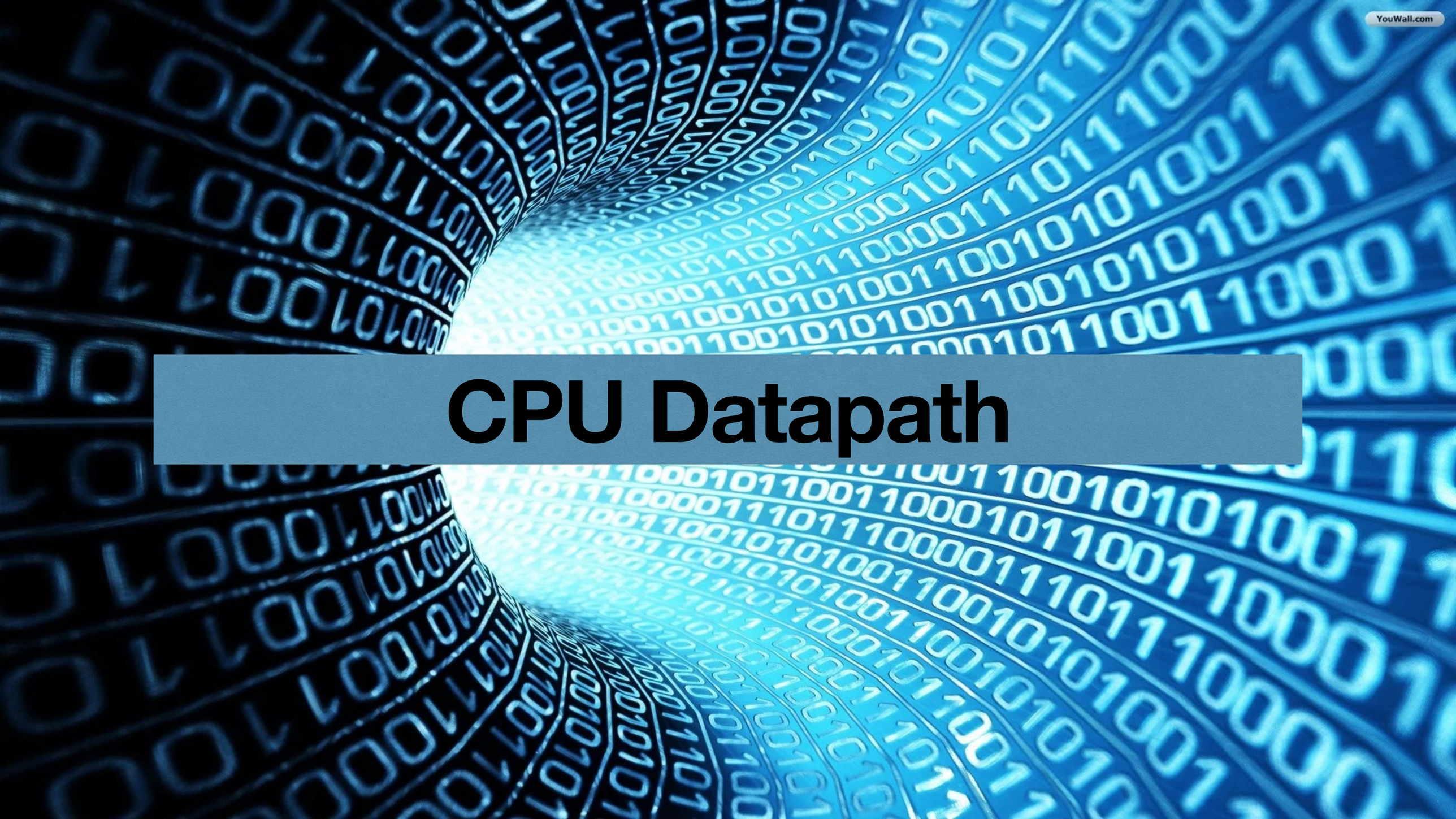
Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay



CPU Datapath

What is “Datapath” for a Processor?

- Same as the datapath control split we studied for GCD processor.
  - **Datapath processes the data**
  - **Controller tells how to process the data**
- But here it’s for processing the instructions in an ISA
- A datapath that can process all the instructions in an ISA

Image generated by ChatGPT

Datapath Elements

- We shall begin with the simplest case
  - Every instruction finishes in **one clock cycle**
  - The clock frequency is determined by the instruction finishes last
- Remember, again how we constructed the GCD datapath
- Let’s first identity what are the components needed

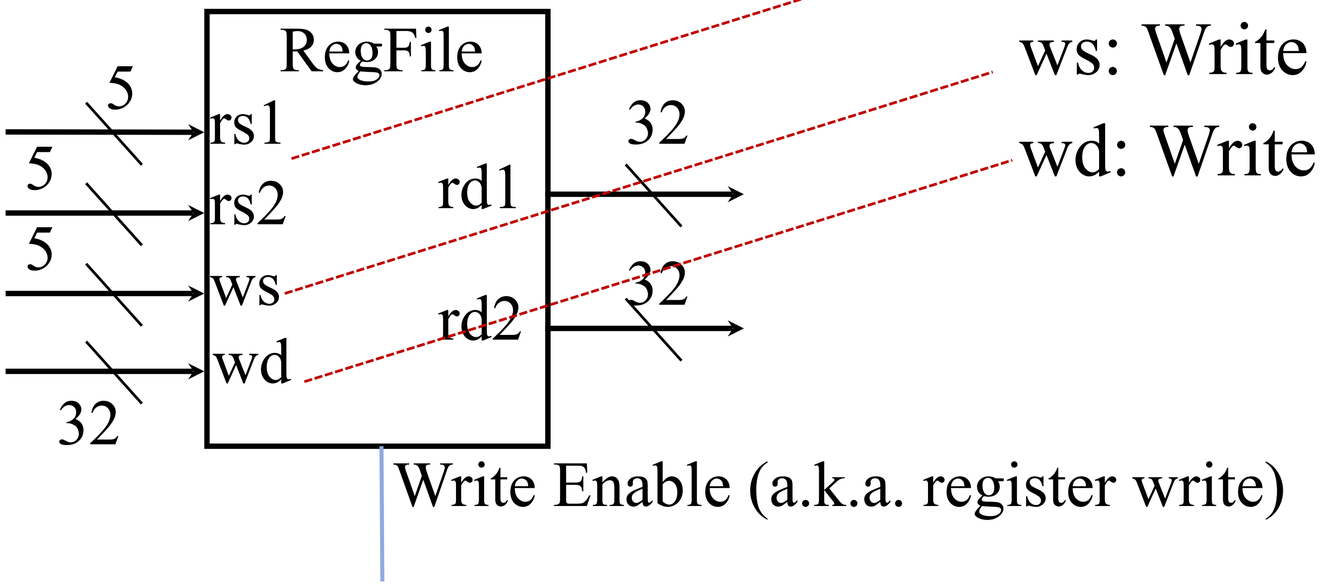
Image generated by ChatGPT

Datapath Elements

- Remember, again how we constructed the GCD datapath
- Let’s first identity what are the components needed

Image generated by ChatGPT

Datapath Elements: Register File

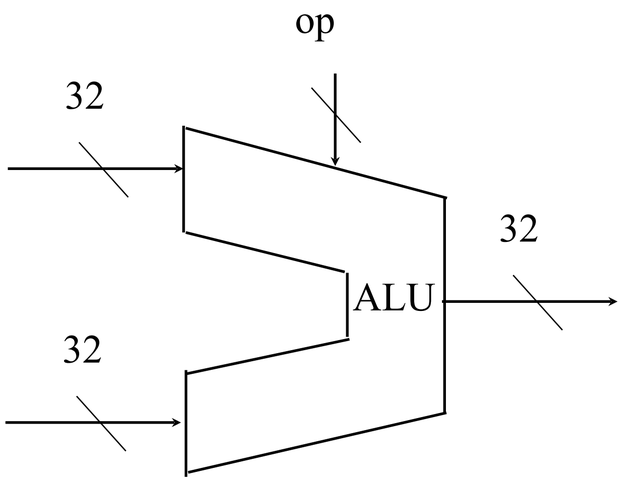


rs1 and rs2: Read register 1 and 2

ws: Write register

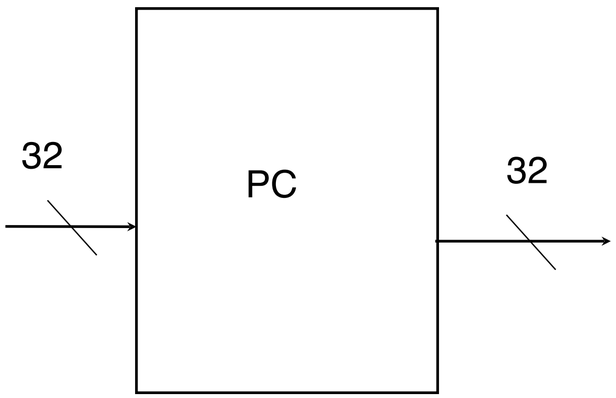
wd: Write data

Datapath Elements: The ALU



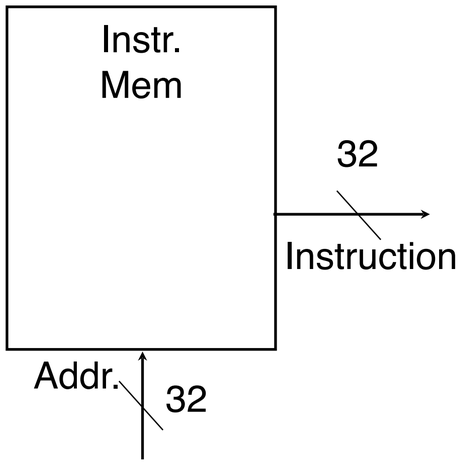
- All arithmetic and logical operation happens here
- Operation selection
- E.g.,
  - Op = 0: Y = A+B
  - OP = 1: Y = A- B
  - OP = 2; Y = A*B

Datapath Elements: Program Counter



- Remember PC register??
- It always points to the next instruction to be executed…
- But where is the next instruction???

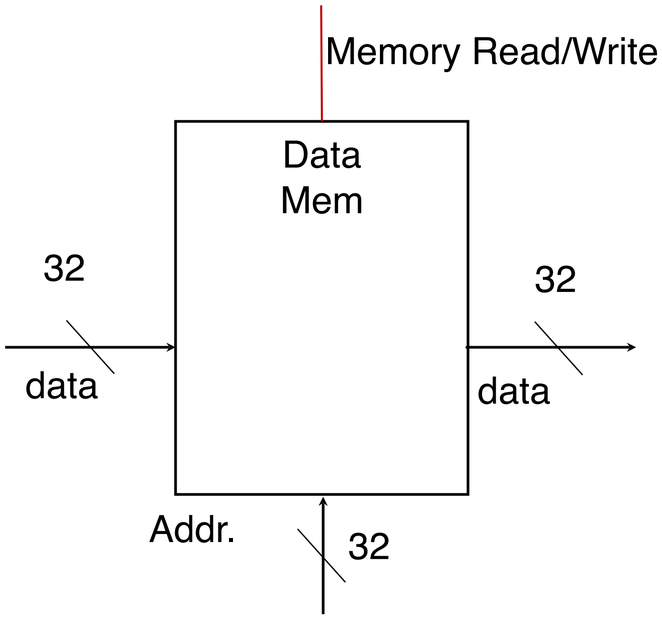
Datapath Elements: Instruction Memory



Remember: No writes to instruction memory

Not concerned about how programs are loaded into this memory.

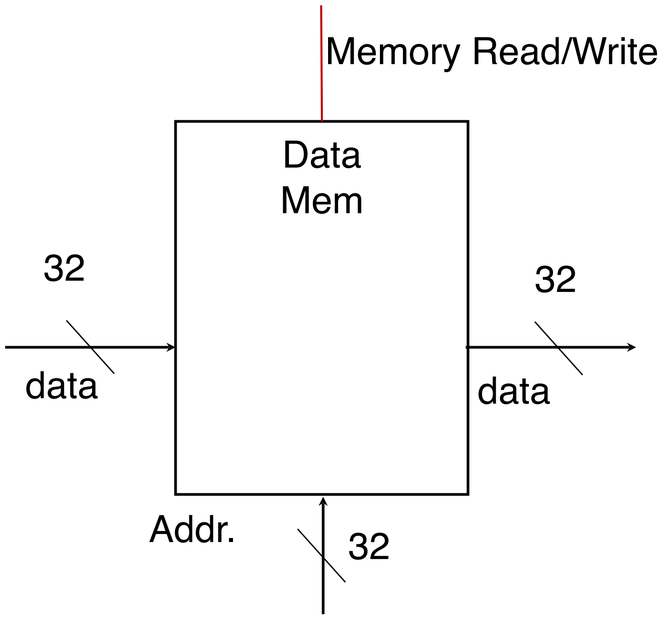
Datapath Elements: Data Memory



Why data and instruction memory and not one memory?

- Recall Harvard vs. Von Neuman
- Not so simple matter, will discuss later.

Datapath Elements: Data Memory



Why data and instruction memory and not one memory?

- Recall Harvard vs. Von Neuman
- Not so simple matter, will discuss later.

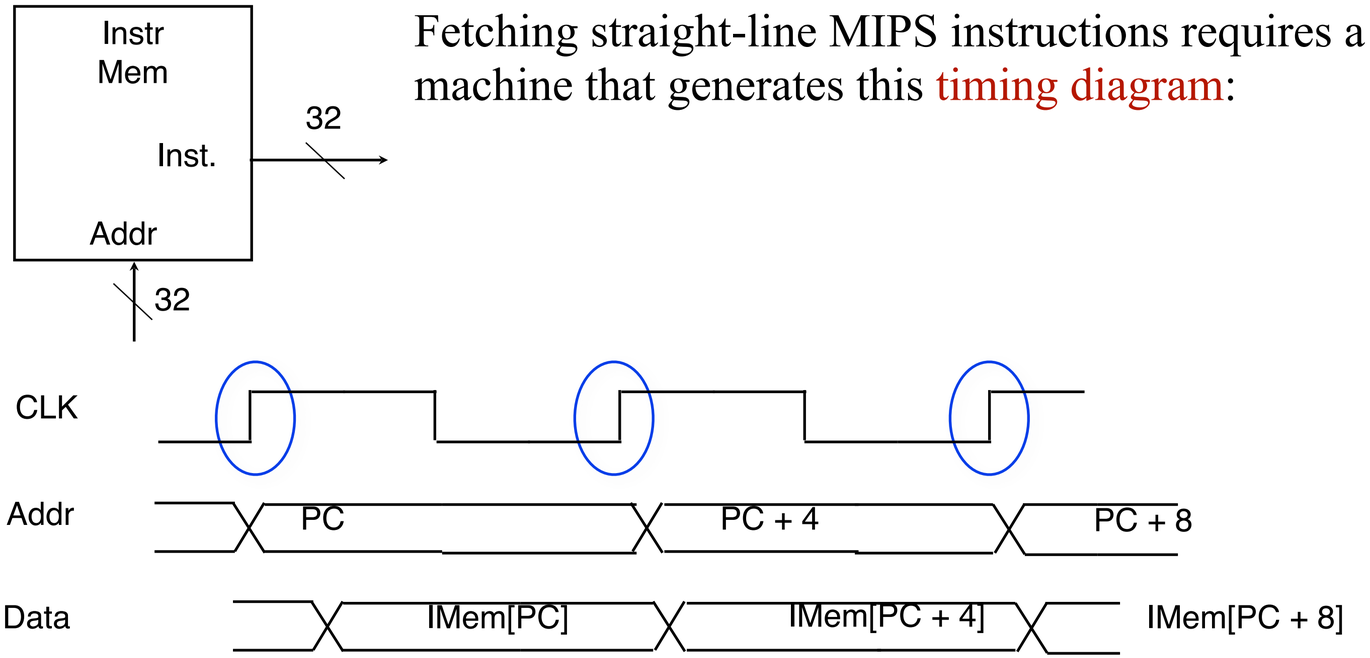
Datapath Elements: Buses

- Same as your public transport
- Transfer data and instructions
- Separate buses for address and data

Datapath Elements: Buses

- Same as your public transport
- Transfer data and instructions
- Separate buses for address and data
  - Why???

Datapath Elements: Timing Diagram

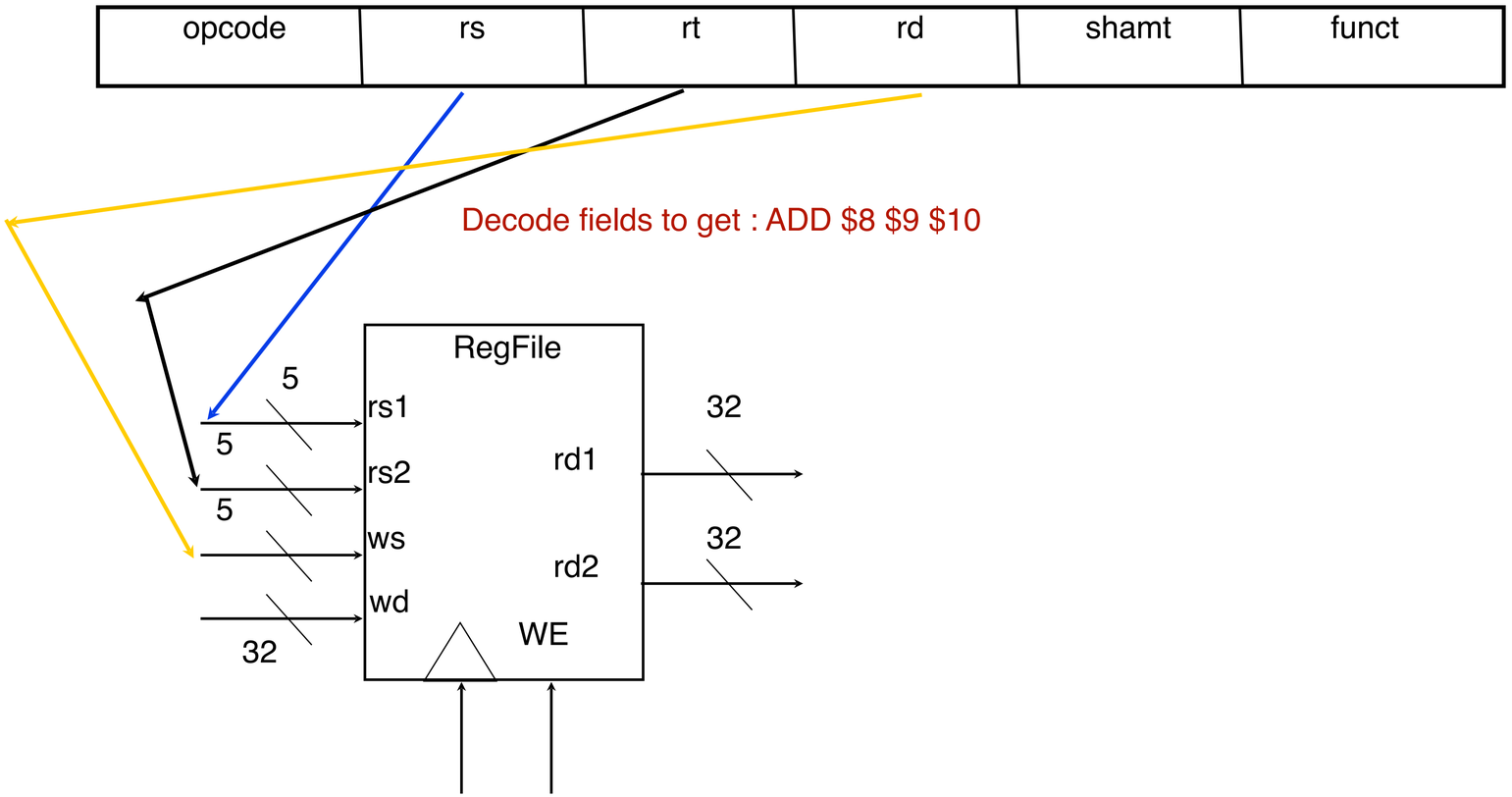


Fetching straight-line MIPS instructions requires a machine that generates this timing diagram:

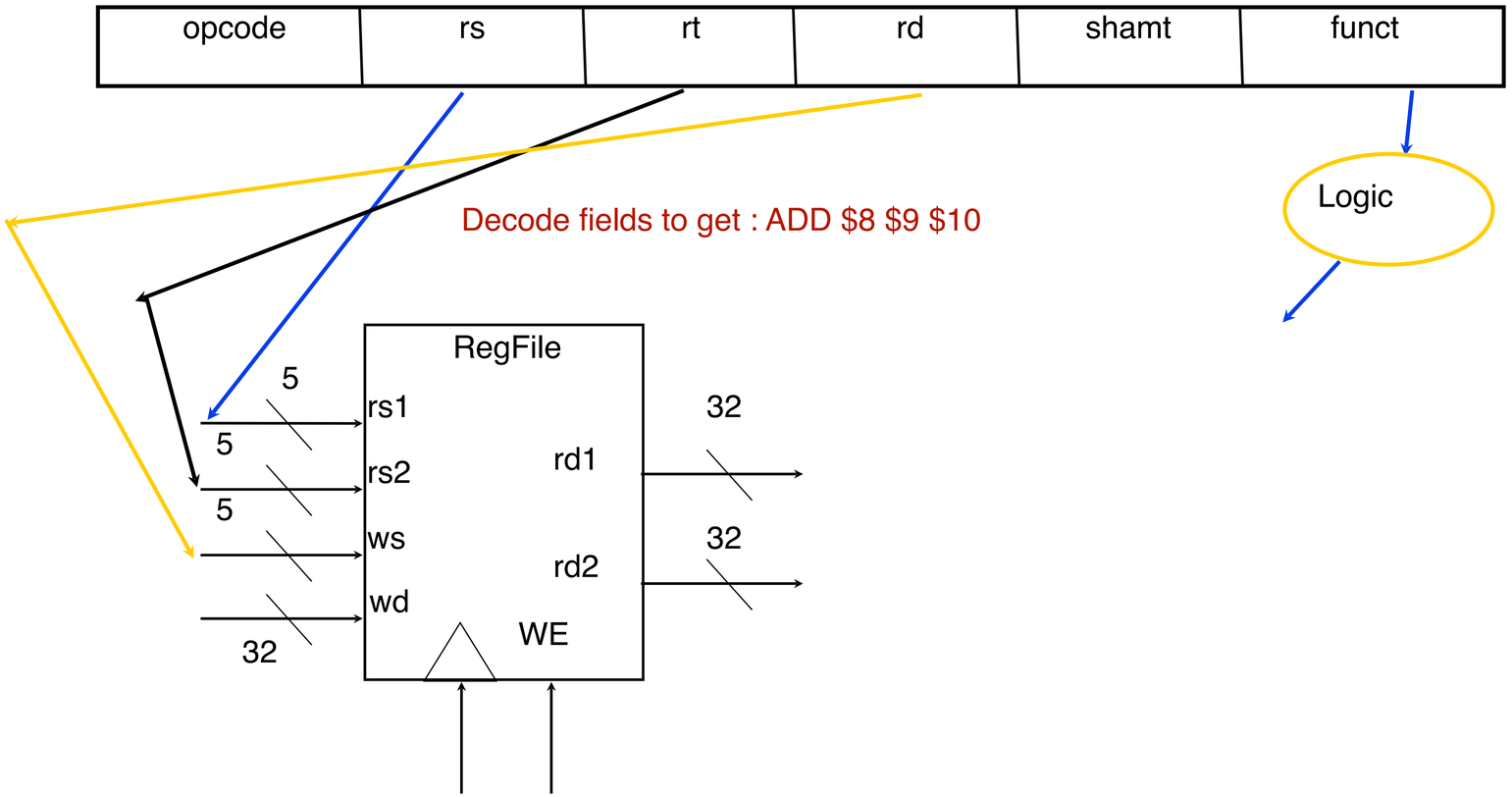
- Every clock cycle we process one instruction (super simple)
- Sending the PC and getting the instruction from memory is called **Instruction Fetch**

PC == Program Counter, points to next instruction.

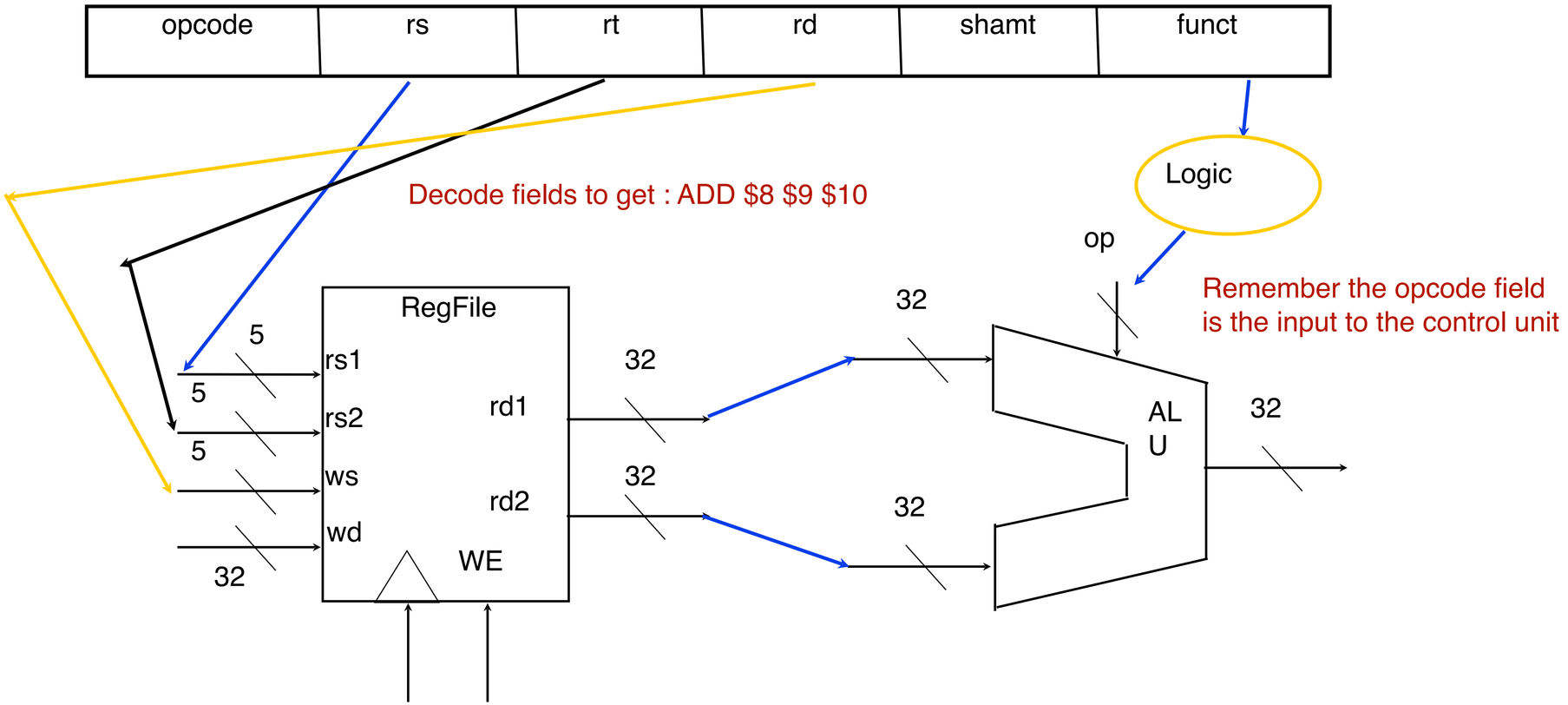
Datapath Elements: Decoding Instructions



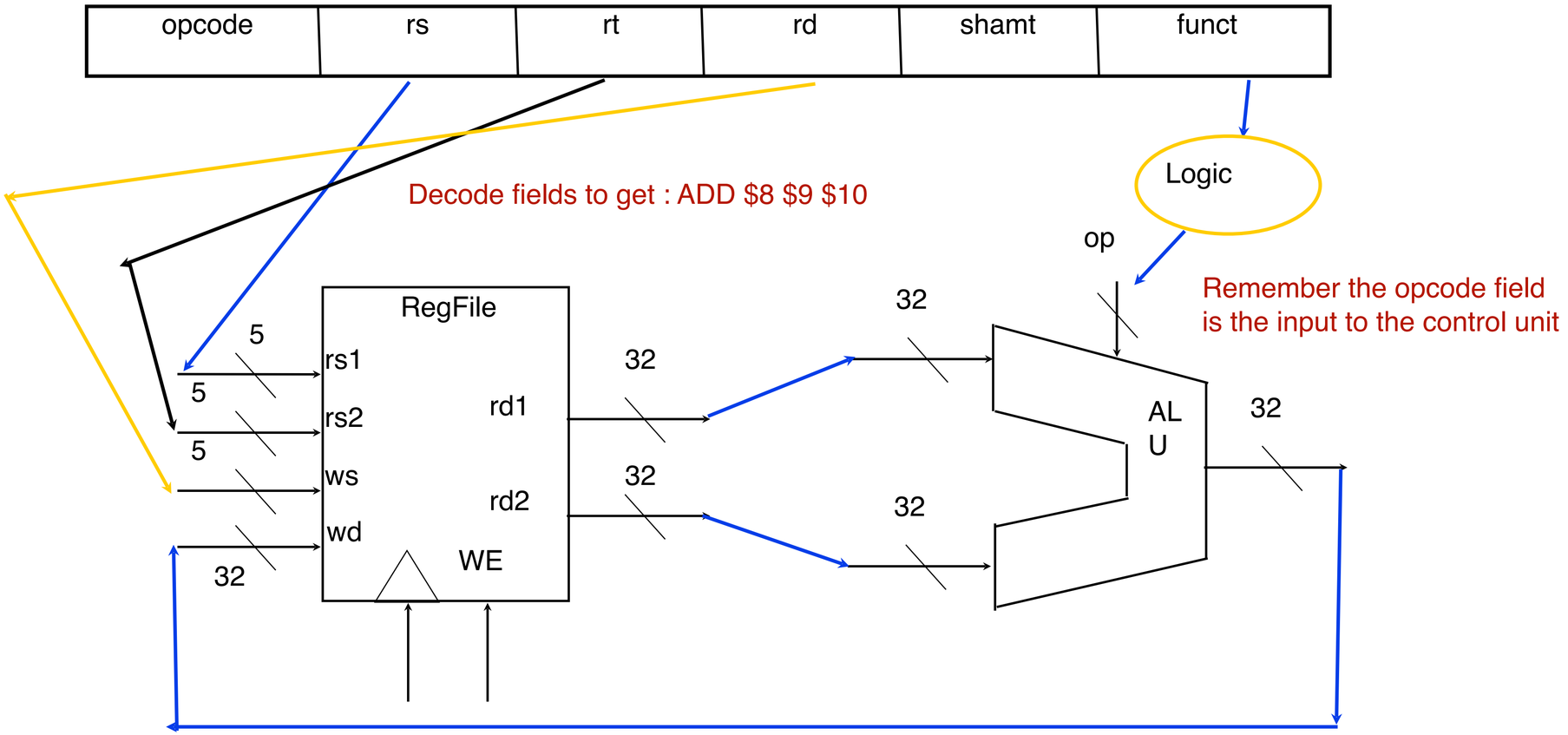
Datapath Elements: Decoding Instructions



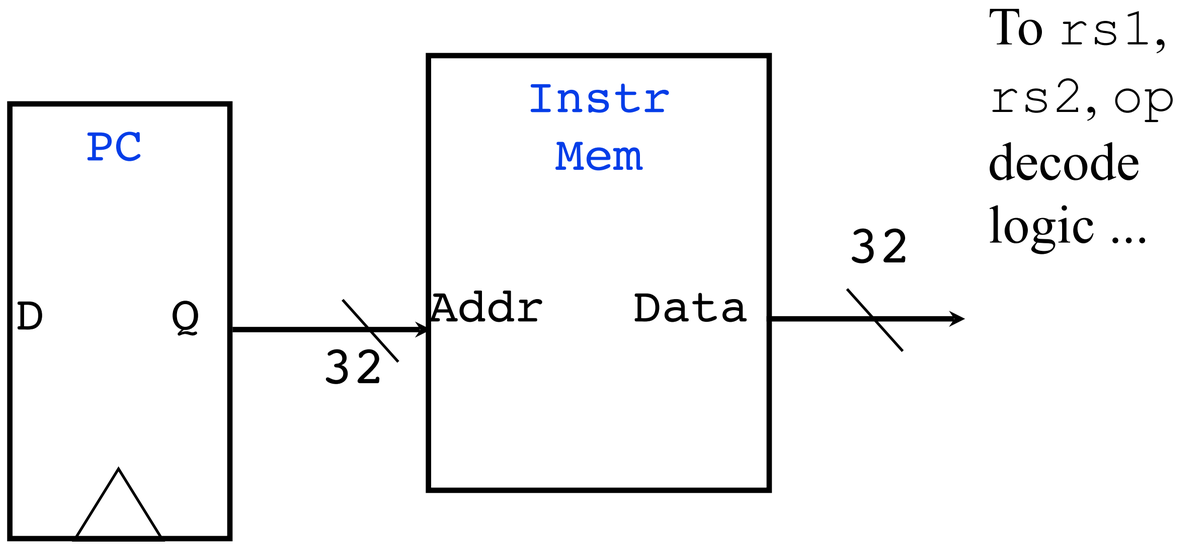
Datapath Elements: Executing Instructions



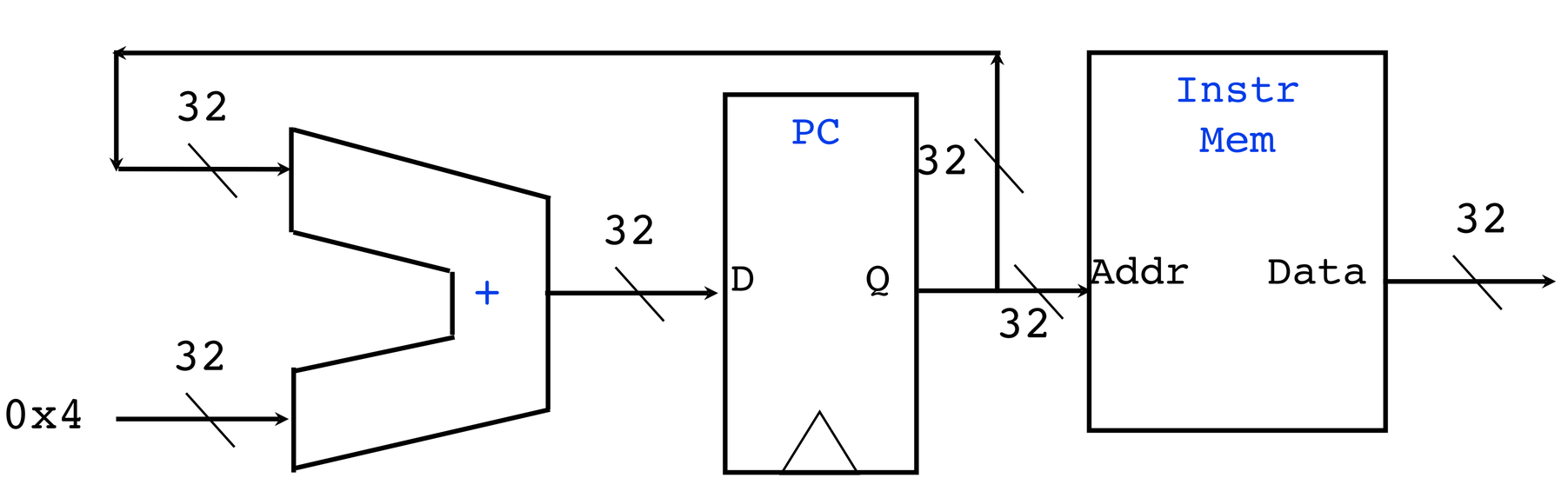
Datapath Elements: Executing Instructions



Datapath For Instruction Fetch

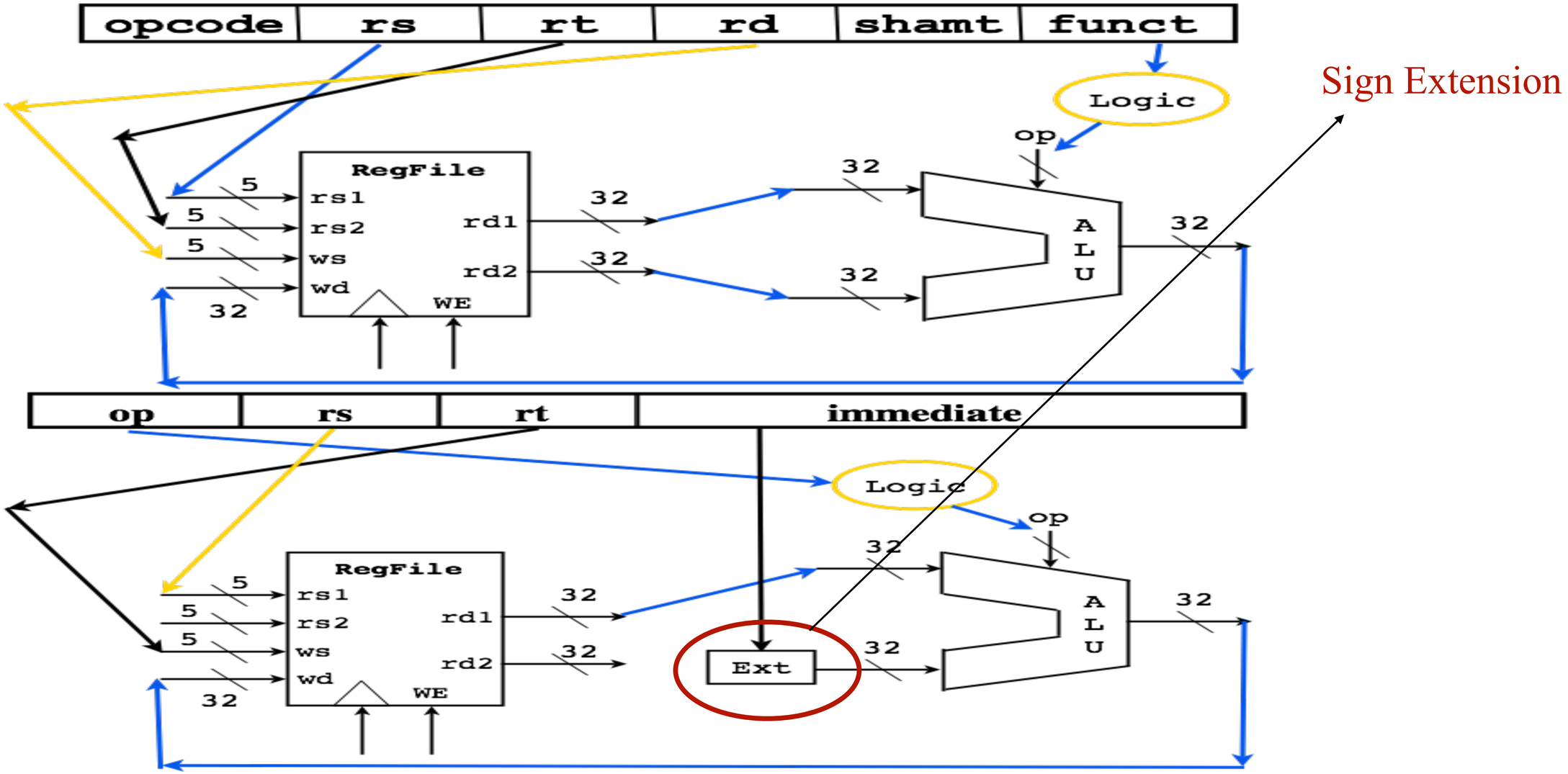


Datapath For Instruction Fetch

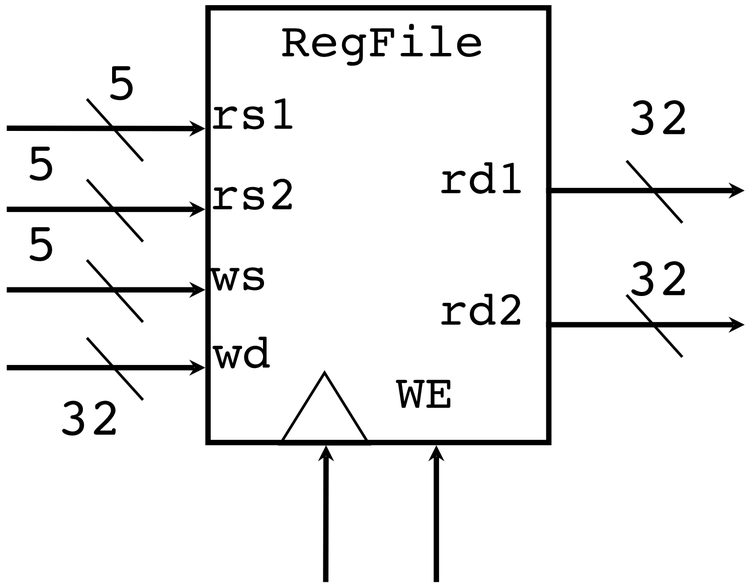


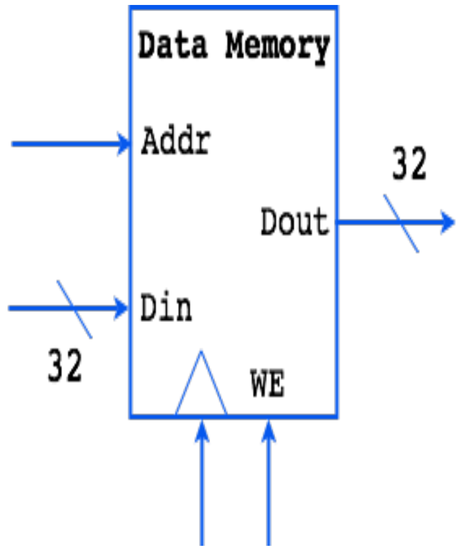
To `rs1`, `rs2`, `op` decode logic ...

What about I format?



Loads from Memory

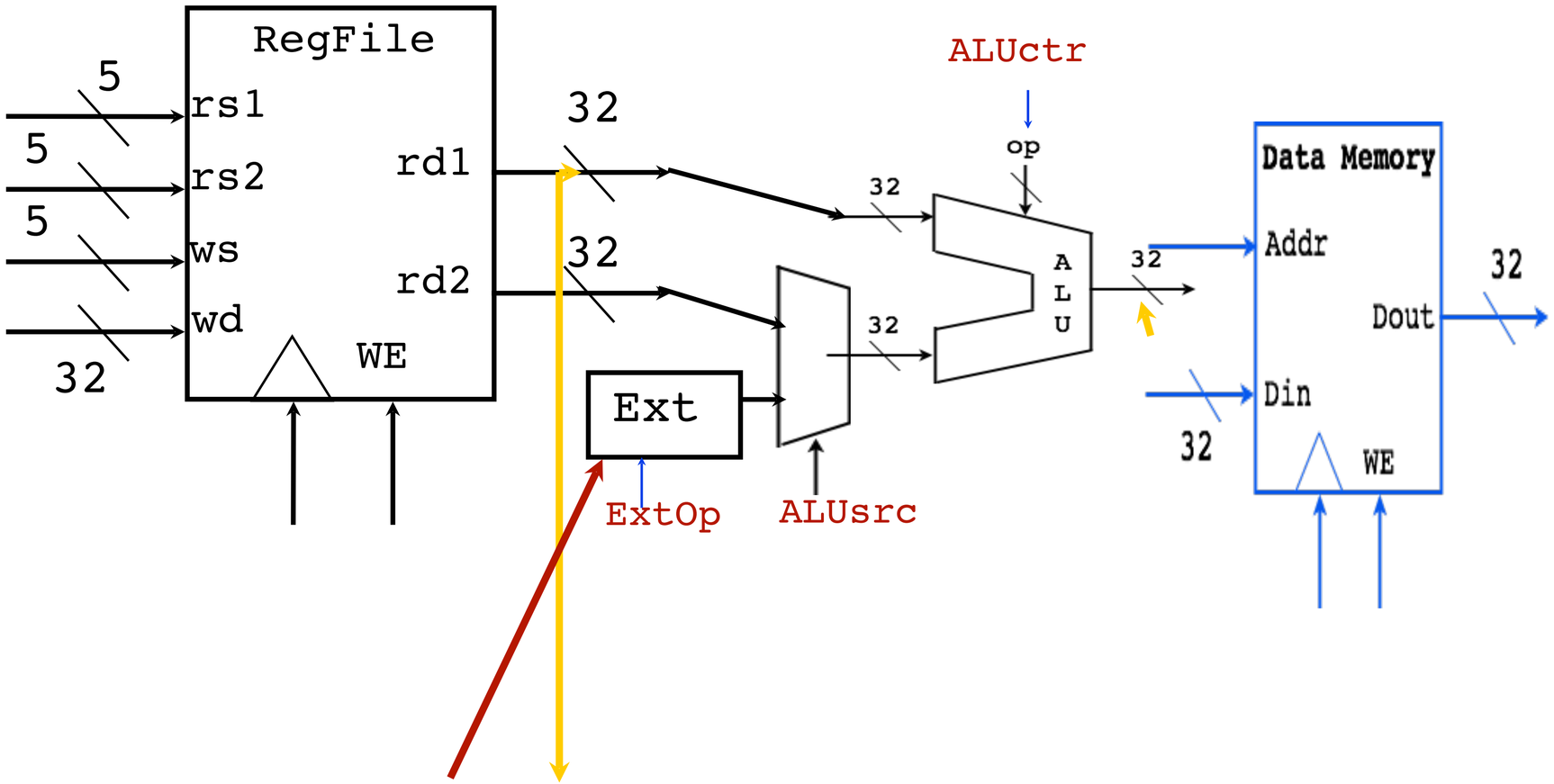




Syntax: LW $1, 32($2)

Action: $1 = M[$2 + 32]

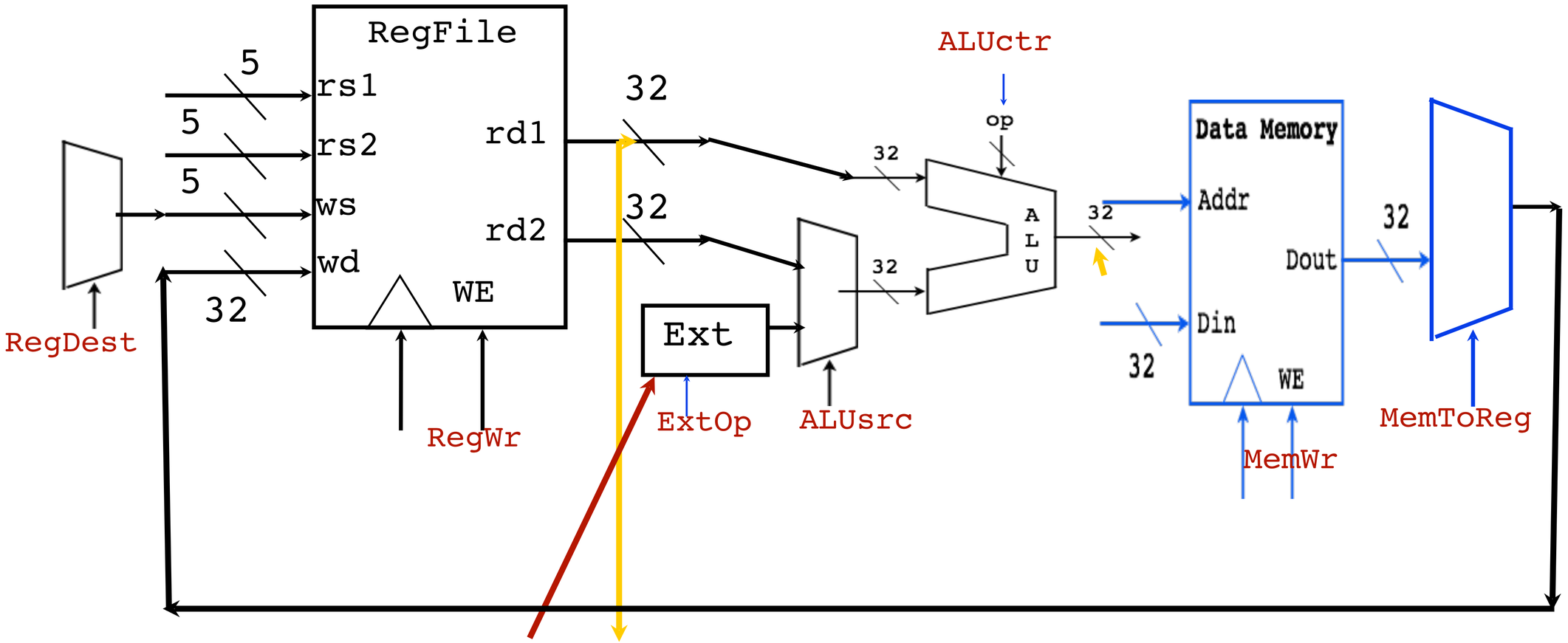
Loads from Memory



Syntax: LW $1, 32($2)

Action: $1 = M[$2 + 32]

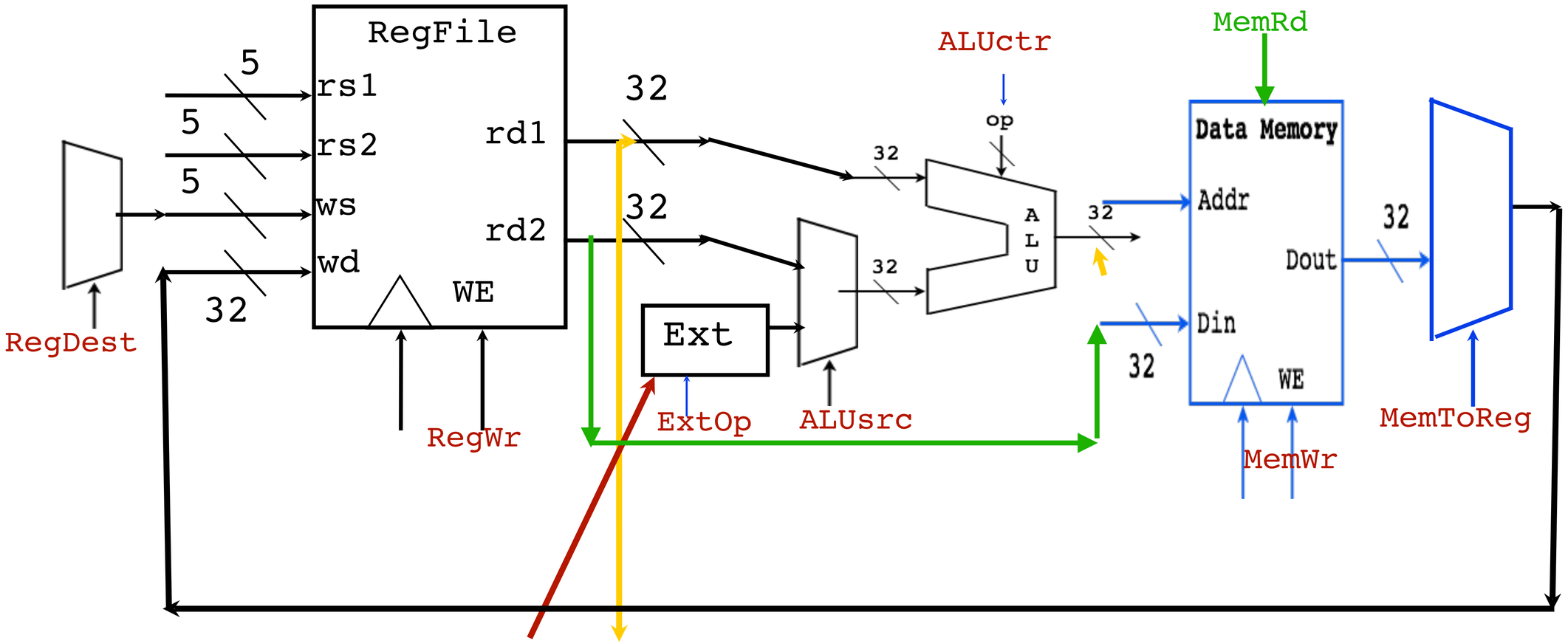
Loads from Memory



Syntax: LW $1, 32($2)

Action: $1 = M[$2 + 32]

Stores to Memory (with Load Datapath)

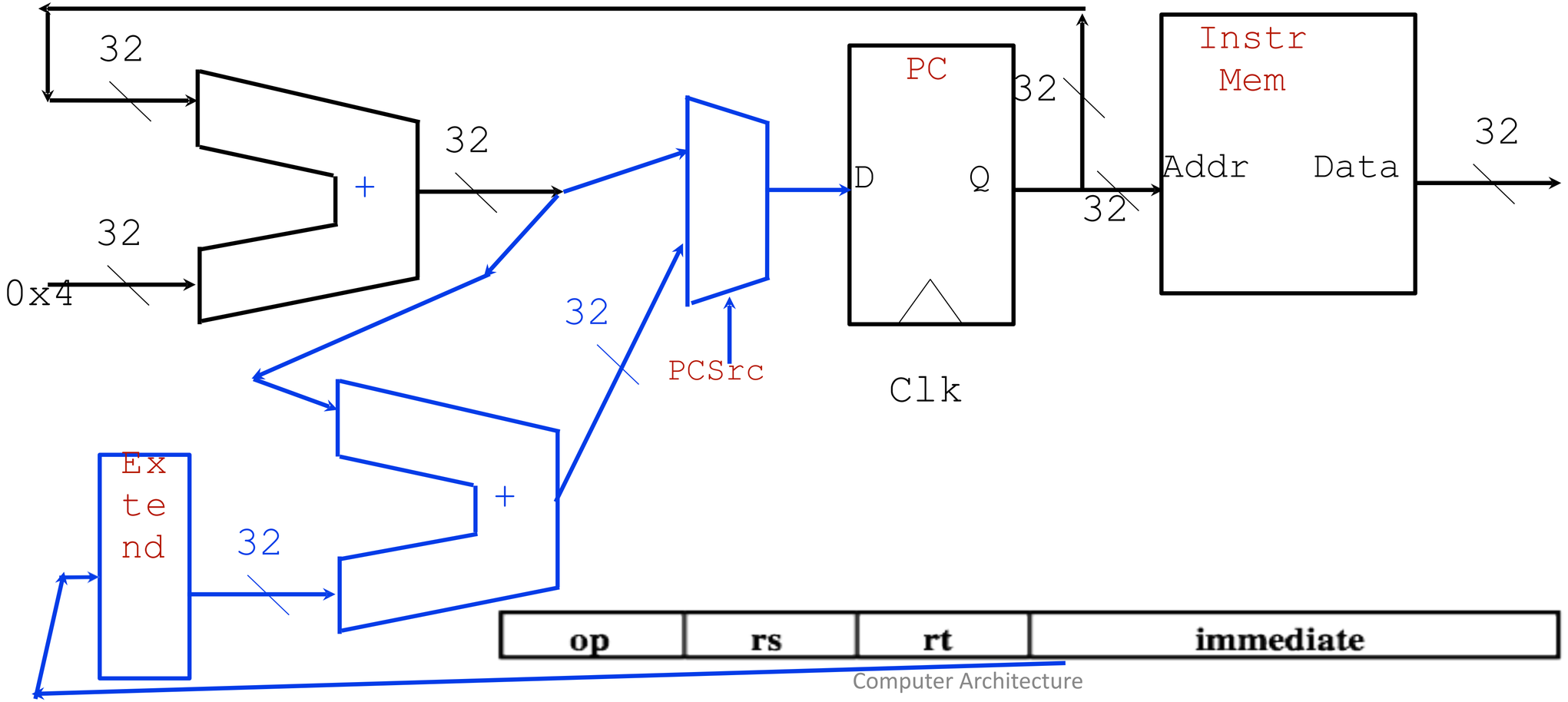


Syntax: SW $1, 32($2)

Action: M[$2 + 32] = $1

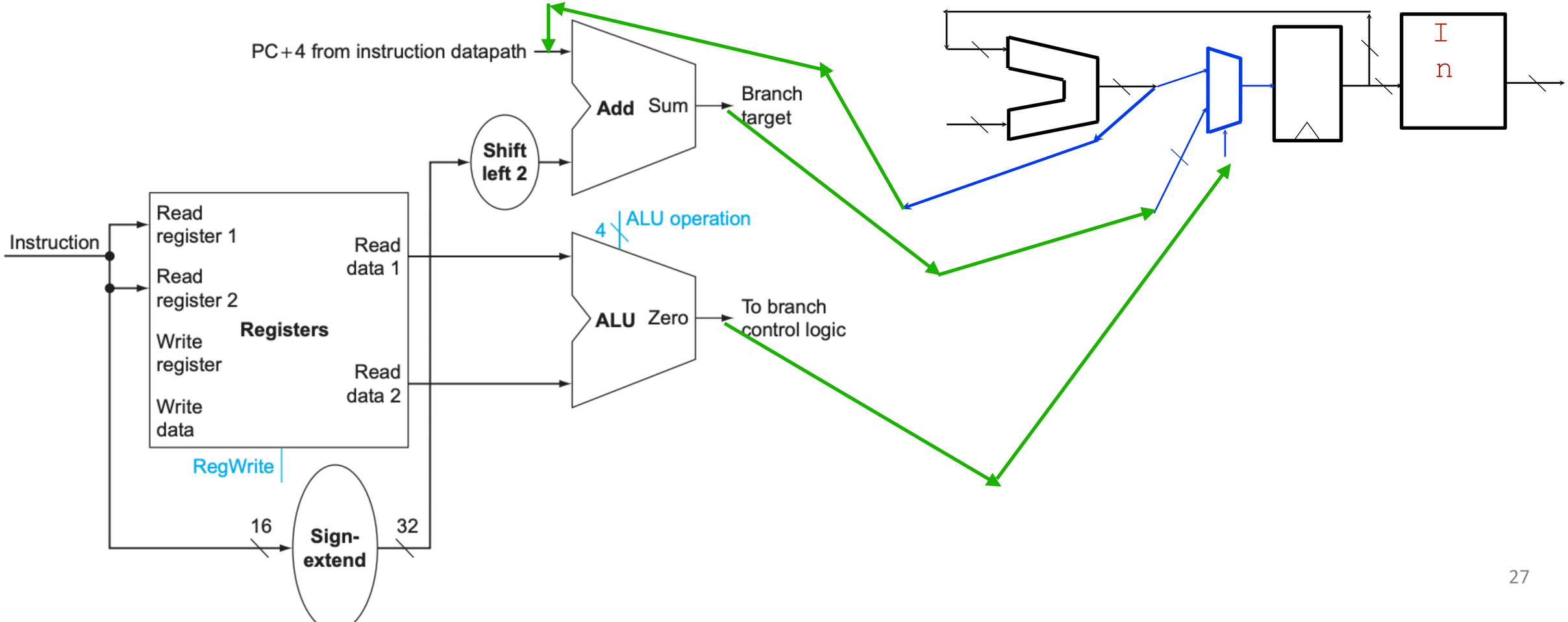
Branch Instructions

```
Syntax: BEQ $1, $2, 12
Action: If ($1 != $2), PC = PC + 4
Action: If ($1 == $2), PC = PC + 4 + 48
```

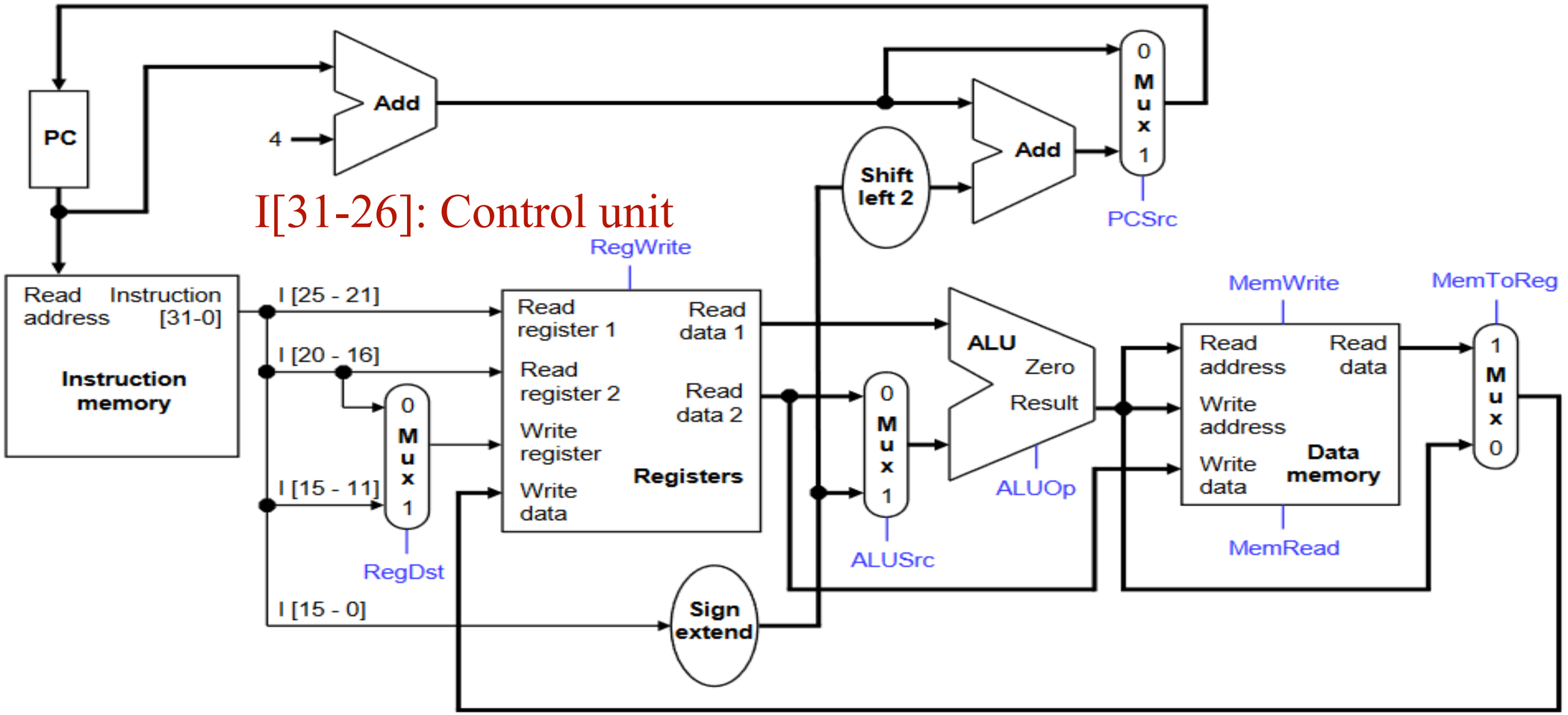


Branch Instructions

```
Syntax: BEQ $1, $2, 12
Action: If ($1 != $2), PC = PC + 4
Action: If ($1 == $2), PC = PC + 4 + 48
```

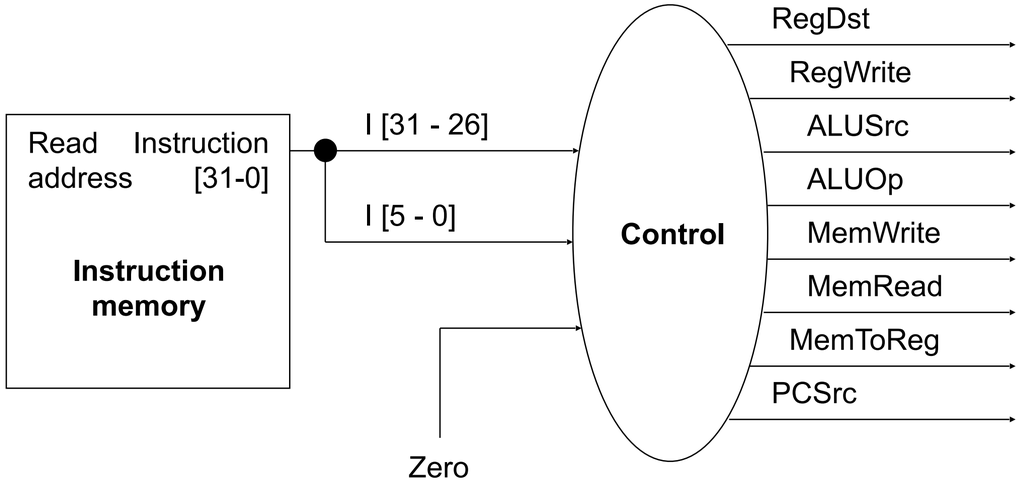


The Complete Picture



Control Signals So far

- MemRead
- MemWrite
- RegWrite
- MemtoReg
- RegDst
- ALUop, ALUSrc
- PCSrc



In Detail

- MemRead: Read from memory when assert
- MemWrite: Write into the memory when assert
- RegWrite: Reg. on Write register updated with the input, on assert
- MemtoReg: On assert, memory to register, on deassert, ALU to register
- RegDst: On assert, use rd field, on deassert use rt field
- ALUSrc: On assert, lower 16 bits of an inst., on deassert from the second register
- PCSrc: On assert, branch target, deassert, PC+4

Control Signal Table

Operation	RegDst	RegWrite	ALUSrc	ALUOp	MemWrite	MemRead	MemToReg
add	1	1	0	010	0	0	0
sub	1	1	0	110	0	0	0
and	1	1	0	000	0	0	0
or	1	1	0	001	0	0	0
slt	1	1	0	111	0	0	0
lw	0	1	1	010	0	1	1
sw	X	0	1	010	1	0	X
beq	X	0	0	110	0	0	X

Why not single cycle?

- The longest possible datapath is the clock cycle time.

What does it mean?

Why not single cycle?

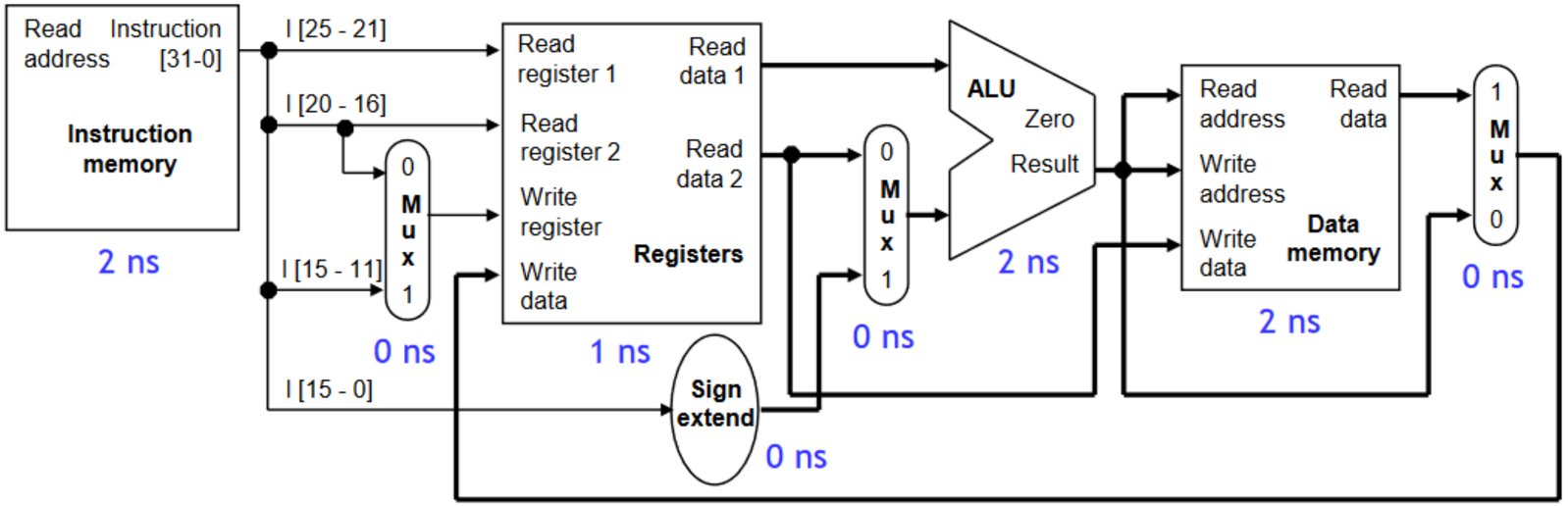
- For example, lw $t0, -4($sp) needs 8ns, assuming the delays shown here.

reading the instruction memory	2ns	8ns
reading the base register $sp	1ns	
computing memory address $sp-4	2ns	
reading the data memory	2ns	
storing data back to $t0	1ns	

one clock cycle: 8ns

Processor frequency: 125MHz

Cycle per Instruction (CPI): 1



An add instruction: no need of 8ns

Why not single cycle?

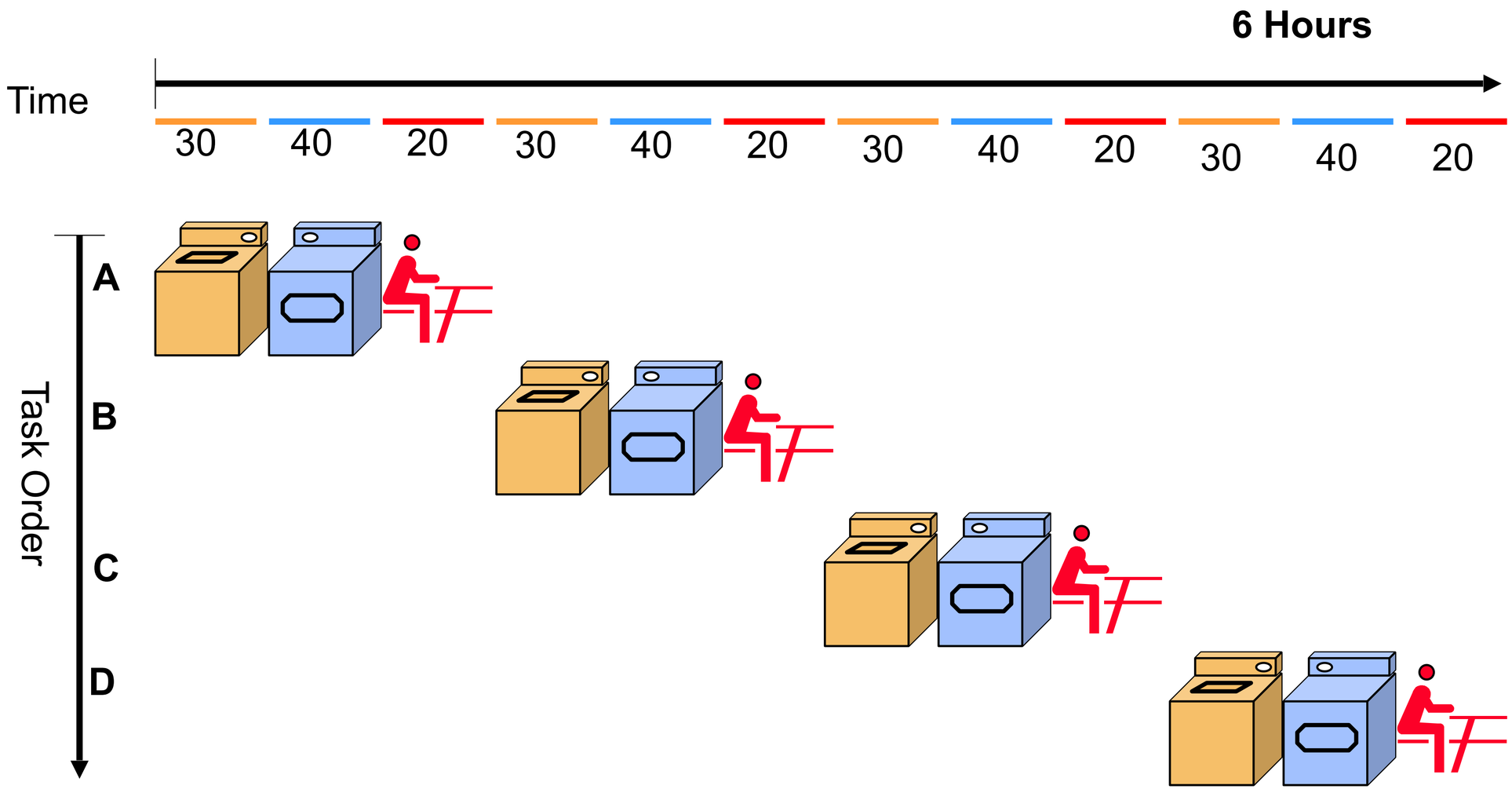
- The longest possible datapath is the clock cycle time.

Violating *common case fast* — Basic principle of computer architecture.

Now what should we do??

Why not single cycle?

A, B, C, D wants to wash cloths



Single Vs. Multi-cycle: Where is the Gain?

Single cycle (Worst case)

Everyone takes 90 mins Full package = 90 minutes

Multi cycle (average case kinda)

One person: 20 to 90 mins avg = 53.33 minutes

Single to Multi Cycle

- Measuring Performance:

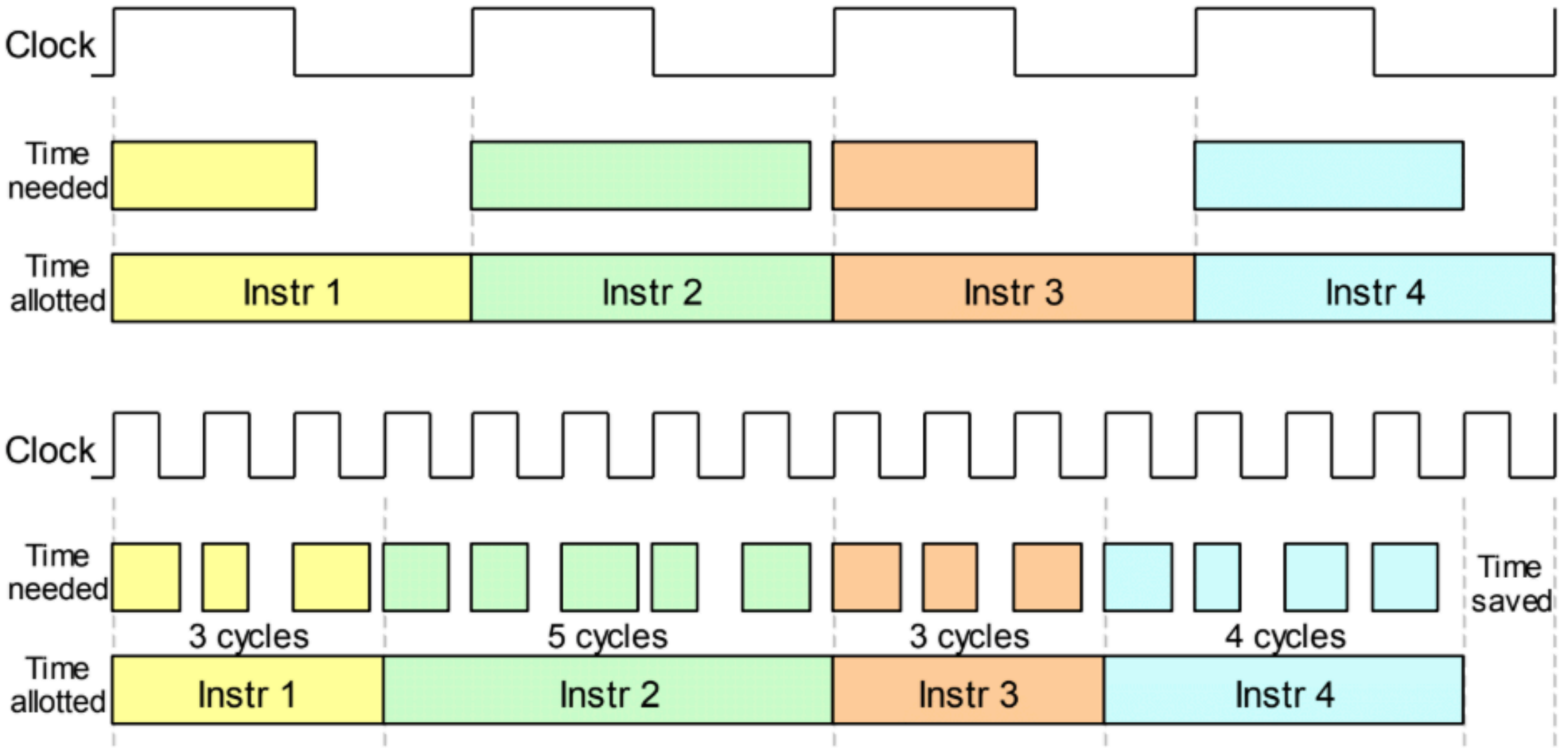
$$Execution\ Time = \#Instructios \times Cycles\ Per\ Instruction \times Clock\ Cycle\ Time$$

$$Speedup = \frac{Execution\ Time_{old}}{Execution\ Time_{new}}$$

  - Cycles Per Instruction (CPI) = 1 for single cycle processor
    - Very good, can't be better in simple scenarios
    - But still it's problematic — Why?
      - The clock cycle time is bounded by the length of the critical path of the datapath — here it is the load instruction
      - In simple words, the overall computation time will be poor!!!

  - How can we improve?
    - Multi-cycle datapath: Improve clock frequency — but poor CPI — not a great option

Single to Multi Cycle

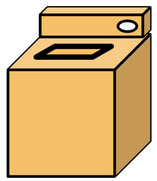


Can We Have Both?

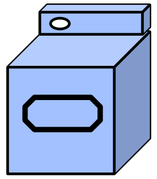
Faster clock rate and also CPI=1?

Basic Intuition

A, B, C, D wants to wash cloths

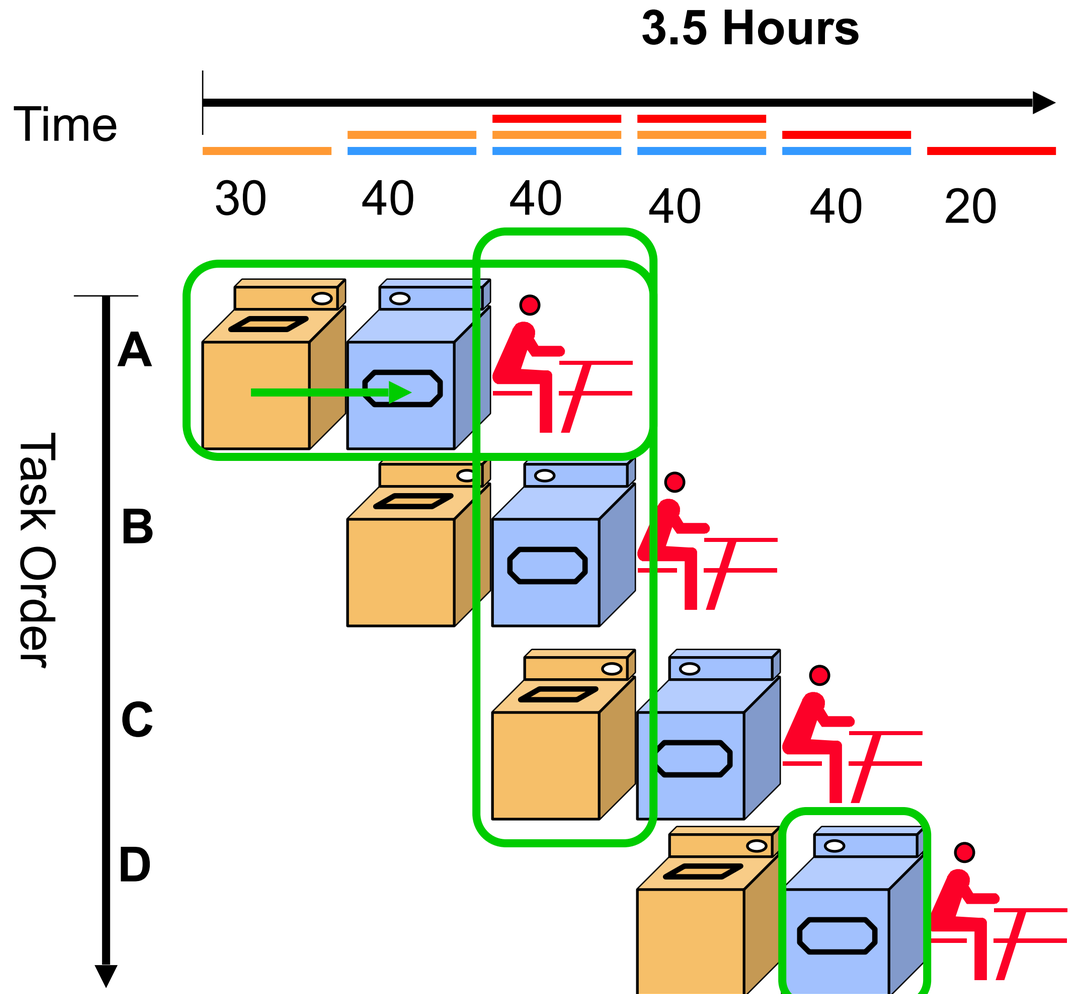


washer
30 mins



dryer
40 mins

folder
20 mins



Main Observations

- Different computation stages which can have overlapped tasks
  - This is the point of speedup!!!
- The timing of each computation stage is determined by the most time consuming task
- **This is called pipelining**
- **Tricky**:
  - Each person still needs 90 mins.
  - But considering all A, B, C, D, the average execution time improves a lot.

Latency and Bandwidth (throughput)

- Latency
  - time it takes to complete one instance

- Throughput
  - number of computations done per unit time

Following slides are adapted (with modifications) from Biswa's slides

Let’s Summarize

Single cycle: CPI: 1 , Cycle time: long

Multi cycle: CPI: >1, Cycle time: short

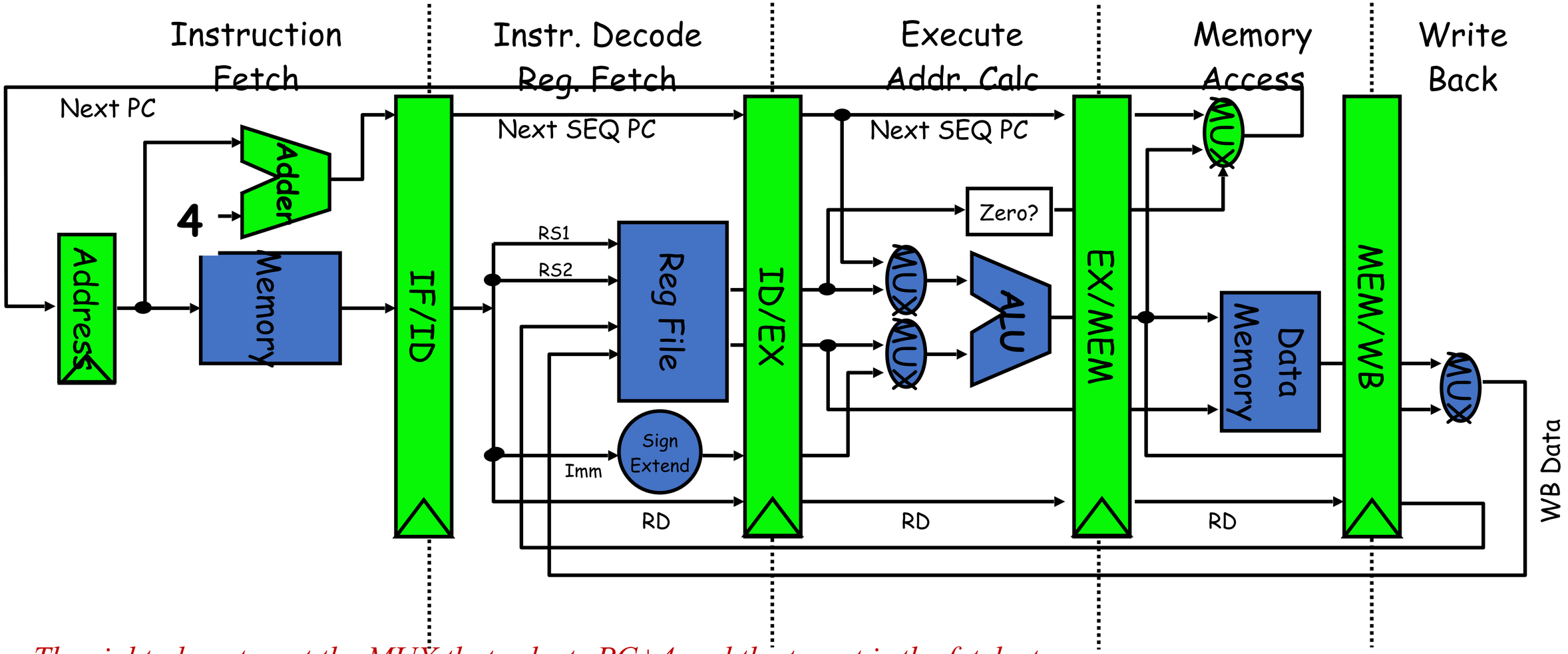
Pipelined: CPI: 1, Cycle time: short (improves throughput but not latency)

Pipelining and Richard Feynman

https://www.youtube.com/watch?v=9miKIWIYi4w

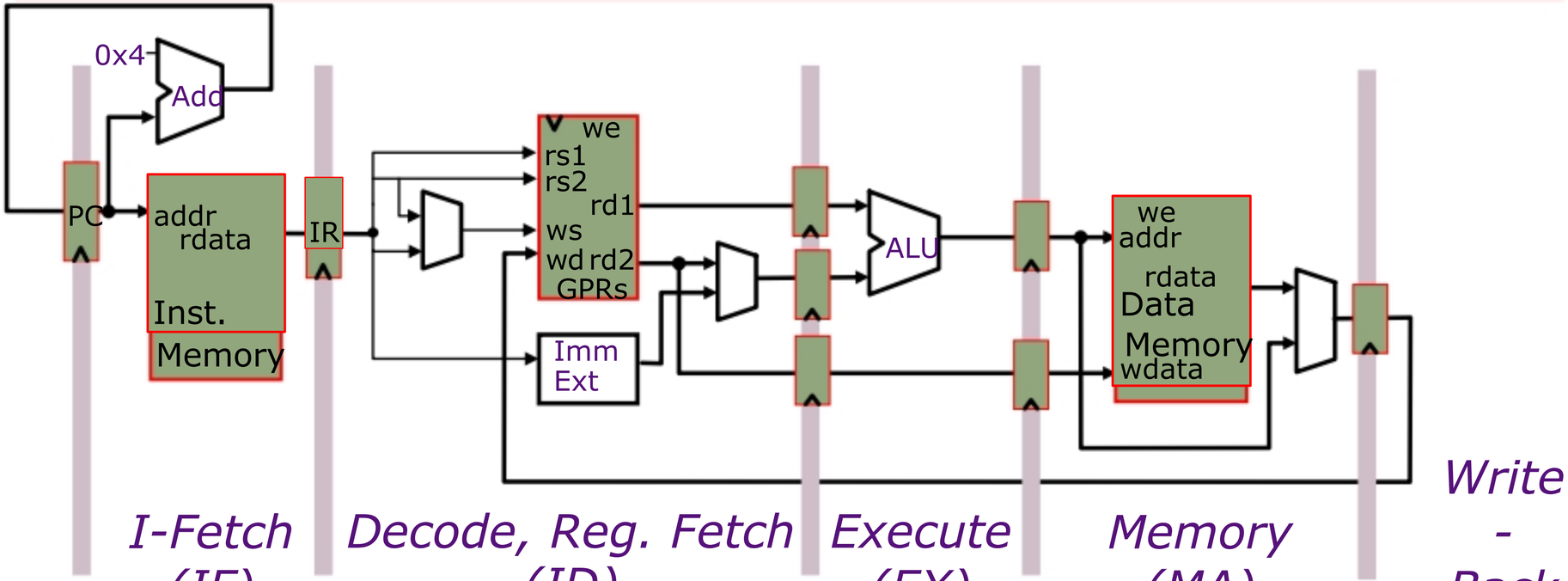
Jump to 1:25

Vanilla 5-stage pipeline



The right place to put the MUX that selects PC+4 and the target is the fetch stage.
The slide shows a vanilla 5-stage pipeline if we just take a single cycle datapath and divide it into five stages.

Resource Utilization

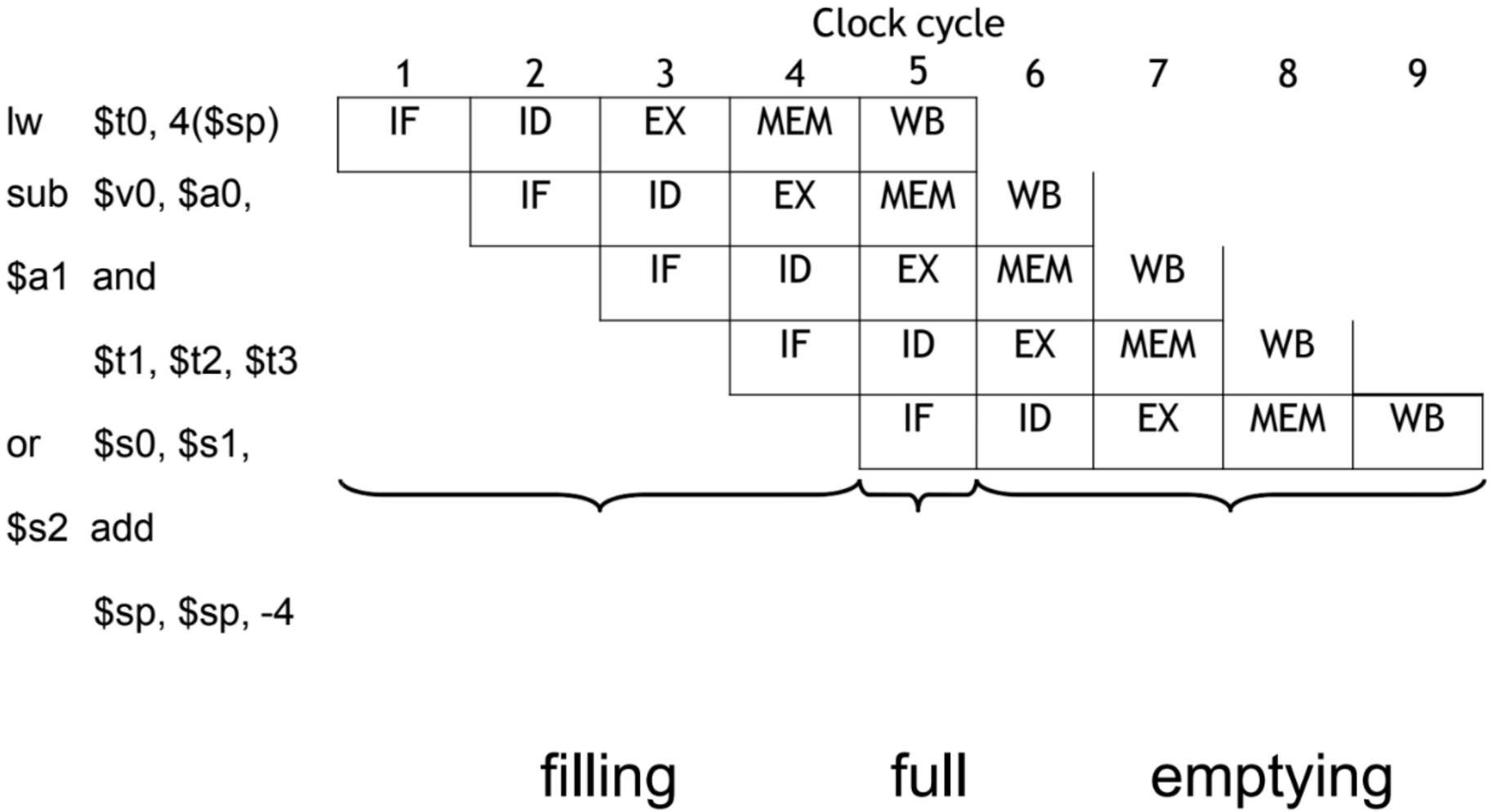


I-Fetch (IF) | *Decode, Reg. Fetch (ID)* | *Execute (EX)* | *Memory (MA)* | *Write - Back (WB)*

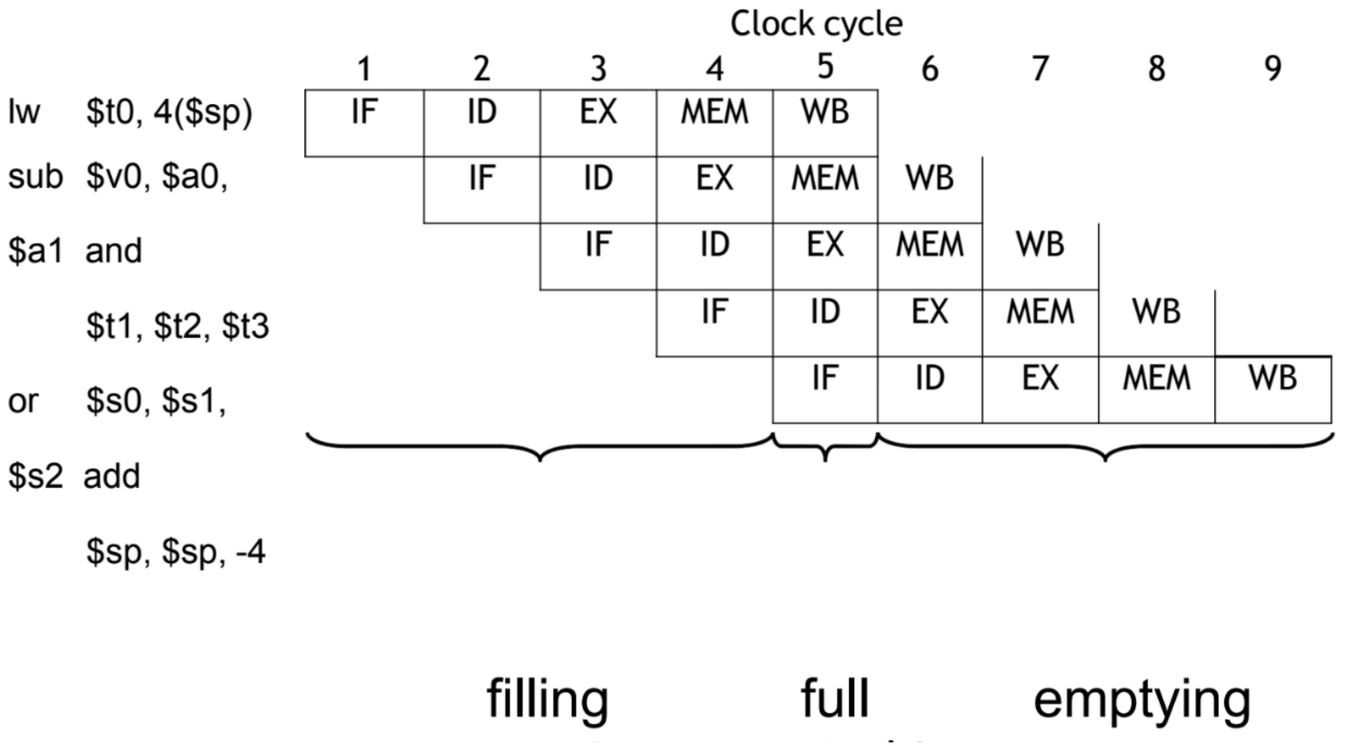
Resources

time	t0	t1	t2	t3	t4	t5	t6	t7	. . . .
IF	I_1	I_2	I_3	I_4	I_5				
ID		I_1	I_2	I_3	I_4	I_5			
EX			I_1	I_2	I_3	I_4	I_5		
MA				I_1	I_2	I_3	I_4	I_5	
WB					I_1	I_2	I_3	I_4	I_5

Visualizing Pipeline



Visualizing Pipeline: Execution Time



For a k-stage pipeline executing N instructions

first instruction: K cycles

Next N-1 instructions: N-1 cycles, total = K + (N-1) cycles

Pipelined versus Single cycle CPU design

Instruction	Ifetch	Decode	Execute	Memory	Writeback	Total time
LOAD	200ns	100	200	200	100	800ns
STORE	200	100	200	200		700ns
ADD	200	100	200		100	600ns
BRANCH	200	100	200			500ns

Total latency in single cycle CPU: 3200 ns

Total latency in pipelined CPU (200ns clock cycle):
1000ns (1st instruction) + 3 X 200 ns (for next three) = 1600 ns

What's the big deal

Speedup = 3200ns/1600ns = 2X

What if we have a billion instructions?

Single cycle = 1 billion * 800ns = 800 seconds

Pipelined = 1000ns + (1 billion -1) * 200ns ~ 200 seconds

Speedup = 4X ☺

Let’s include latch latency too

Inter-stage latch = 10ns

New clock cycle time in the pipelined design = 210ns

First instruction will get completed by 1040ns (five stages X 200 ns + four inter-stage latches * 10ns)

New Speedup = 800s/210s ~ 3.8X

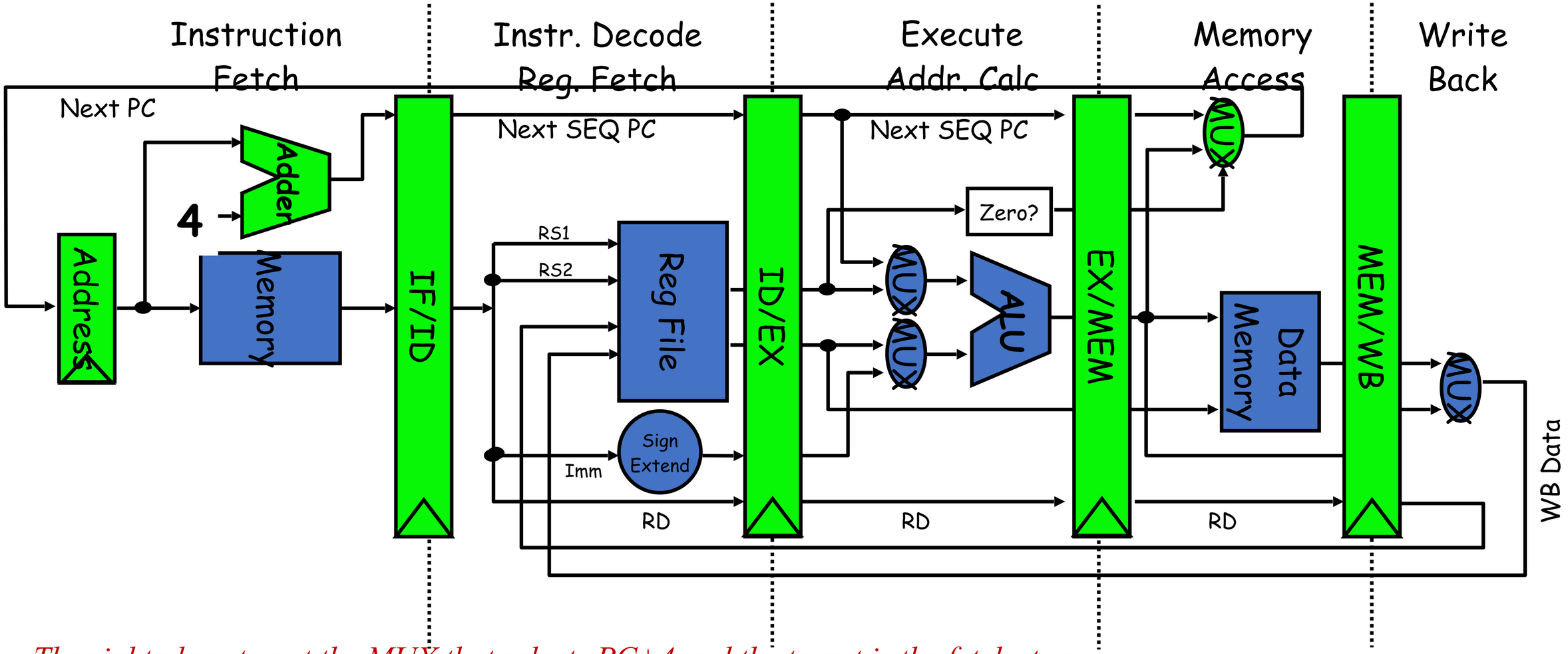
How to Divide the Datapath?

Suppose memory is significantly slower than other stages. For example, suppose

t_{IM} = 10 units
t_{DM} = 10 units
t_{ALU} = 5 units
t_{RF} = 1 unit
t_{RW} = 1 unit

Since the slowest stage determines the clock, it may be possible to combine some stages without any loss of performance

Vanilla 5-stage pipeline

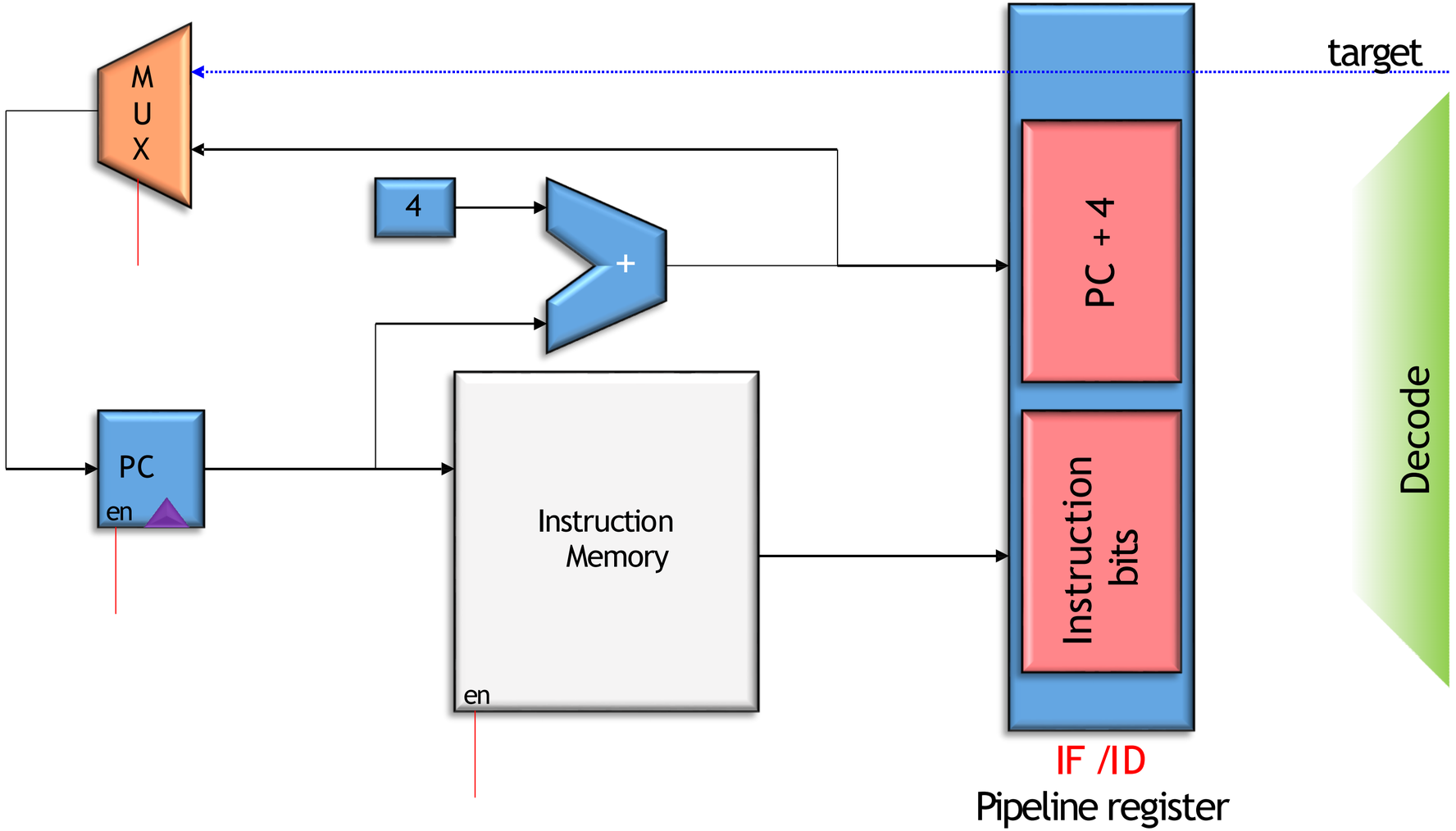


The right place to put the MUX that selects PC+4 and the target is the fetch stage.
The slide shows a vanilla 5-stage pipeline if we just take a single cycle datapath and divide it into five stages.

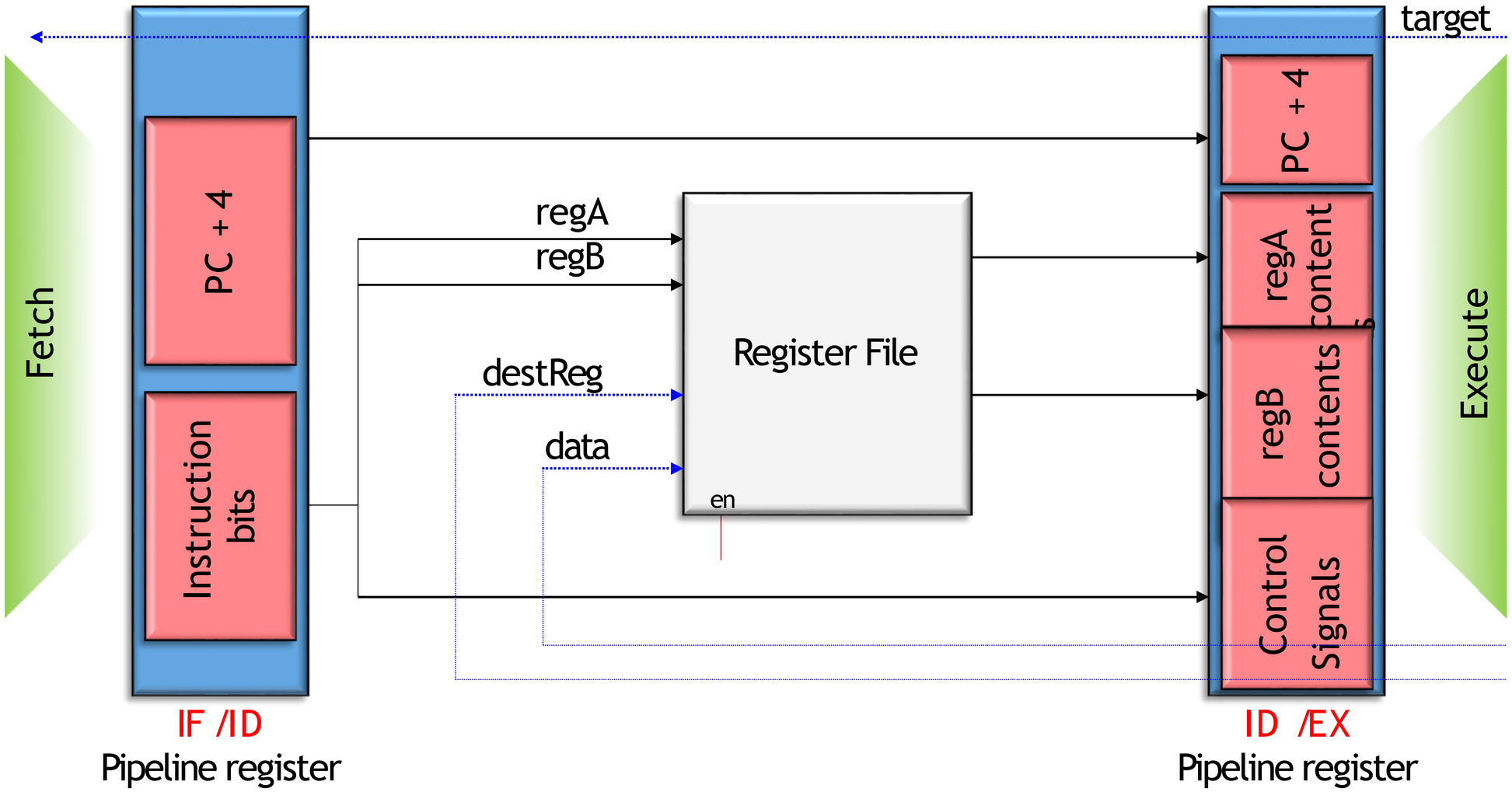
#Stages and Speedup

Assumptions	Unpipelined t_C	Pipelined t_C	Speedup
1. $t_{IM} = t_{DM} = 10$, $t_{ALU} = 5$, $t_{RF} = t_{RW} = 1$, 4-stage pipeline	27	10	2.7
2. $t_{IM} = t_{DM} = t_{ALU} = t_{RF} = t_{RW} = 5$, 4-stage pipeline	25	10	2.5
3. $t_{IM} = t_{DM} = t_{ALU} = t_{RF} = t_{RW} = 5$, 5-stage pipeline	25	5	5.0

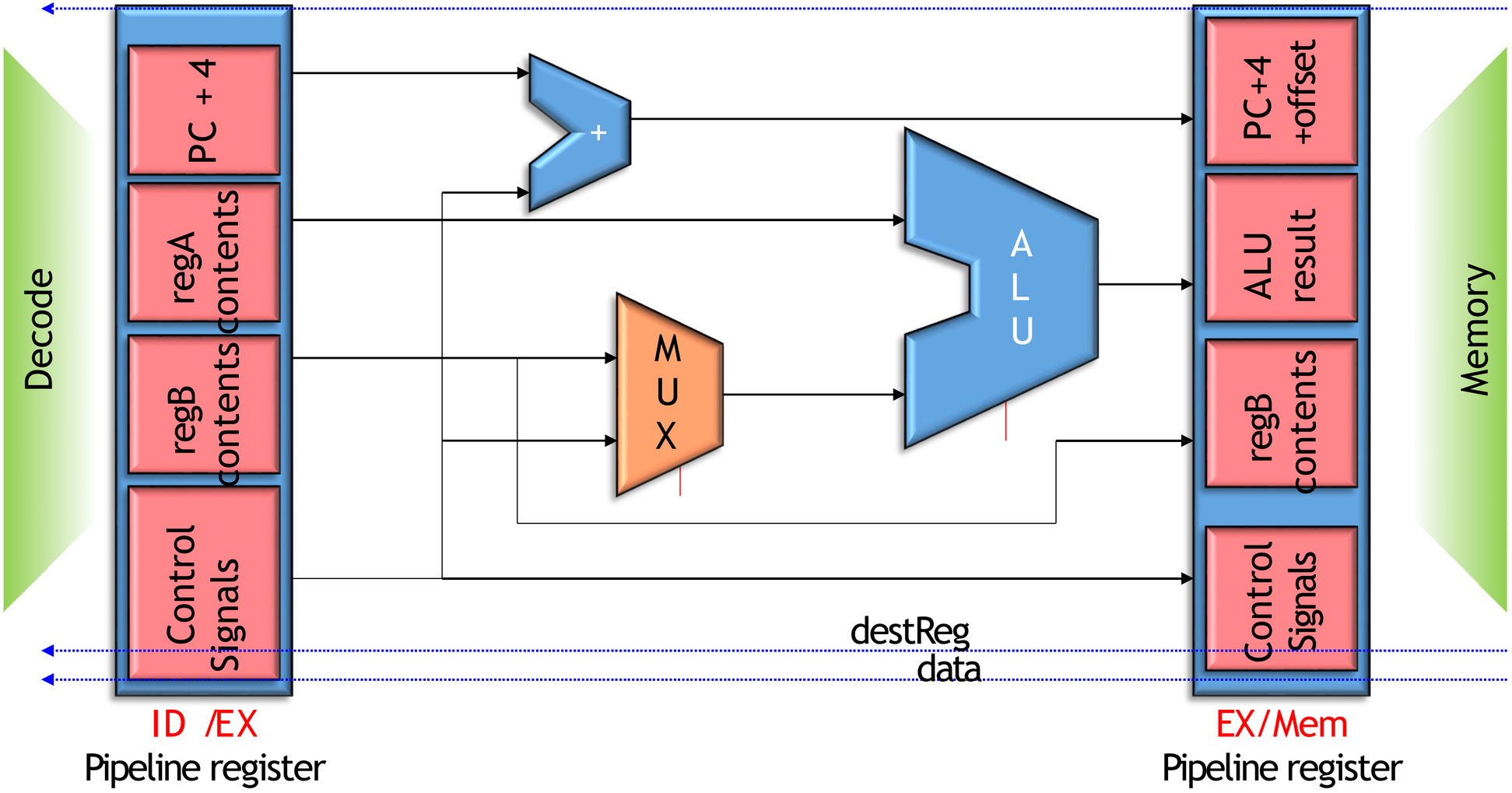
Stage-1: Fetch



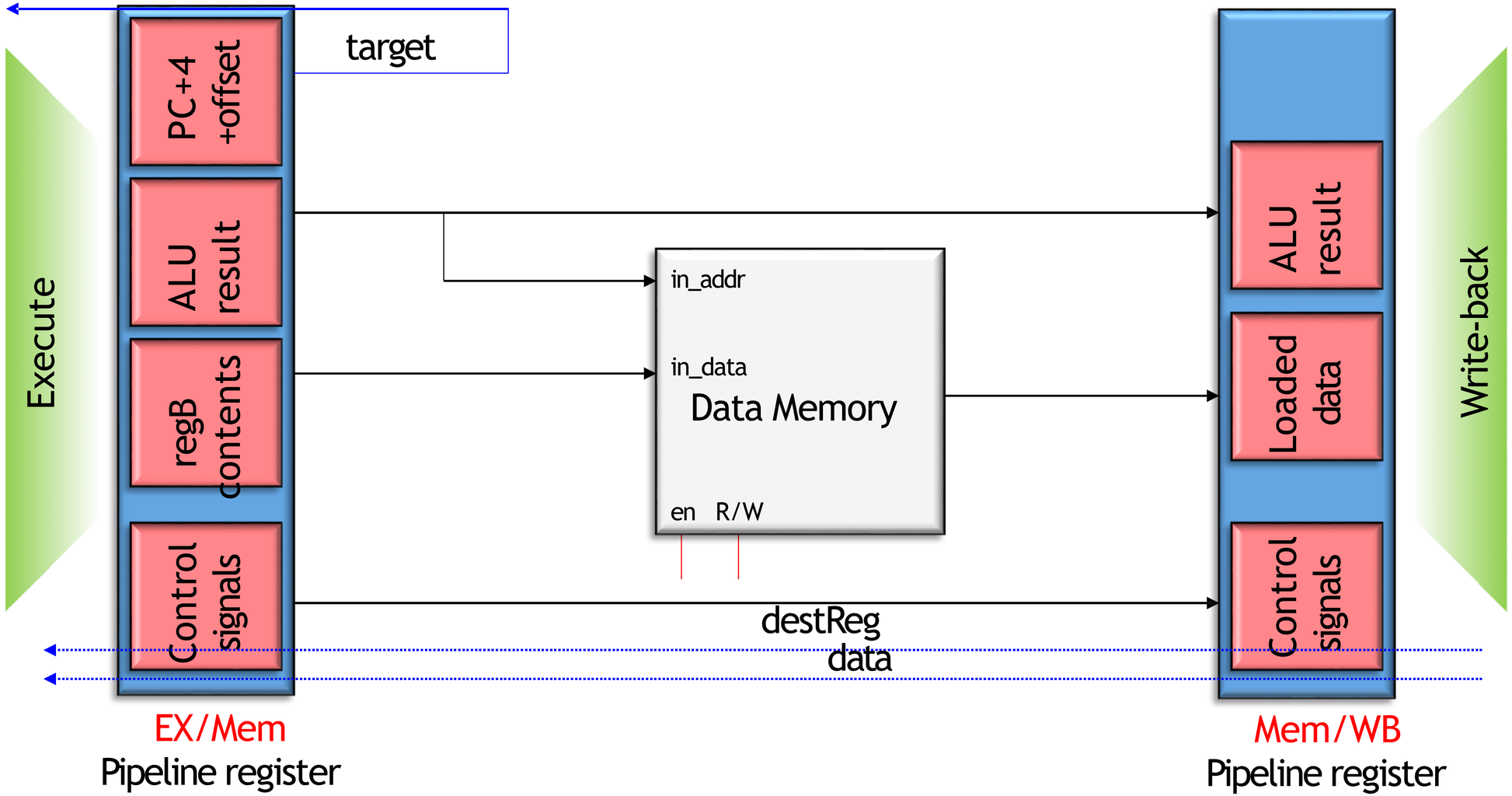
Stage 2: Decode



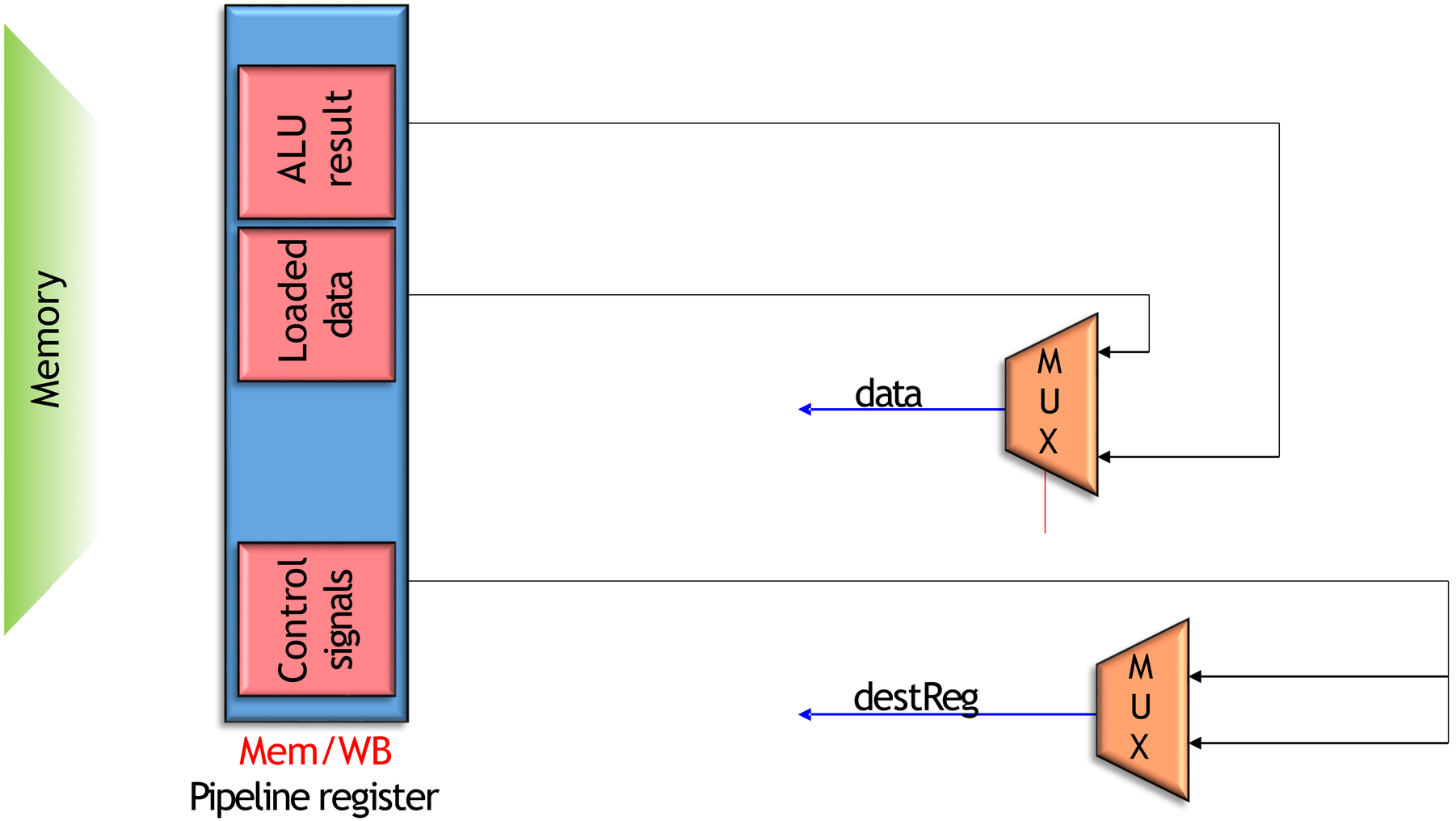
Stage 3: Execute



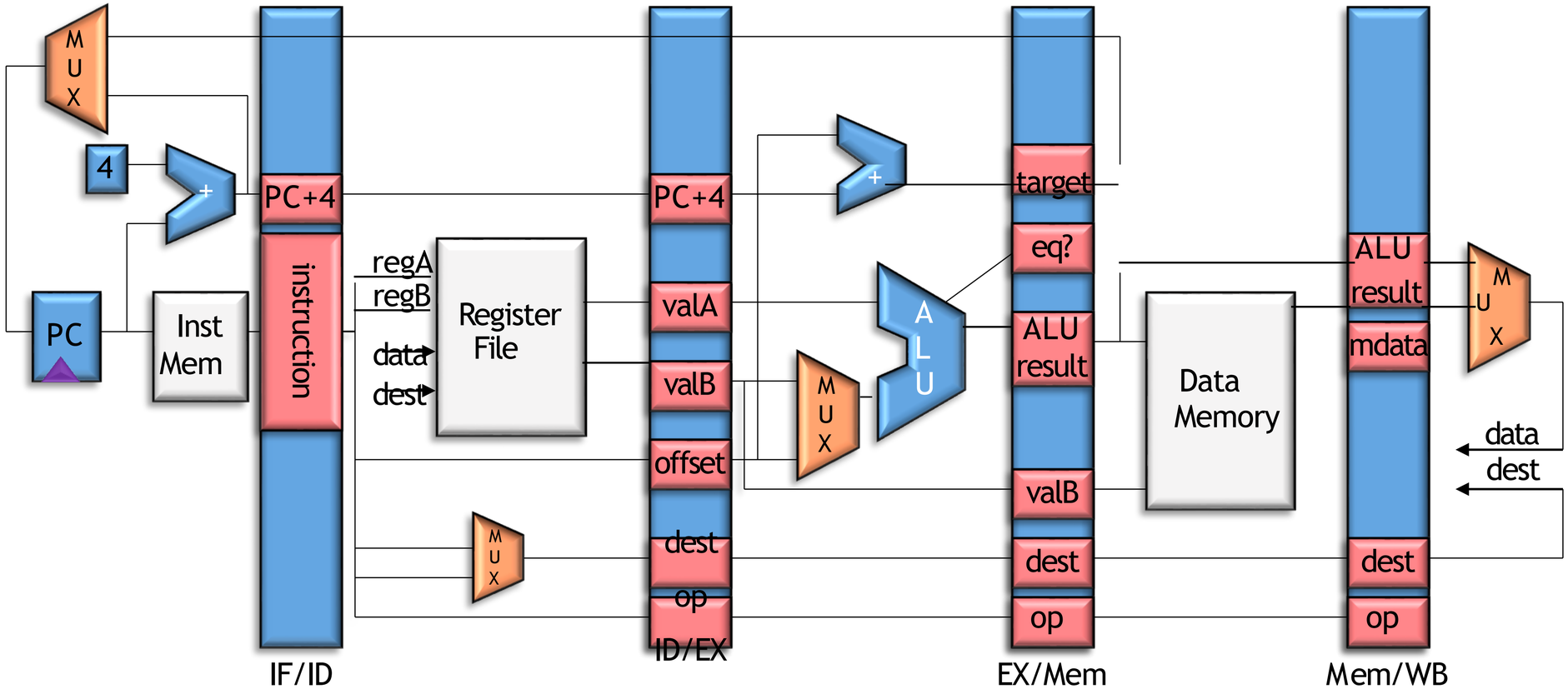
Stage 4: Memory Stage



Stage 5: Write-back



The Complete Picture



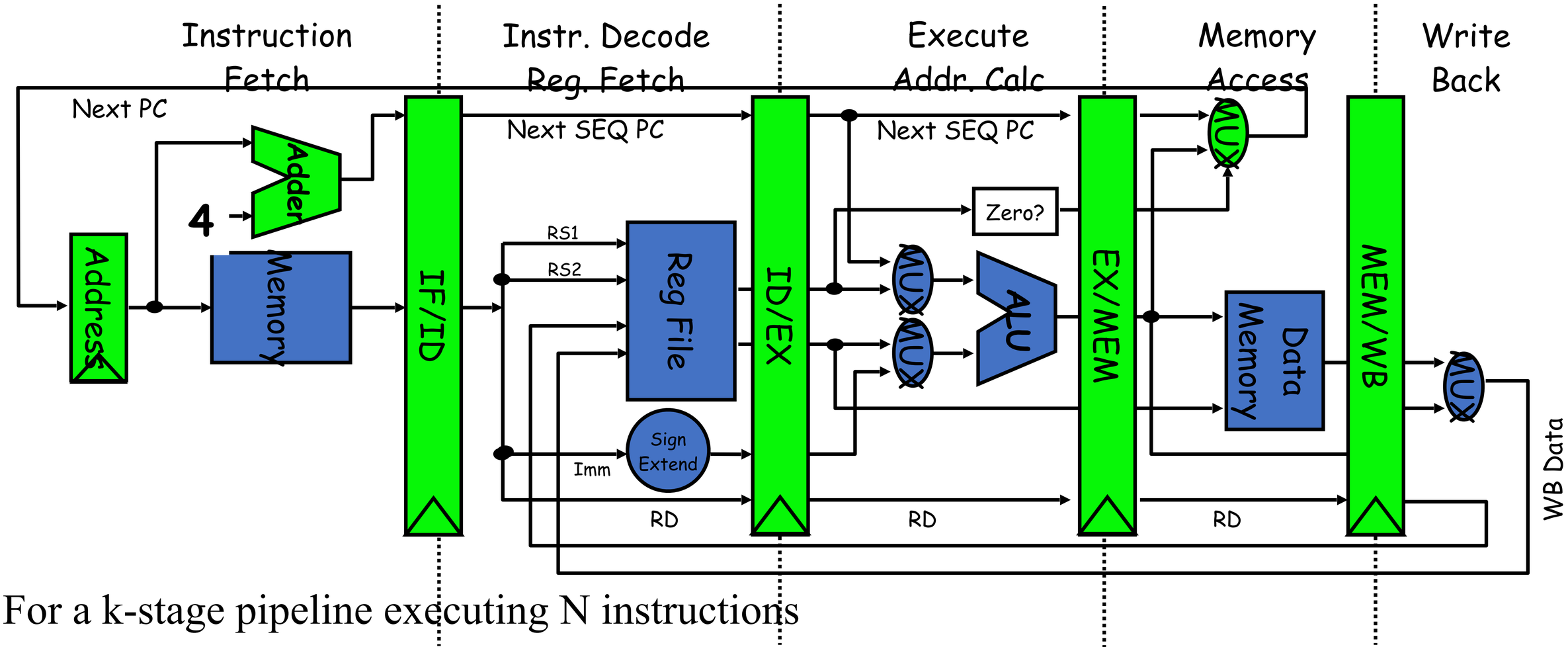
Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay

Pipeline Hazards

Pipeline Recap…



For a k-stage pipeline executing N instructions

first instruction: k cycles

Next N-1 instructions: N-1 cycles, total = K + (N-1) cycles

Pipeline In Real World

Limited resources

Stages does not take uniform time

Inter-instruction depenceny

Branches

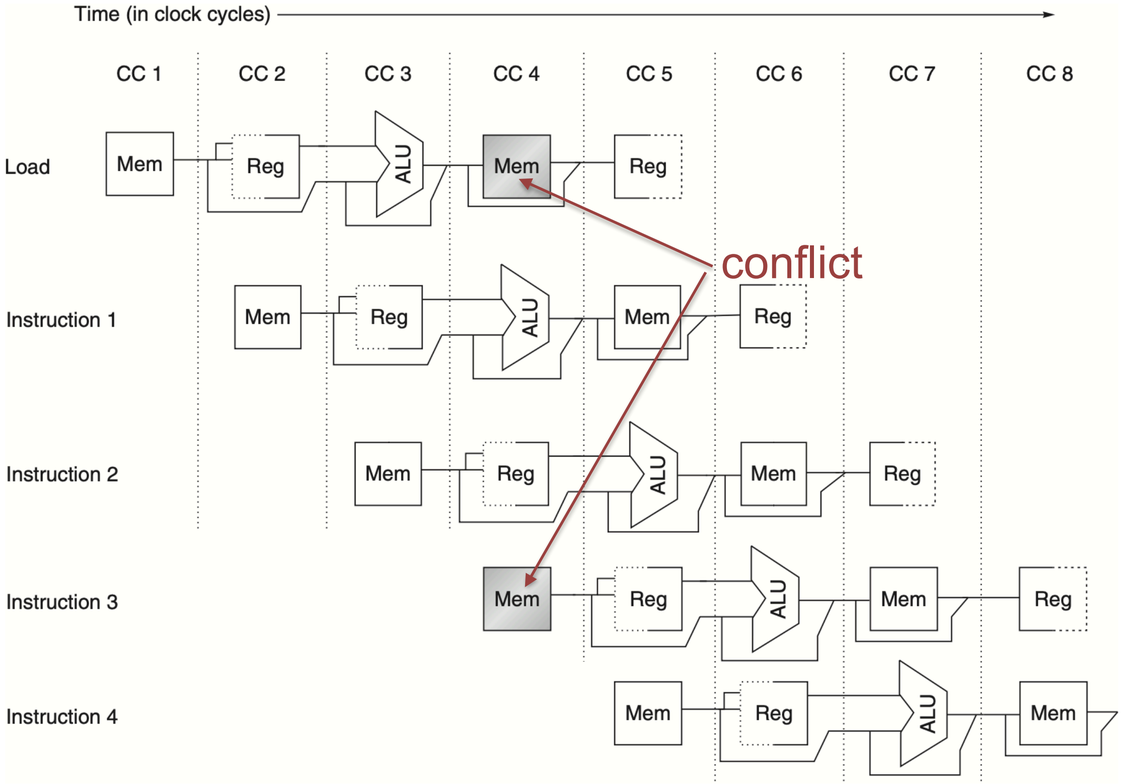
Limited resources

Pipeline Hazards

- Hazards are events in which prevents an instruction going down the pipeline. The pipeline is stalled
  - Structural hazards
  - Data hazards
  - Control hazards

Structural Hazards

- When two instructions wants to access the same resource at the same clock cycle.

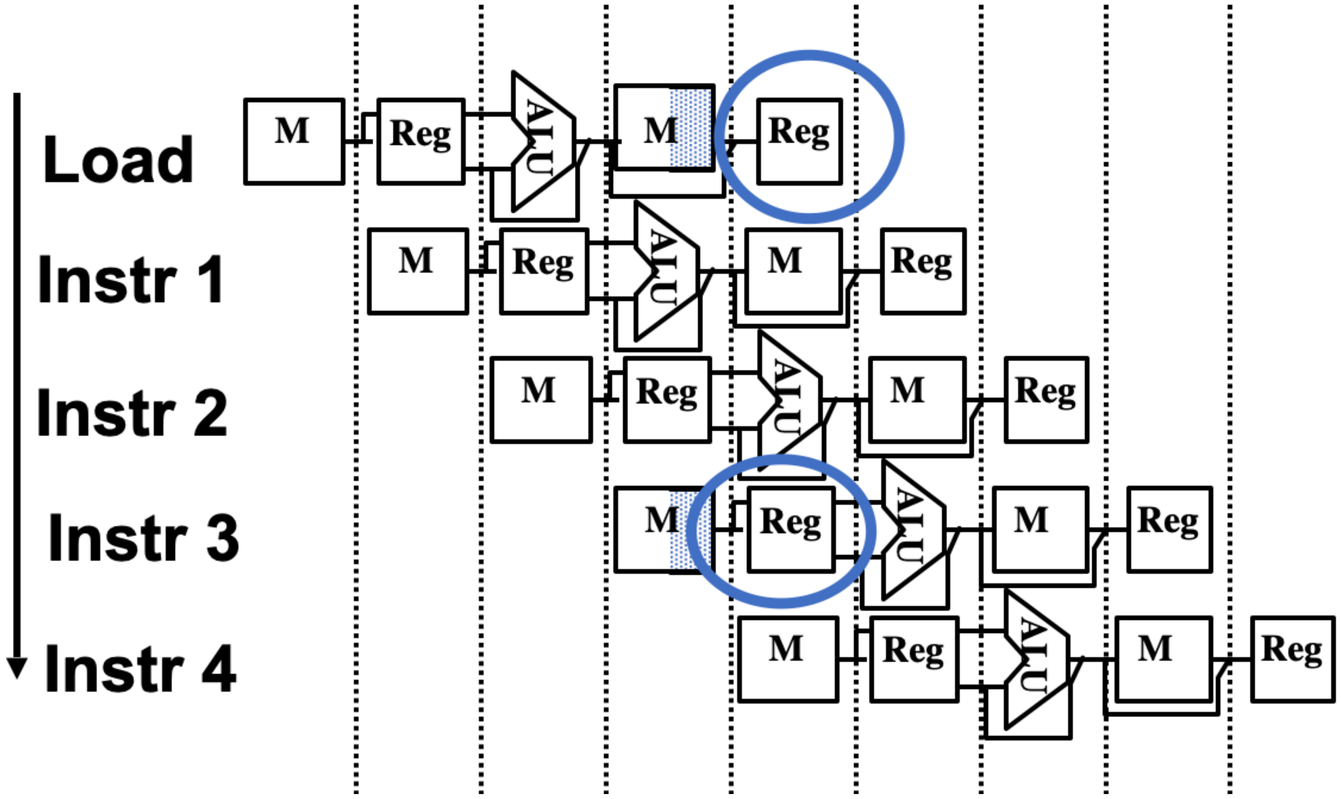


	Clock cycle number									
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Load instruction	IF	ID	EX	MEM	WB					
Instruction $i+1$		IF	ID	EX	MEM	WB				
Instruction $i+2$			IF	ID	EX	MEM	WB			
Instruction $i+3$				Stall	IF	ID	EX	MEM	WB	
Instruction $i+4$						IF	ID	EX	MEM	WB
Instruction $i+5$							IF	ID	EX	MEM
Instruction $i+6$								IF	ID	EX

- Issue: two instruction wants to access the memory simultaneously
  - One reading data
  - Other reading instruction.
- Solution: Separate instruction and data memory

Structural Hazards

- Can also happen for register files.



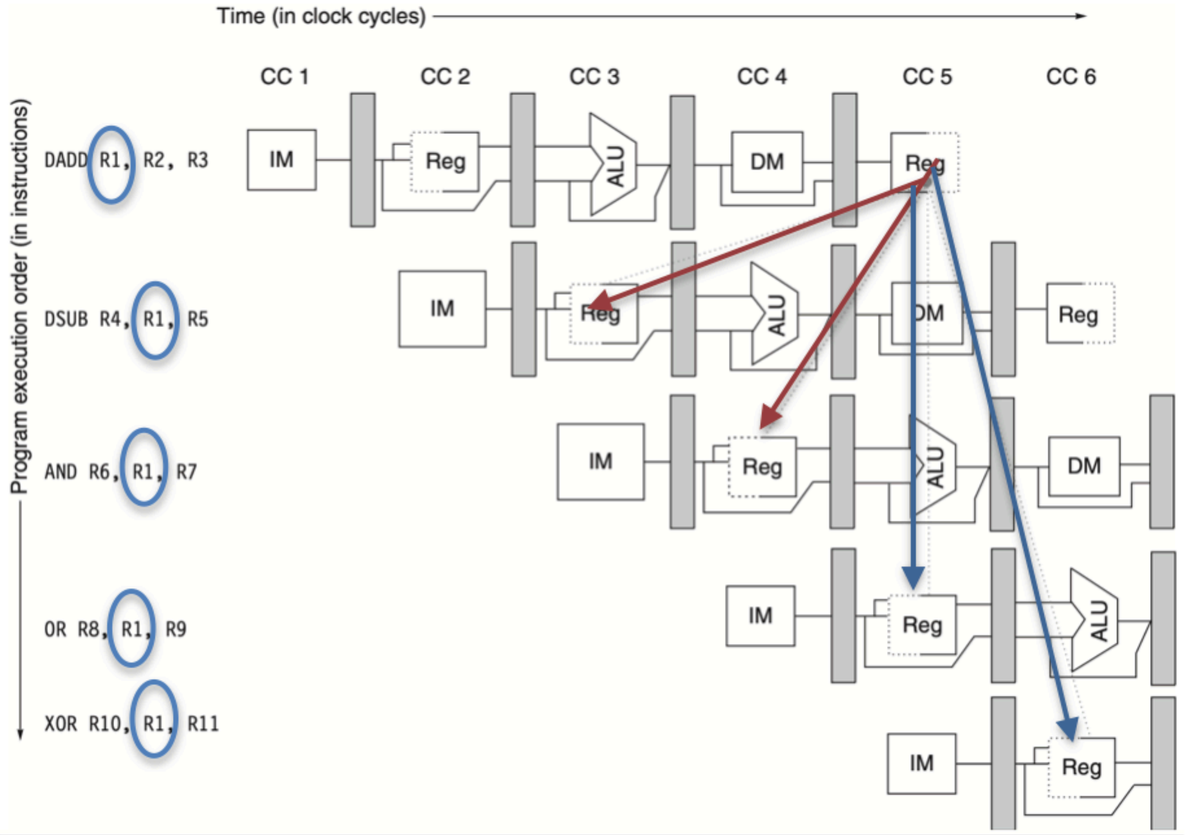
- Issue: conflict in ID and WB stage
  - Insufficient number of read/write ports
  - Read write in the same cycle, but no "write before read" convention.

  Solution:
  - Separate and multiple read write ports
  - Write in the first half of the clock cycle and read in the second half.

- Structural hazards are relatively rare in modern processors — compilers are smart.
- Only happens for less frequently used functional units

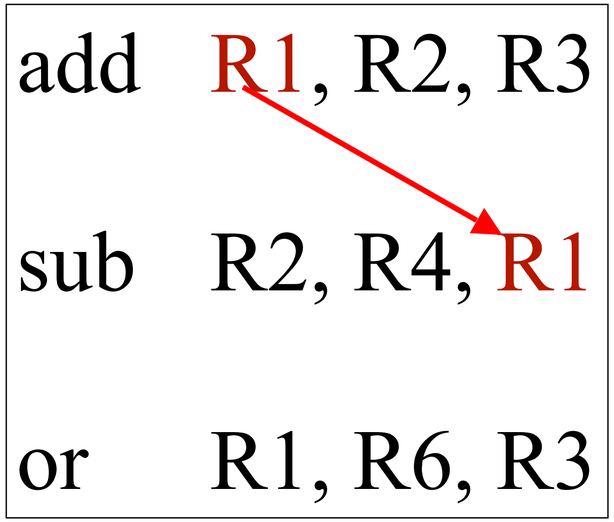
Data Hazards

- Hazards arising due to data dependency

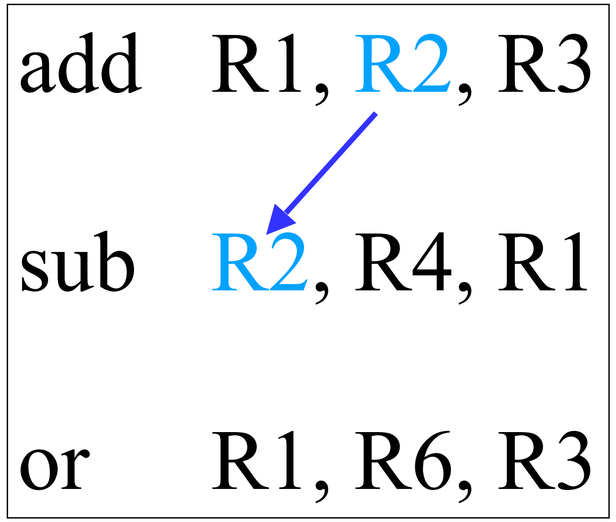


- The first DADD writes R1 at WB stage.
- All succeeding instructions reads the R1 result
- So, all instruction except the last OR and XOR has to wait for the WB of the first instruction.
- So we need two stall cycles or something else!!

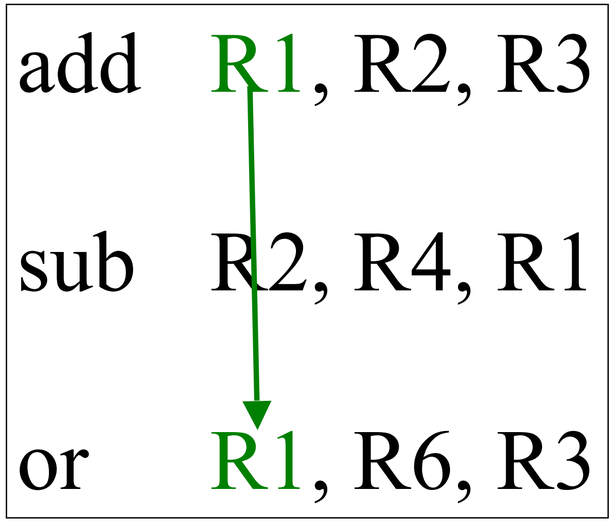
Data dependences



read-after-write
(RAW)
True dependence



write-after-read
(WAR)
Anti dependence



write-after-write
(WAW)
Output dependence

Data Hazards

Read-After-Write (RAW)

- Read must wait until earlier write finishes

Anti-Dependence (WAR)

- Write must wait until earlier read finishes. Not possible with vanilla 5-stage pipeline

- Output Dependence (WAW)
  - Earlier write can't overwrite later write

  Not possible with vanilla 5-stage pipeline)

Control Hazards

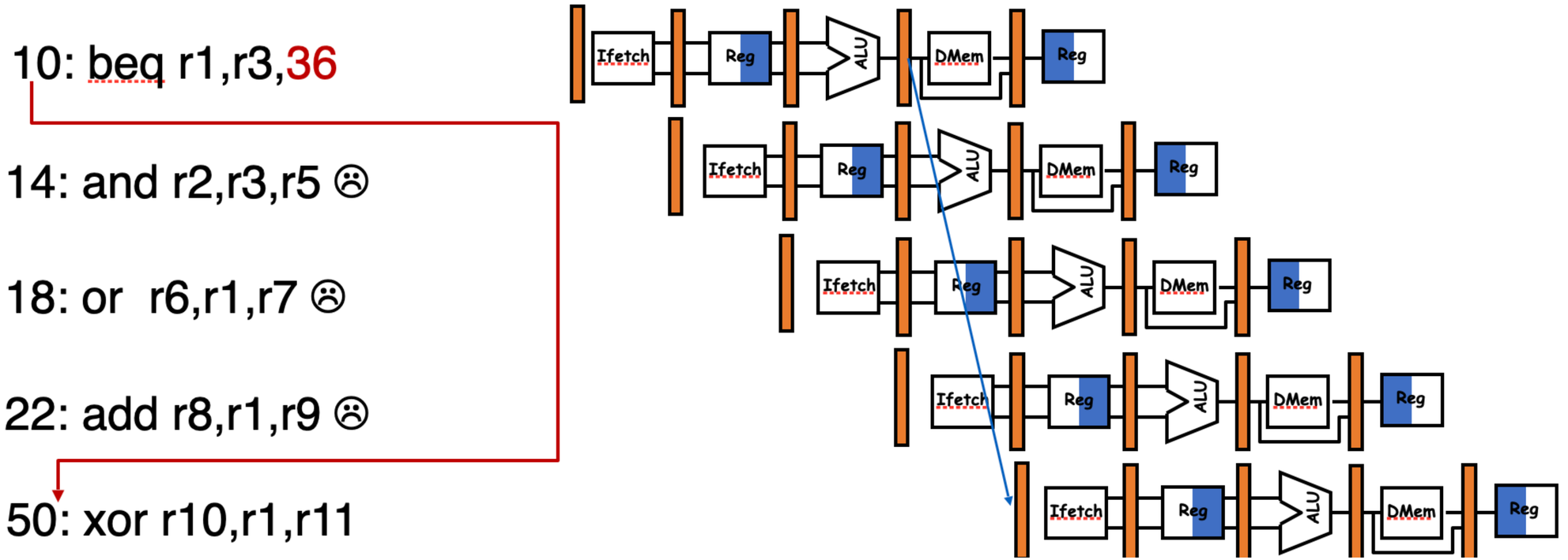
- Hazards arising due to branching…

- **Remember!!!** Branch target is not known during fetch.
- if a branch changes the PC to its target address, it is a *taken* branch.

- Else it is *untaken*.

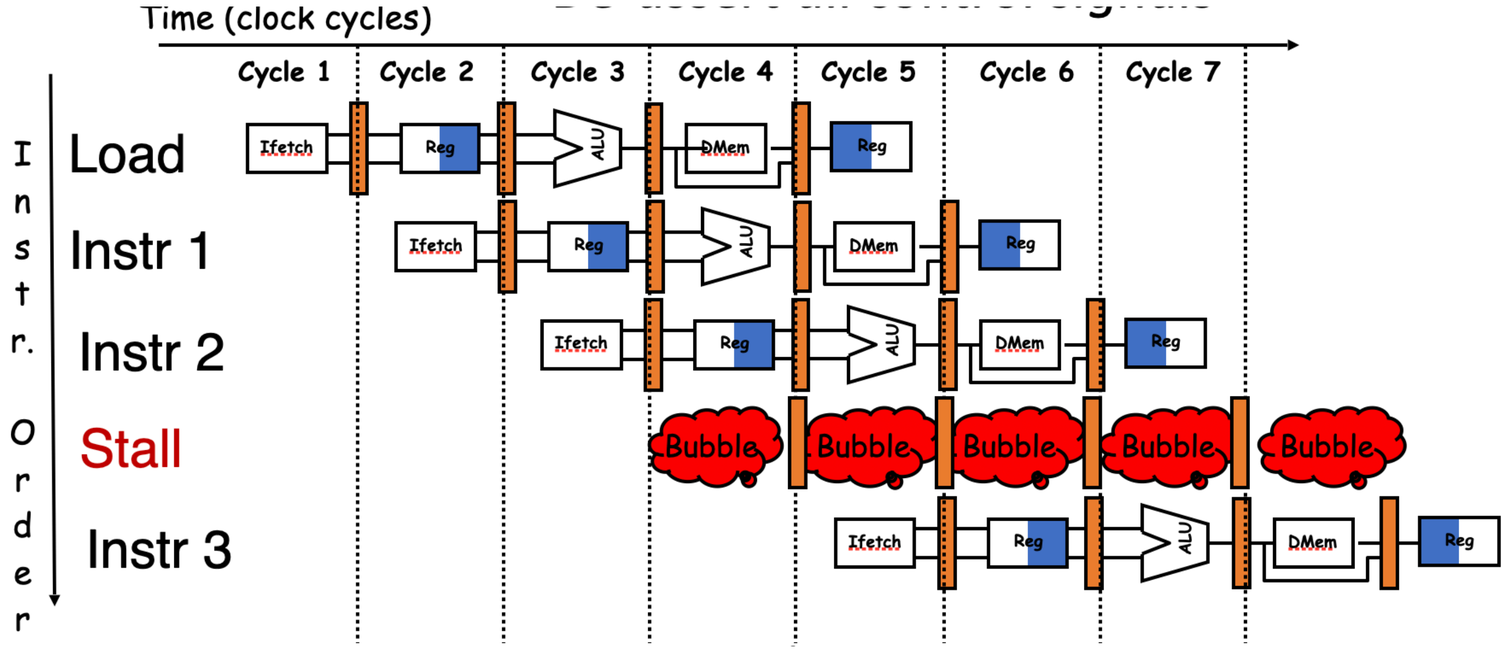
Control Hazards

- Hazards arising due to branching…
- What happens to the instructions at 14, 18, 22?



What is a Stall

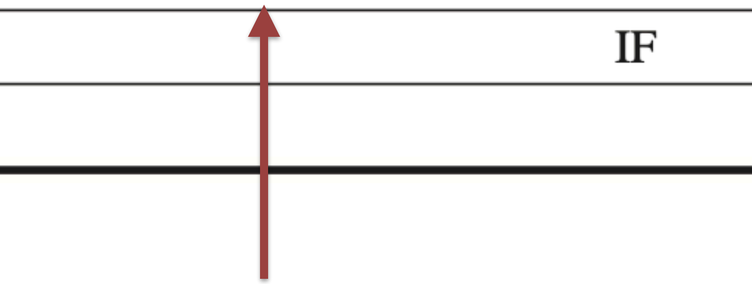
- Putting bubbles in pipeline.
- Actually wasting clock cycles — in a stall cycle no instruction can enter the pipeline
- De-assart all control signals
- Compiler way — put a `nop` instruction. — eg, `sll $0 $0 (in MIPS)`



Control Hazard and Stalls

- The earliest we can get to know the branch outcome is at the end of ID stage (needs some simple hardware modification)

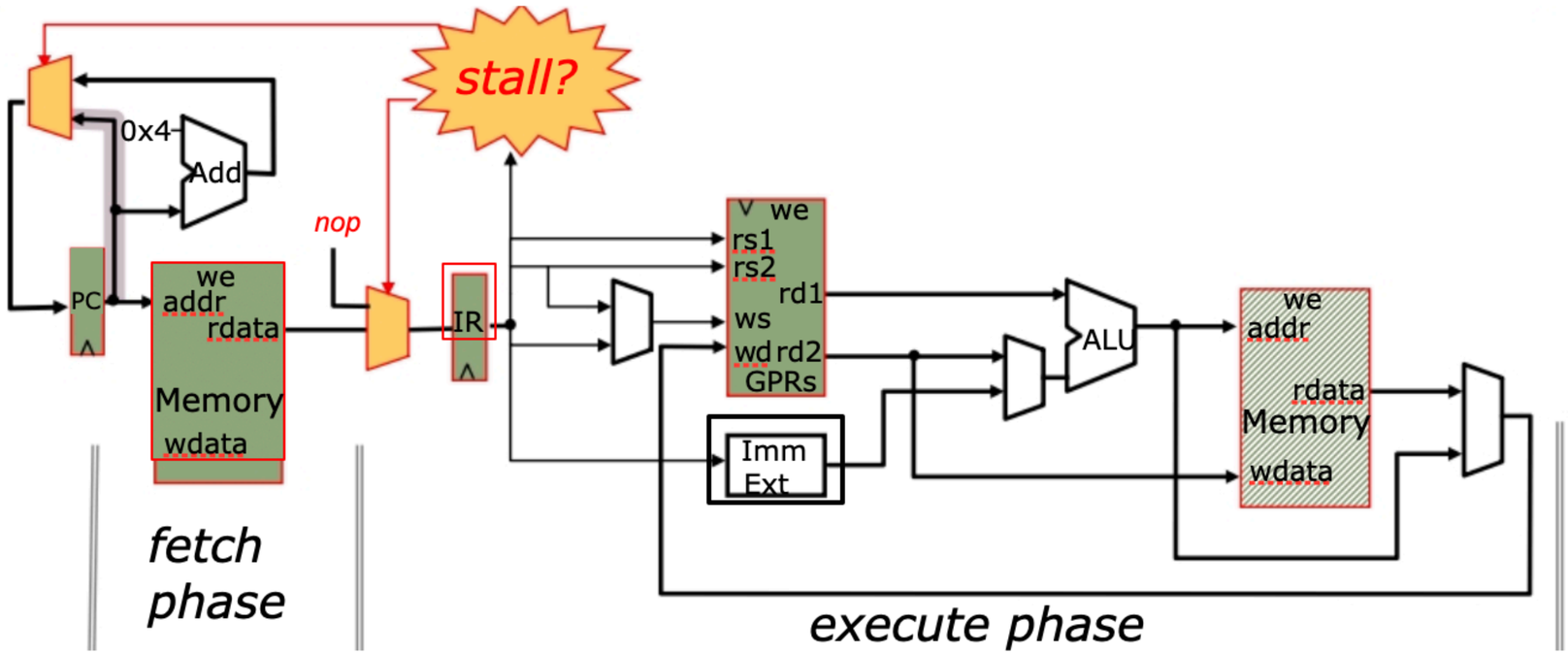
Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor + 1				IF	ID	EX	MEM
Branch successor + 2					IF	ID	EX



This IF has to be undone — results in a stall cycle

How Stall is implemented in Pipelines

- Can be detected from the content of the pipeline registers.
- Upon detecting stall, just do not update the PC and dessert all control signals



PC: Biswa

Data Hazard Detector and stalls

EX to DEC:

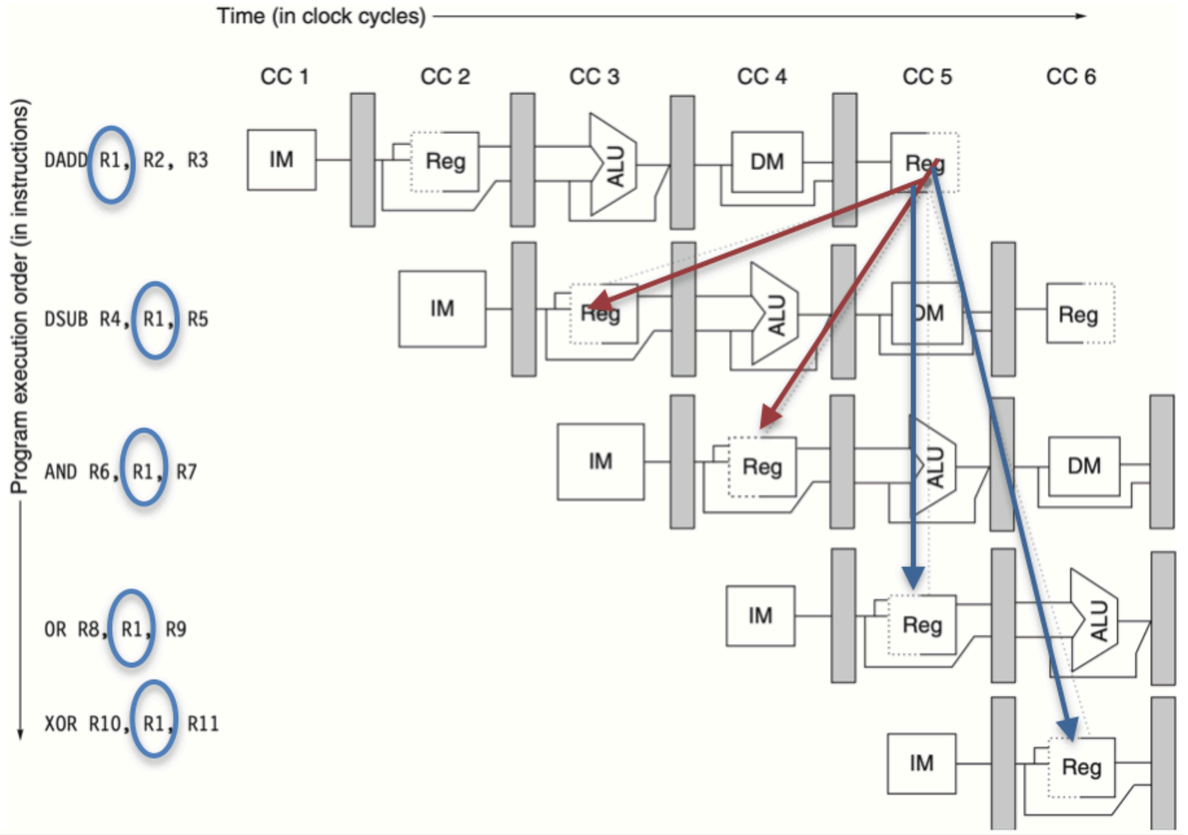
EX/MEM.RegisterRd = ID/EX.RegisterRs
EX/MEM.RegisterRd = ID/EX.RegisterRt

MEM to DEC:

MEM/WB.RegisterRd = ID/EX.RegisterRs
MEM/WB.RegisterRd = ID/EX.RegisterRt

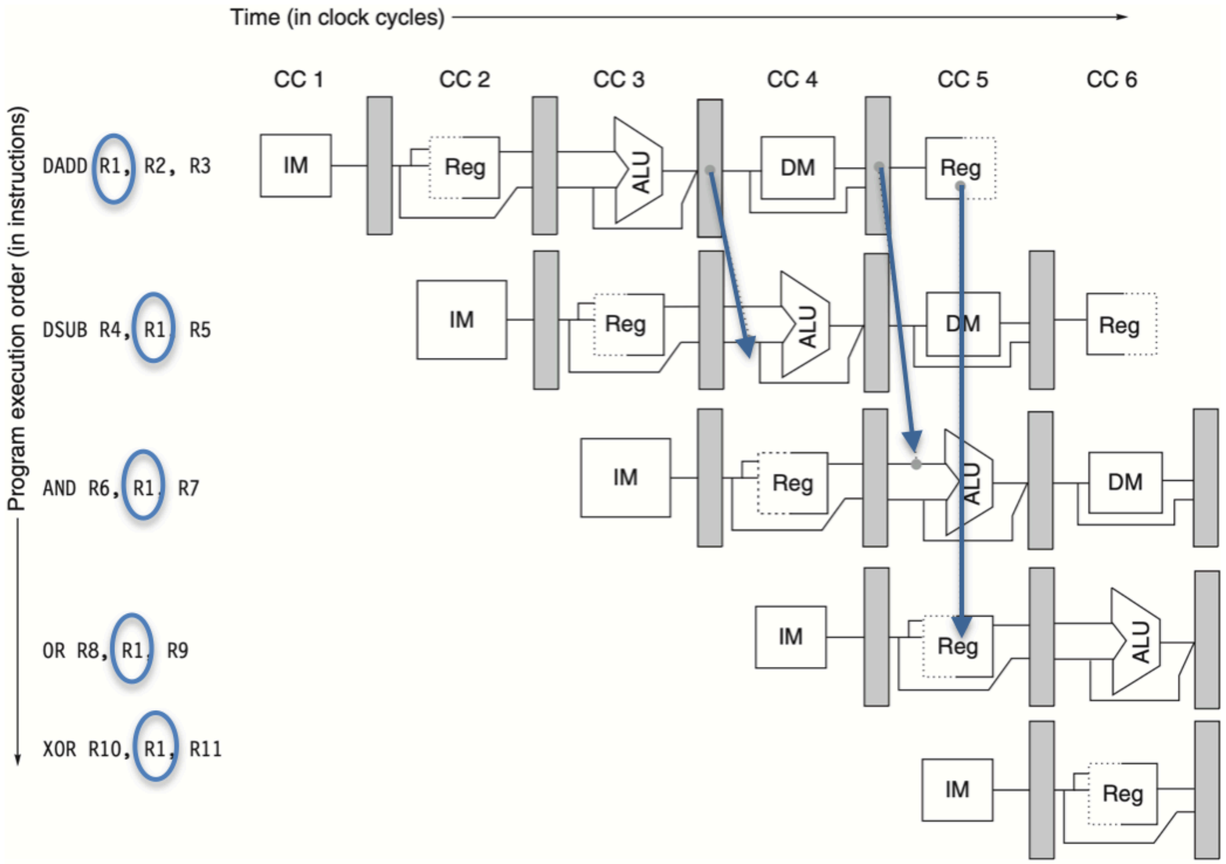
Data Hazards

- Hazards arising due to data dependency



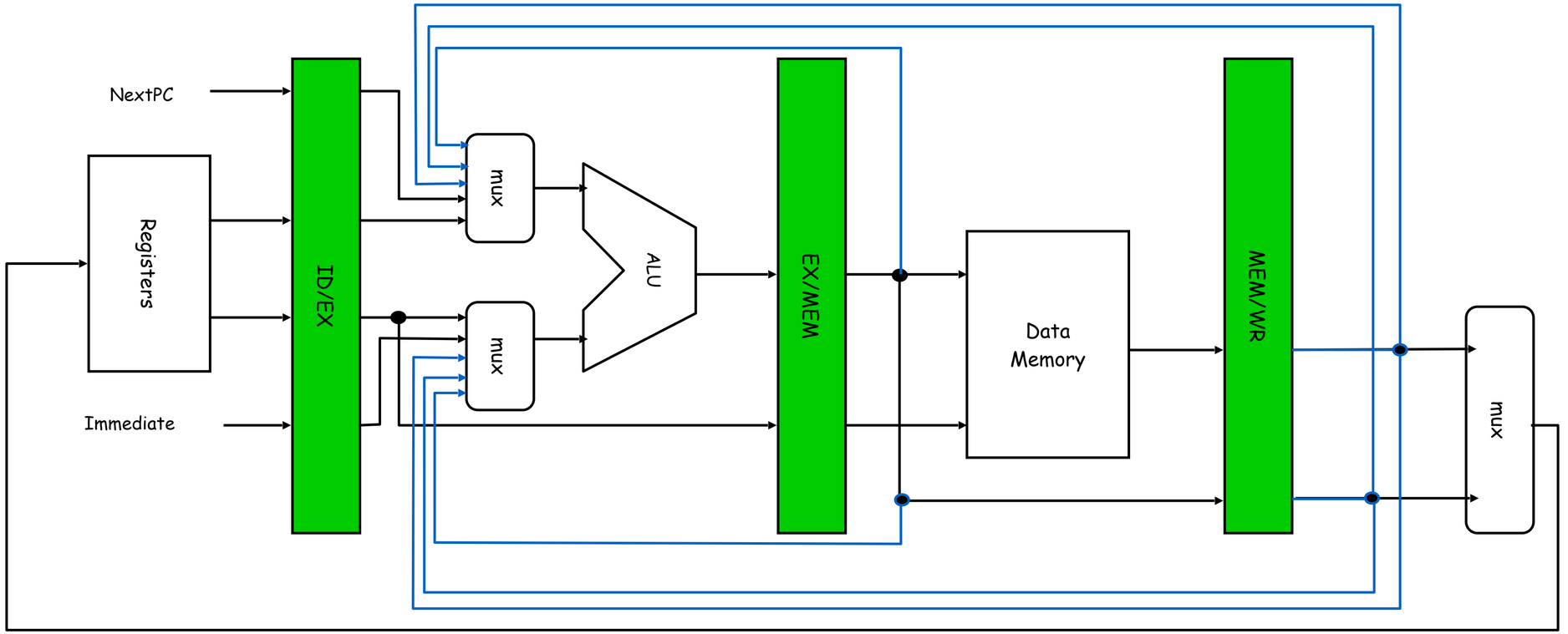
- The first DADD writes R1 at WB stage.
- All succeeding instructions reads the R1 result
- So, all instruction except the last OR and XOR has to wait for the WB of the first instruction.
- So we need two stall cycles or something else!!

Handling Data Hazards: Forwarding



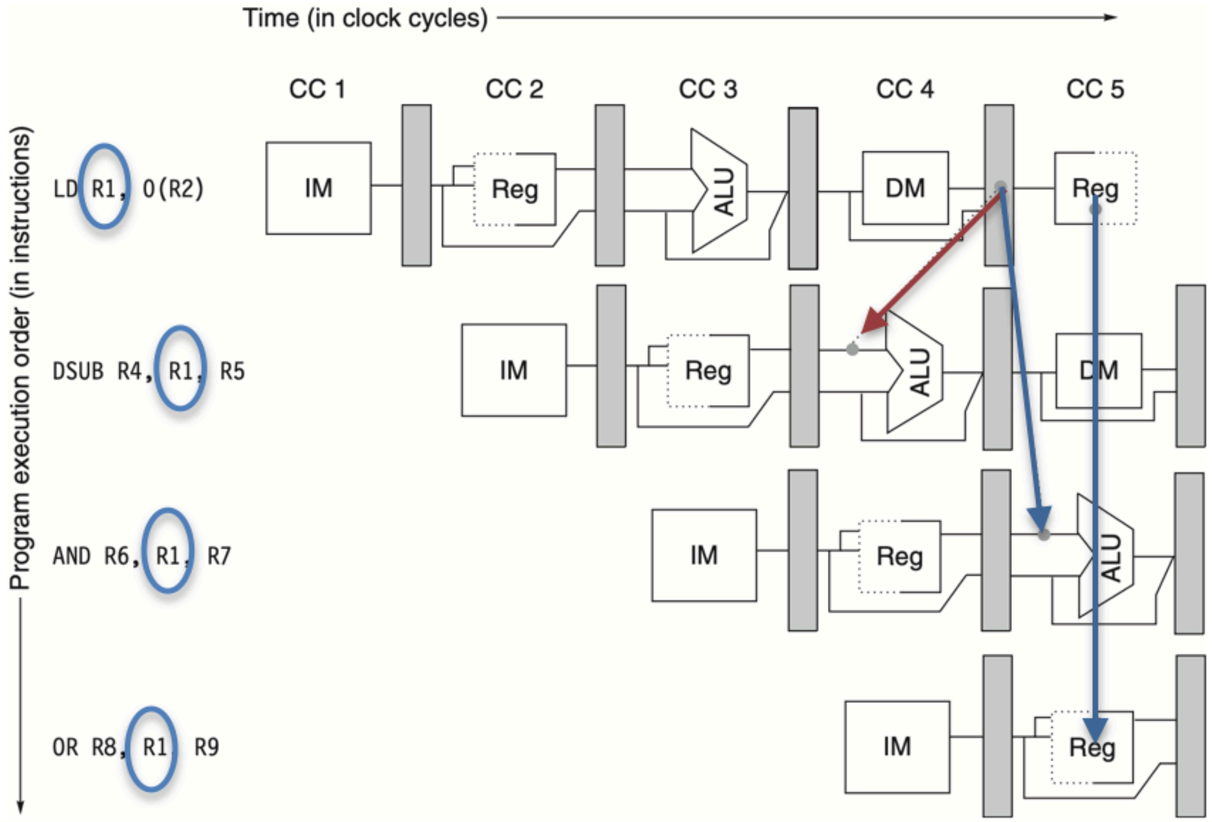
- No stalls!!
- Can be generalised — any functional unit generating data can forward to any other input whenever needed.

Handling Data Hazards: Forwarding



PC: Biswa

Can Forwarding Solve All the Problems?



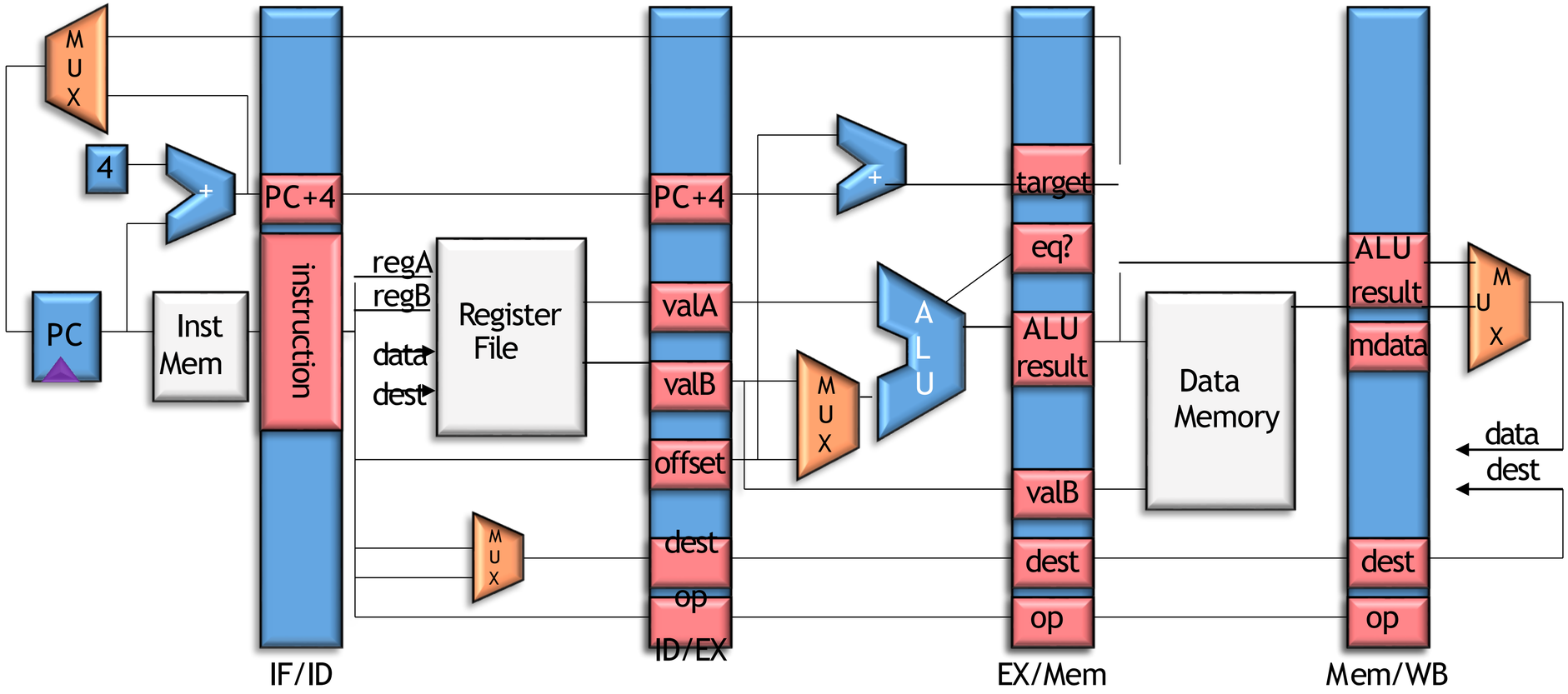
- No problems for the AND and OR — forwarding works fine
- But we cannot forward for the DSUB as it is backward in time.
- So one cycle stall is needed.

What Happens to Speedup with Stalls

$$Speedup = \frac{CPI\ Unpipelined}{CPI\ Ideal + stall\ cycles}$$

- CPI Ideal = 1
- Also assume stages are perfectly balanced so that if the unpipelined cycle time is T, the pipelined cycle time becomes T/k for a k stage pipeline. — so easy to cancel out T

The Complete Picture



Control Hazards

What do we need to calculate—next PC?

- For Jumps
  - Opcode, offset, and PC
- For Jump Register
  - Opcode and register value
- For Conditional Branches
  - Opcode, offset, PC, and register (for condition)
- For all others

Control Hazards

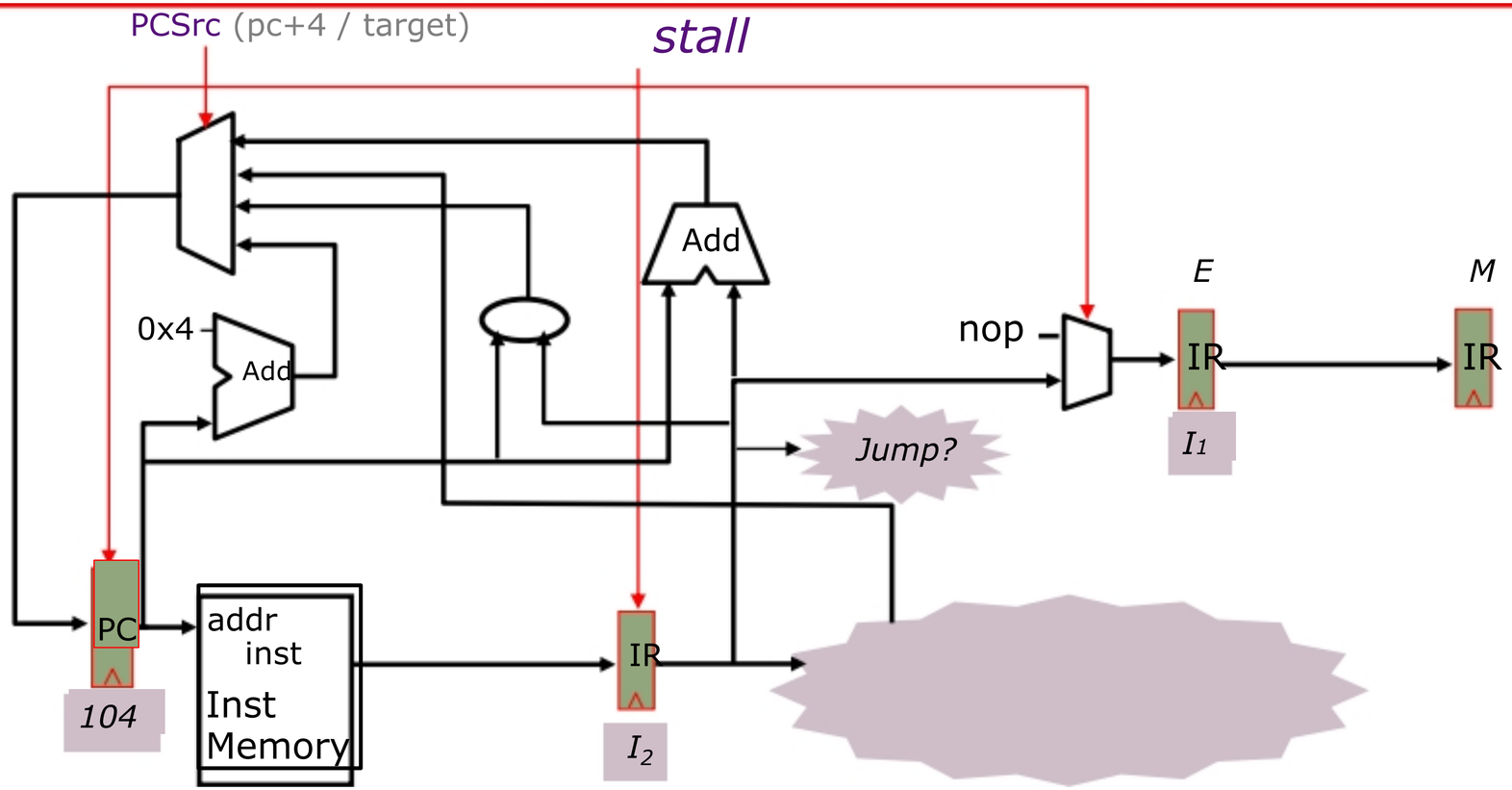
What do we need to calculate next PC?

- For Jumps
  - Opcode, offset, and PC
- For Jump Register
  - Opcode and register value
- For Conditional Branches
  - Opcode, offset, PC, and register (for condition)

In what stage do we know these?

- PC - Fetch
- Opcode, offset - Decode (or Fetch?)
- Register value - Decode
- Branch condition ((rs)==0) - Execute (or Decode?)

Speculate, PC=PC+4



I_1	096	ADD
I_2	100	J 304
I_3	~~104~~	~~ADD~~
I_4	304	ADD

What happens on mis-speculation, i.e., when next instruction is not PC+4?

kill *How? Insert NOPs*

Conditional branches

I_1	096	ADD
I_2	100	BEQZ r1 200
I_3	104	ADD
I_4	304	ADD

Branch condition is not known until the execute stage

Instructions between a branch instruction and the target are in the wrong-path if the branch is not taken

Again (stalls/NOPs)

	time t0	t1	t2	t3	t4	t5	t6	t7	. . . .
(I_1) 096: ADD	IF_1	ID_1	EX_1	MA_1	WB_1				
(I_2) 100: BEQZ 200		IF_2	ID_2	EX_2	MA_2	WB_2			
(I_3) 104: ADD			IF_3	ID_3	nop	nop	nop		
(I_4) 108:				IF_4	nop	nop	nop	nop	
(I_5) 304: ADD					IF_5	ID_5	EX_5	MA_5	WB_5

Resource Usage

	time t0	t1	t2	t3	t4	t5	t6	t7	. . . .
IF	I_1	I_2	I_3	I_4	I_5				
ID		I_1	I_2	I_3	nop	I_5			
EX			I_1	I_2	nop	nop	I_5		
MA				I_1	I_2	nop	nop	I_5	
WB					I_1	I_2	nop	nop	I_5

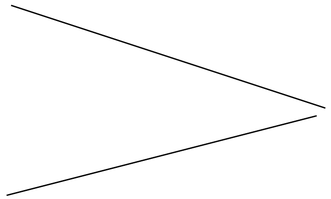
What else can be done? Compiler?

Delayed branch: Define branch to take place AFTER a following instruction (used to be in early RISC processors)

branch instruction
sequential successor$_1$
sequential successor$_2$

.....
sequential successor$_n$

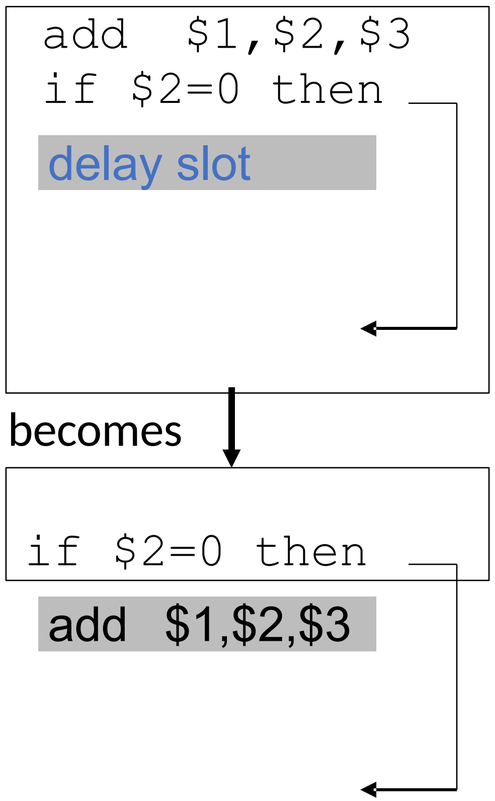
branch target if taken



Branch delay of length *n*

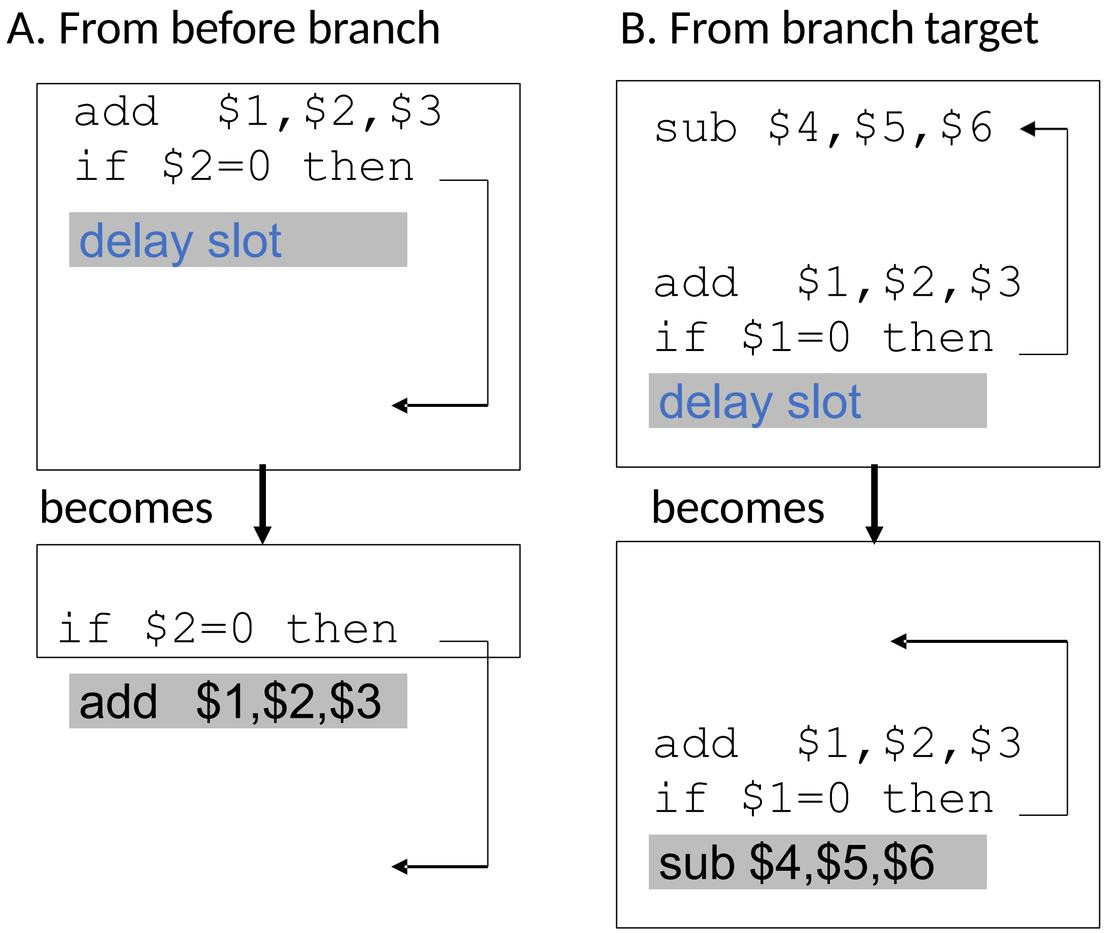
Scheduling Branch Delay Slots

A. From before branch



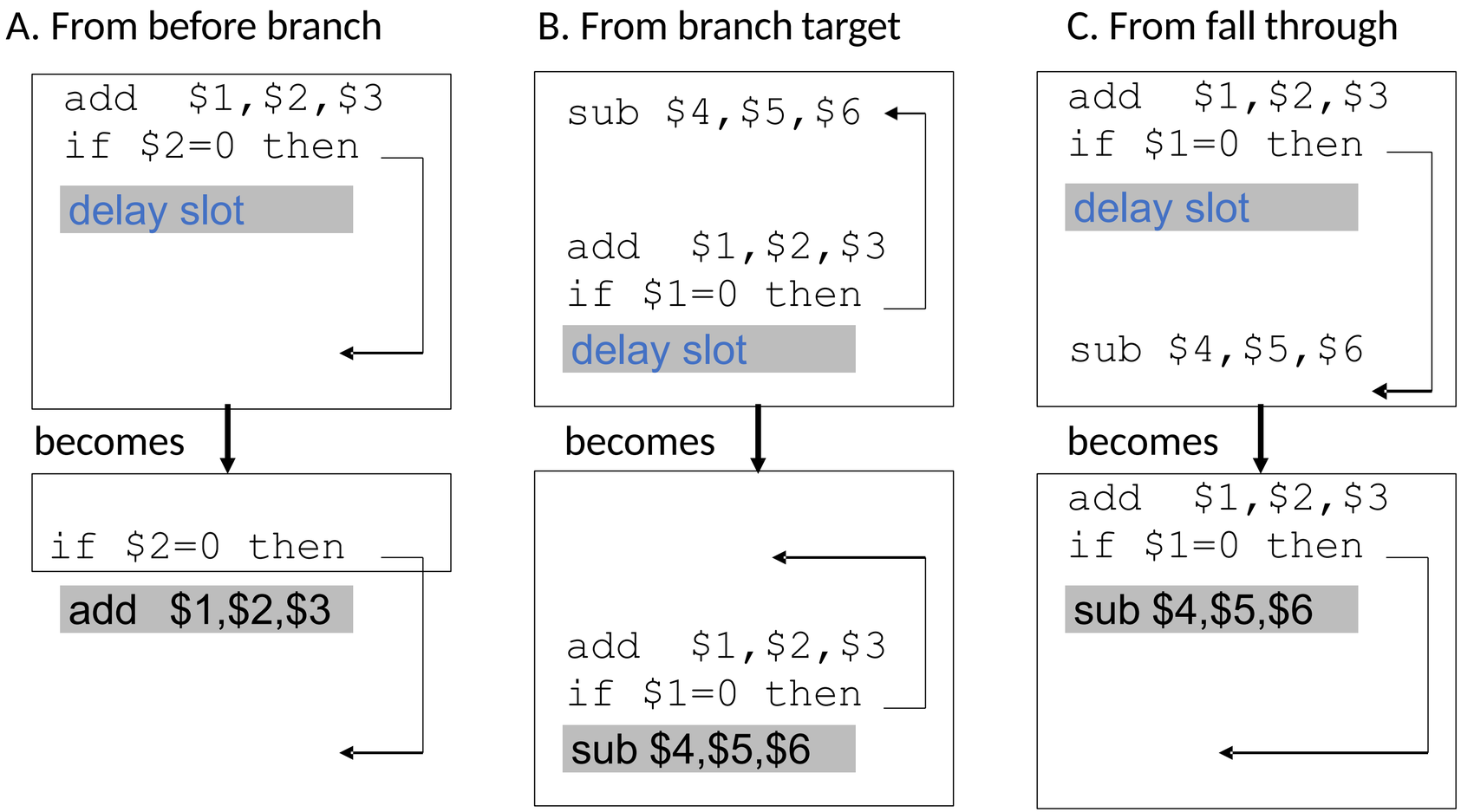
A is the best choice, fills delay slot & reduces instruction count (IC)

Scheduling Branch Delay Slots



A is the best choice, fills delay slot & reduces instruction count (IC)

Scheduling Branch Delay Slots



A is the best choice

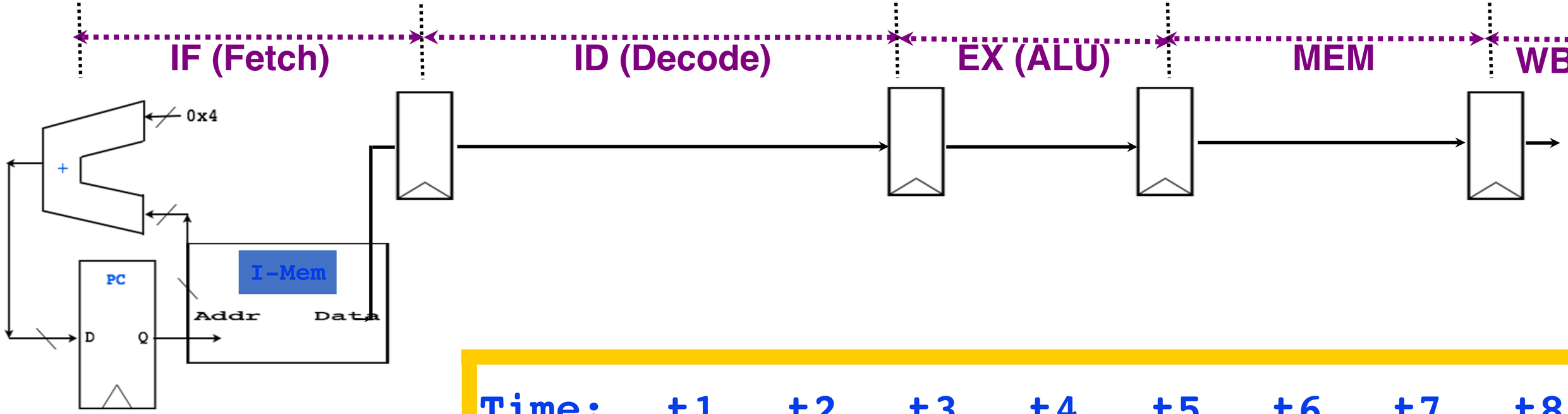
Do not put a branch in the delay slot :P

New Pipeline Speedup

$$\text{Pipeline Speedup} = \frac{\text{Pipeline Depth}}{1+\text{pipeline stalls because of branches}}$$

Pipeline stalls (branches) = Branch frequency X penalty

Branch instructions

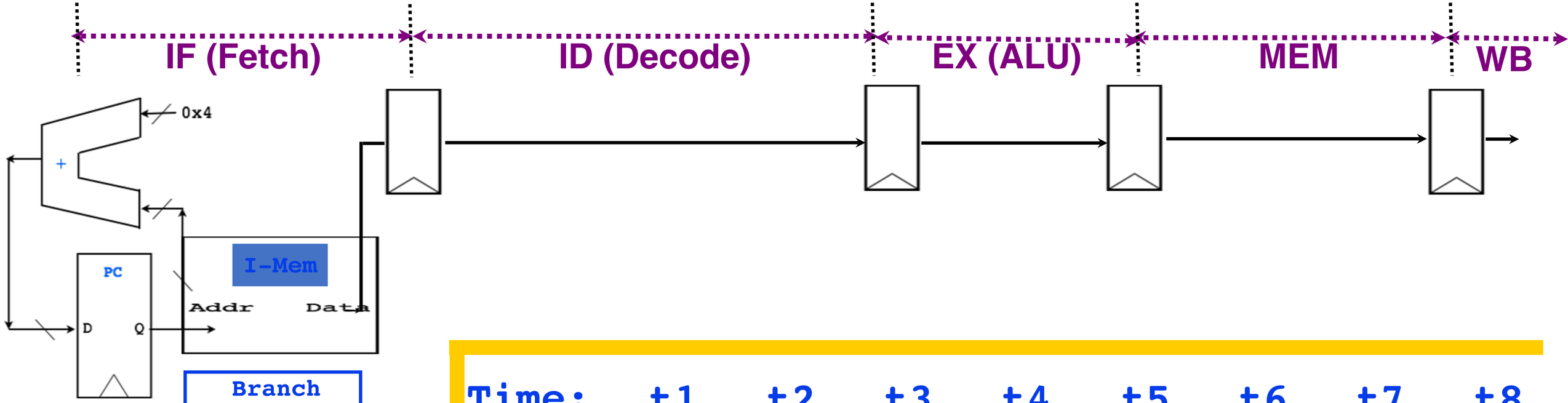


Time:	t1	t2	t3	t4	t5	t6	t7	t8
Inst								
I1:	IF	ID	EX	MEM	WB			
I2:		IF	ID					
I3:			IF					
I4:								
I5:								
I6:								

EX stage computes if branch is taken

If branch is taken, these instructions MUST NOT complete!

Branch Predictors



Branch Predictor

Predictions

A control instr?

Taken or Not Taken?

If taken, where to? What PC?

```
Time:   t1   t2   t3   t4   t5   t6   t7   t8
Inst
 I1:    IF   ID   EX   MEM  WB
 I2:         IF   ID
 I3:              IF
 I4:
 I5:
 I6:
```

EX stage computes if branch is taken

If branch is taken, these instructions MUST NOT complete!

Branch Predictors

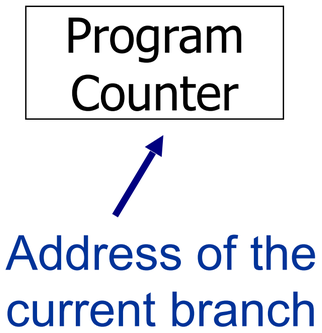
- Predict whether the next PC is a branch PC in the fetch stage
- But:
  - If it is branch, will it be taken?
  - What is the target address?
    - Not known at fetch…..

Branch Predictor: A bit deeper

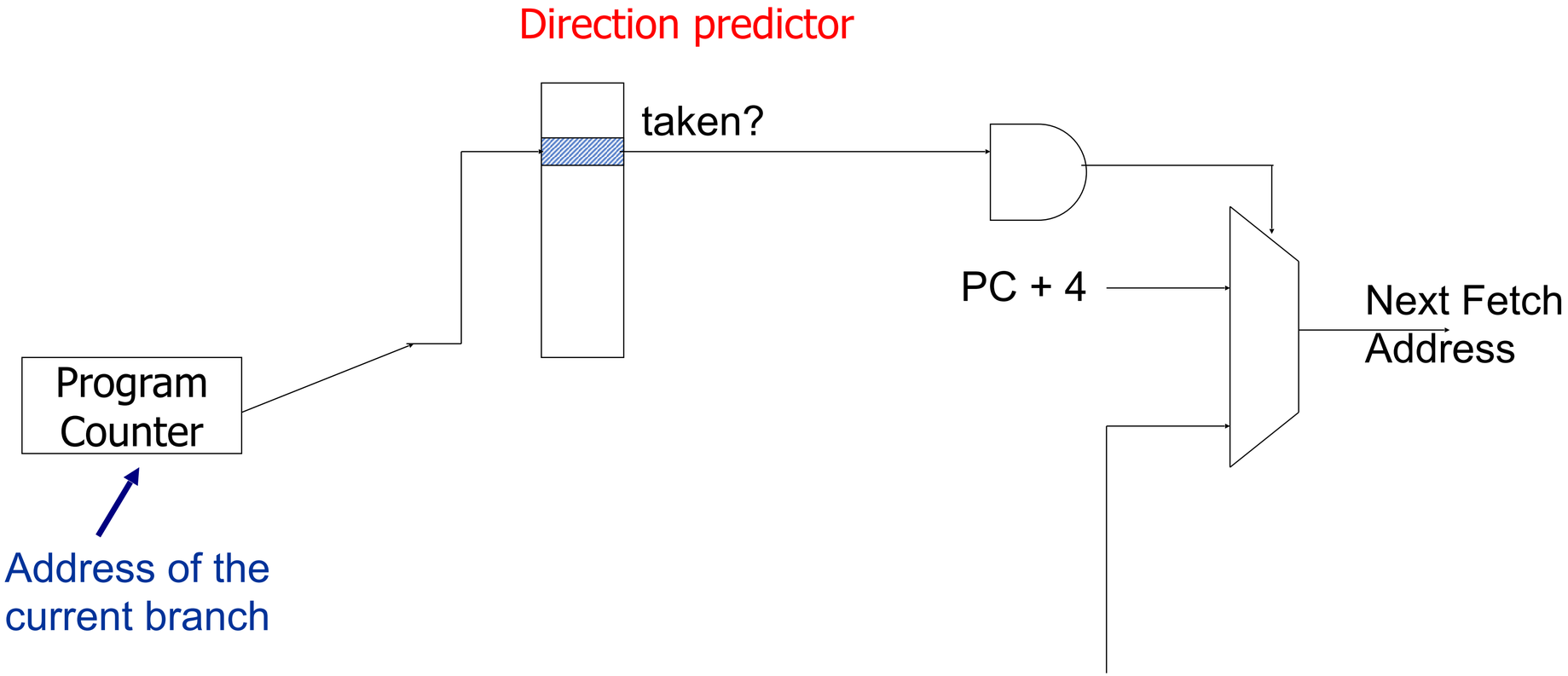
Three tasks

1. Is the PC a branch/jump? YES/NO
2. If Yes, can we predict the direction? Taken or not-taken
3. If taken, can we predict the target address?

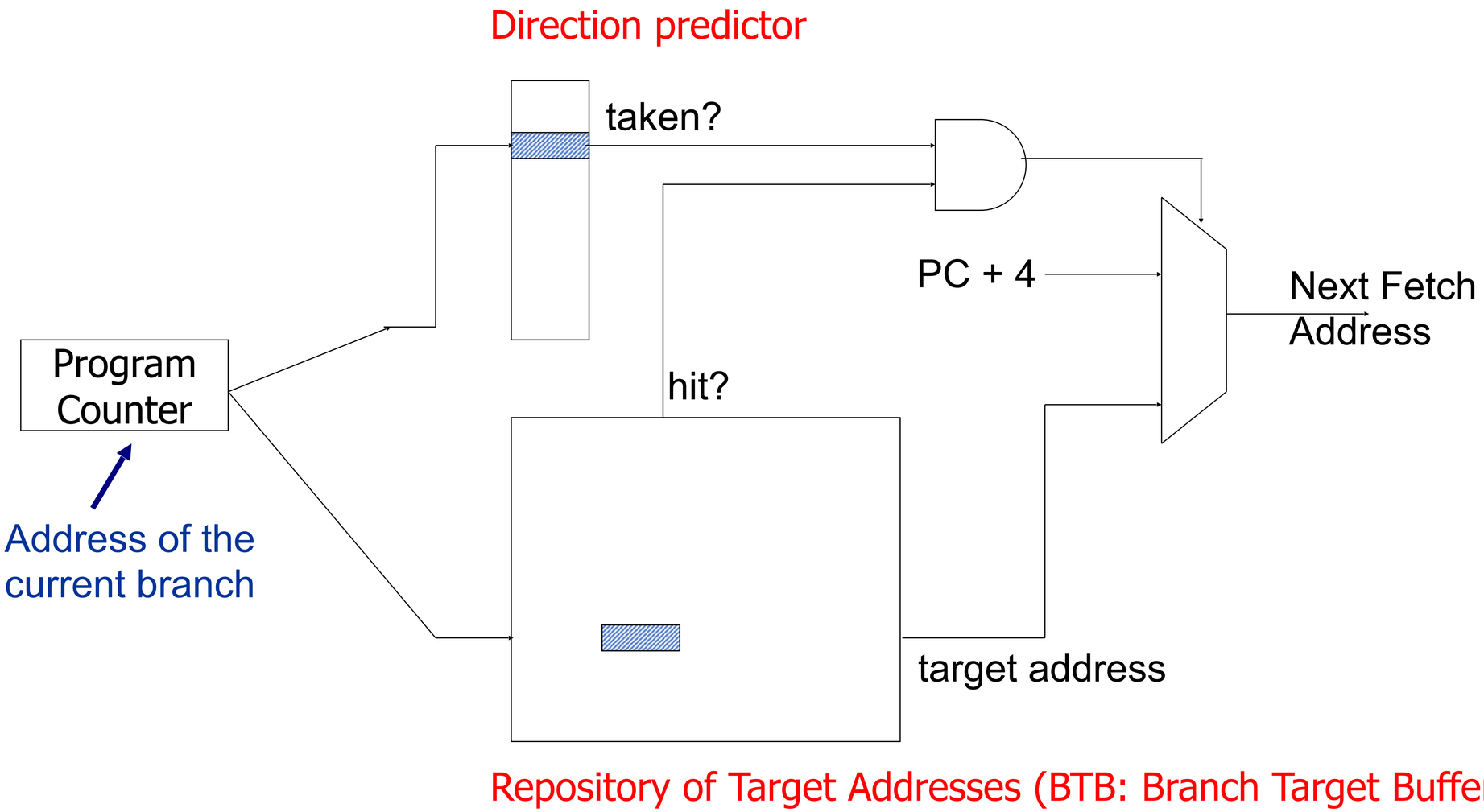
Branch Predictor: A bit deeper



Branch Predictor: A bit deeper



Branch Predictor: A bit deeper



Static (compiler) Direction Prediction

Always not-taken: Simple to implement: no need for BTB, no direction prediction

Low accuracy: ~30-40%

Always taken: No direction prediction, we need BTB though

Better accuracy: ~60-70%

Backward branches (i.e., loop branches) are usually taken

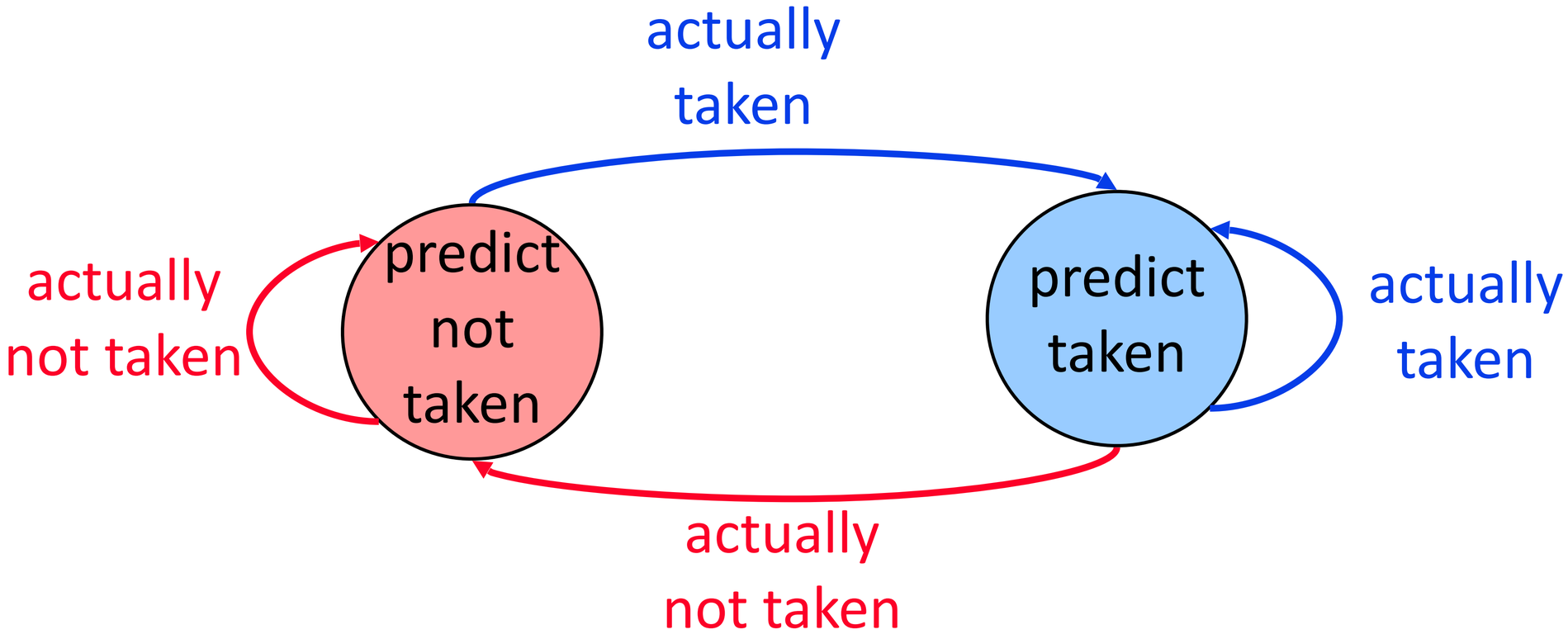
Dynamic Predictors

Microarchitectural way of predicting it.

Simple one: Last time predictor

Last-time predictor

Last-time predictor



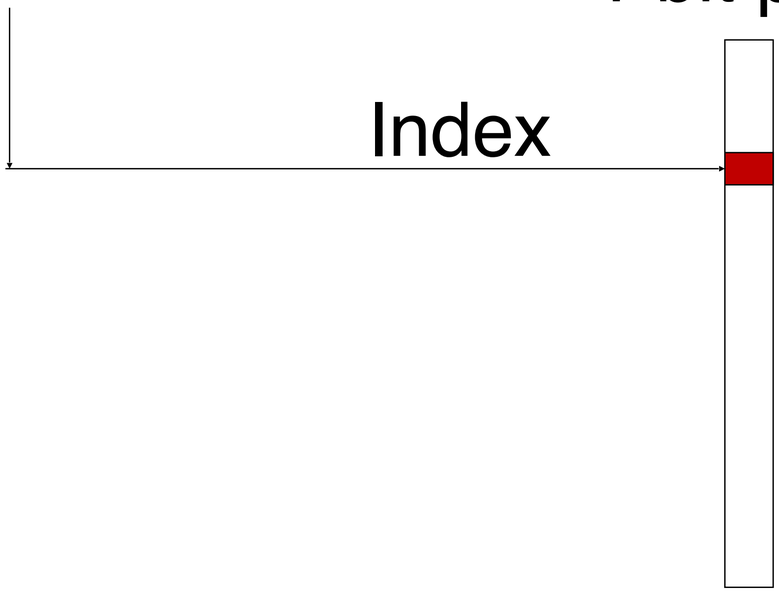
Implementation

K bits of branch instruction address

Implementation

K bits of branch instruction address

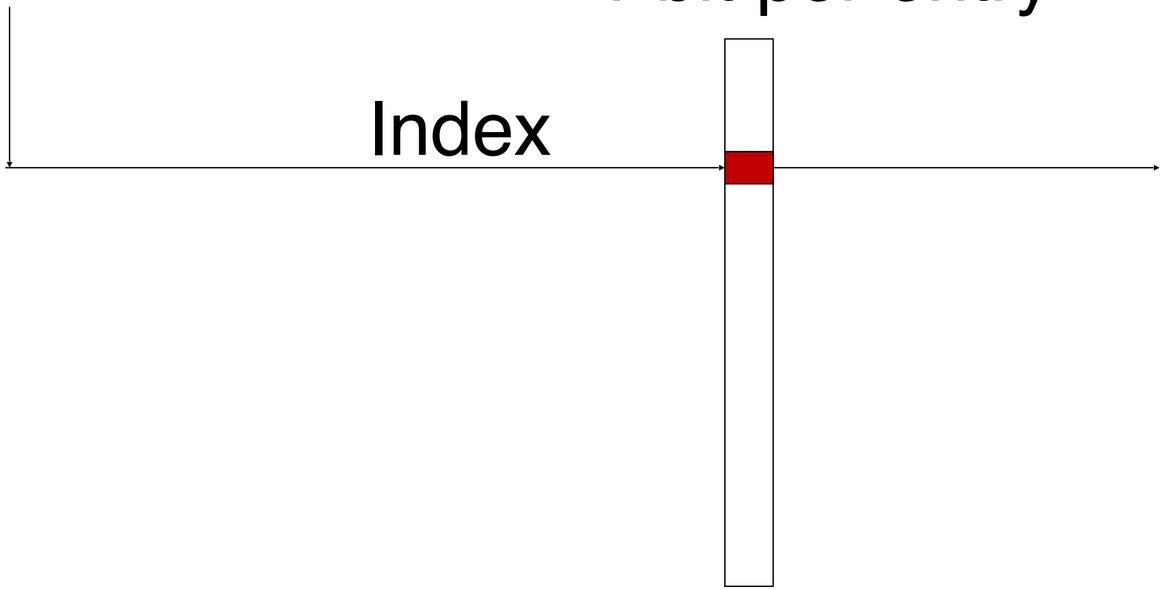
Branch history table of 2^K entries, 1 bit per entry



Implementation

K bits of branch instruction address

Branch history table of 2^K entries, 1 bit per entry



Use this entry to predict this branch:

0: predict not taken
1: predict taken

Performance of Last-time predictor

TTTTTTTTTTNNNNNNNNNN - 90% accuracy

Always mispredicts the last iteration and the first iteration of a loop branch

Accuracy for a loop with N iterations = (N-2)/N

+ Loop branches for loops with large number of iterations

-- Loop branches for loops will small number of iterations

TNTNTNTNTNTNTNTNTNTN — 0% accuracy

Performance: Calculating the CPI

20% of all instructions are branches, 85% accuracy

Last-time predictor CPI =

[1 + (0.20*0.15) * 2] =

1.06 (minimum two stalls to resolve a branch)

Types of Branches

	Conditional	Unconditional
Direct	if - then- else for loops (bez, bnez, etc)	procedure calls (jal) goto (j)
Indirect		return (jr)

Why do we need branch prediction?

- Allows useful work to be completed while waiting for a branch to resolve.

- Processors with deep pipelines
  - Intel Core 2 Duo: 14 stages
  - AMD Athlon 64: 12 stages
  - Intel Pentium 4: 31 stages
- Many cycles before branch is resolved
  - Wasting time if wait…
  - Would be good if can do some useful work…

Branch Prediction

- Key Idea: Predict branch outcome heuristically.
  - If successful, then we've gained a performance improvement.
  - Otherwise, discard instructions that have been executed speculatively.
    - Program execution state is still correct, all we've done is "waste" a little power.

Branch Prediction Strategies

- **Static:**
  - Decided before runtime
  - Examples:
    - Always-Not Taken
    - Always-Taken
    - Backwards Taken, Forward Not Taken (BTFNT)
- **Dynamic** (aka profile-driven prediction):
  - Prediction decisions may change during the execution of the program

What happens when a branch is mispredicted?

- On a mispredict:
  - No speculative state may commit
    - Squash instructions in the pipeline
    - Cannot allow stores to registers for instructions which would not get to commit
  - Need to handle exceptions appropriately

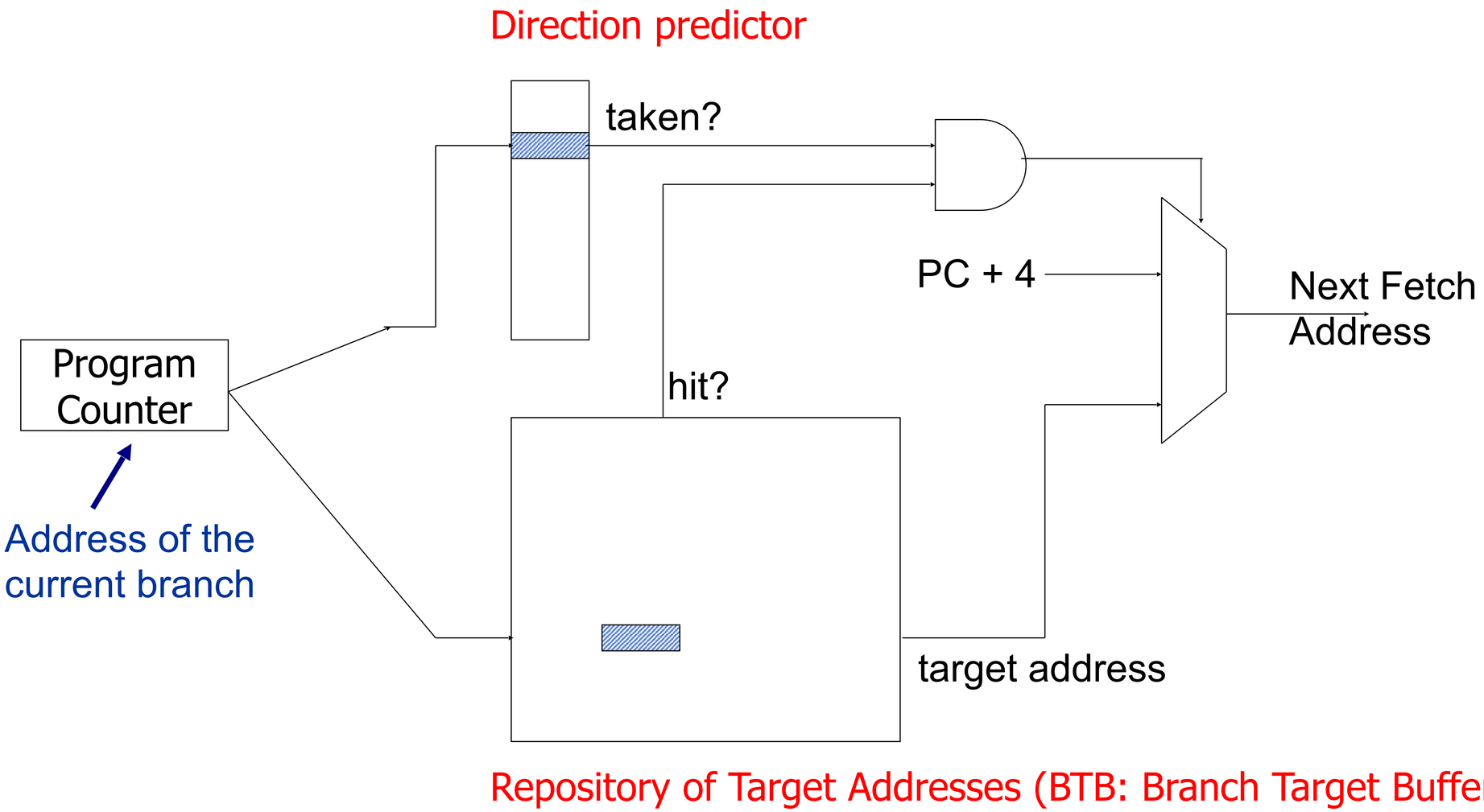
Direction Based Prediction

- **Pro:** Simple to implement
  - But, branch behaviour is often variable (dynamic) and depends on how the program is behaving recently.
  - Can’t capture such behaviour at compile time with simple direction based prediction!
- **Need history (aka profile)-based prediction.**

Direction-Based Branch Prediction

- Which things exactly to predict?
  - **Direction:**
    - Taken / Not Taken
    - Can only be Direction
  - **Target Address**
    - PC+offset (Taken)/ PC+4 (Not Taken)
    - How implemented?
      - Using Branch Target Address Cache (BTAC) or Branch Target Buffer (BTB)

Branch Predictor: A bit deeper



Example: Branch Penalty Calculation

- Assume a MIPS pipeline using predict taken:
  - 16% of all instructions are branches:
    - 4% unconditional branches: 3 cycle penalty
    - 12% conditional:
      - 50% taken: 3 cycle penalty
      - 50% not taken: 0 cycle penalty

Solution

- For a sequence of N instructions:
  - 3 * 0.04 * N delays due to unconditional branches
  - 0.5 * 3 * 0.12 * N delays due to conditional taken
- Overall CPI=
  - 1.3*N
  - (or 1.3 cycles/instruction)
  - 30% Performance Hit!!!

History-based Branch Prediction

- An important example is State-based branch prediction:
- Consists of 2 parts:
  - "Predictor" to guess where/if instruction will branch (and to where)
  - "Recovery Mechanism": A way to fix mistakes

History-based Branch Prediction

- **One bit predictor:**
  - Use the outcome from the last time the branch instruction was executed.
- **Problem:**
  - Even if branch is almost always taken, we will be wrong at least twice
  - if branch alternates between taken, not taken
    - **We get 0% accuracy**

Example

- Let initial value = **T**
- Suppose actual outcome of branches

  NT, NT,NT,T,T,T

  - Predictions are: **T, NT,NT,NT,T,T**
    - 2 wrong (in red), 4 correct = 66% accuracy
- 2-bit predictors can do better
- In general, can have k-bit predictors.

1-bit Predictor: Exercise

- Program assumptions:
  - 23% loads and in ½ of cases, next instruction uses load value
  - 13% stores
  - 19% conditional branches
  - 2% unconditional branches
  - 43% other

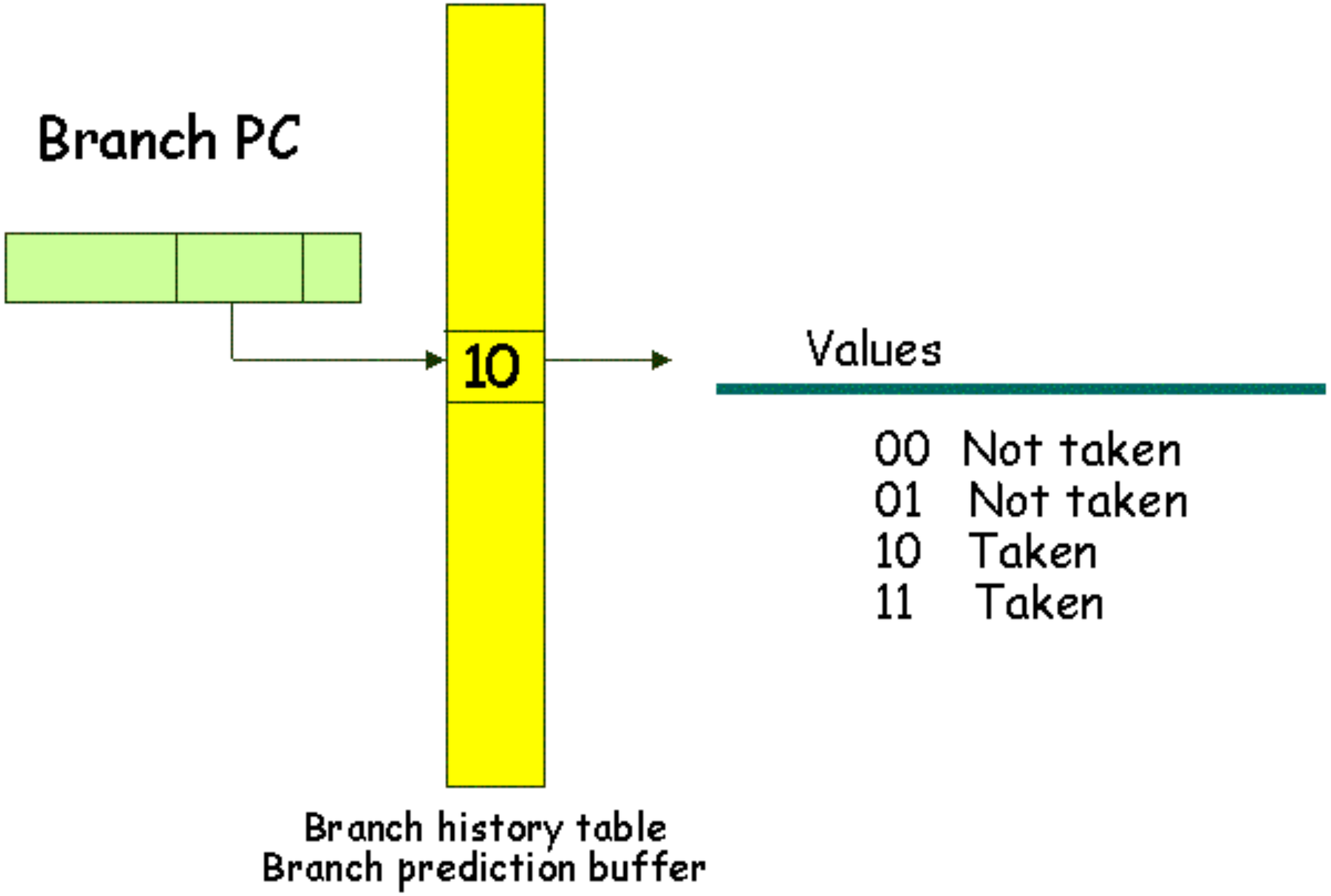
Exercise

- Machine Assumptions:
  - 5 stage pipe
    - Penalty of 1 cycle on use of load value immediately after a load.
    - Jumps are resolved in ID stage and incur a 1 cycle branch penalty.
    - 75% branch prediction accuracy and 1 cycle delay (penalty) on misprediction.

Solution

- CPI penalty calculation:
  - Loads:
    - 50% of the 23% of loads have 1 cycle penalty: 0.5*.23=0.115
  - Jumps:
    - All 2% of jumps have 1 cycle penalty: 0.02*1 = 0.02
  - Conditional Branches:
    - 25% of the 19% are mispredicted, have a 1 cycle penalty: 0.25*0.19*1 = 0.0475
- Total Penalty: 0.115 + 0.02 + 0.0475 = 0.1825
- Average CPI: 1 + 0.1825 = 1.1825

2-bit branch prediction

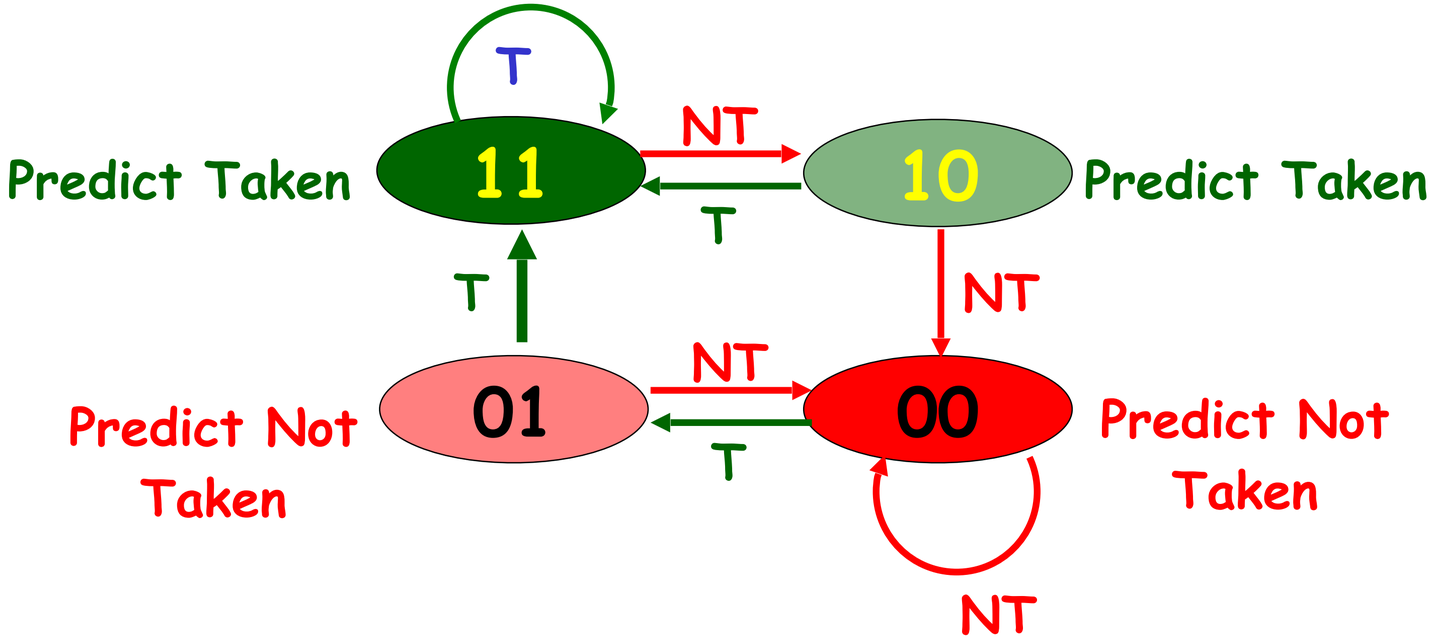


2-Bit Branch Prediction

- Approach: Prediction is changed only if mispredicted twice
- Adds hysteresis to decision making process

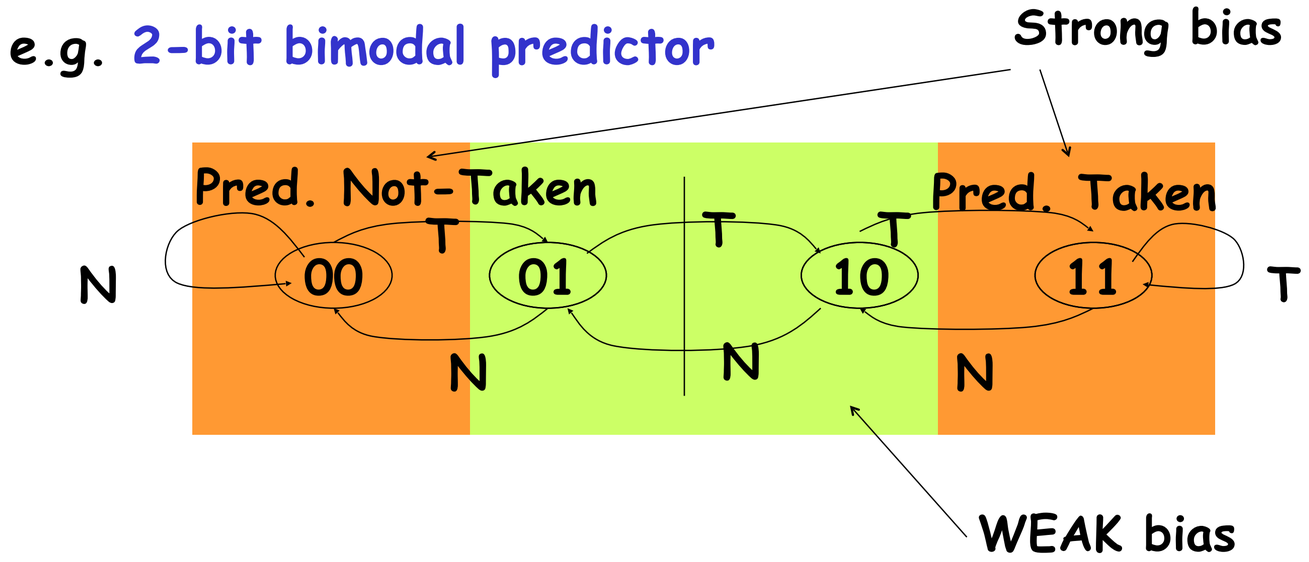
Red: stop, not taken

Green: go, taken



AKA Saturation Counter Predictor

- Observation: branches highly **bimodal**
- n-bit saturation counter
  - Hysteresis
  - n-bit entries in branch prediction table



n-bit Saturating Counter

- Values: 0 ~ 2^n-1
- When the counter is greater than or equal to one-half of its maximum value,
  - the branch is predicted as taken. Otherwise, not taken.
- Studies have shown that the 2-bit predictors do almost as well

2-bit Predictor

- What is the prediction accuracy using a 4096 entry 2-bit branch predictor for a typical application?
  - 99% to 80% depending upon the application.
- Can an n-bit (n>2) predictor do better?
  - Not really! 2-bit predictors do almost as well as any n-bit predictors.
- How can then the accuracy of branch prediction be improved?
  - **Correlating branch predictor.**

Predictors in Simple Pipelines

- Initial pipelined processors, e.g. MIPS, SOLARIS, etc.:
  - Did only trivial branch predictions.
- Possible reasons could be:
  - The penalty of mispredictions not as severe as in deeper pipelined processors.
  - Sophisticated branch predictors did not exist.
- Advanced branch prediction techniques have now become very important:
  - With the use of deeper pipelines.
  - Introduction of superscalar processors.

Improving Accuracy of Branch Predictors

- It may be possible to improve the accuracy of branch prediction:
  - By observing the recent behavior of other branches.
- **Example:**

```
if (a==2){
        b=2;}
if(b==2}{
        b=0;}
```

The Main Idea

- **Record m most recently executed branches as taken or not taken,**
  - **Use that pattern to select the proper n-bit branch history table (BHT).**

Main Idea

- An (m,k) predictor:
  - Makes use of the outcomes observed for the last m branches:
    - Uses m number of k-bit predictors.
    - Behavior of a branch can be predicted by choosing from 2**m branch predictors.

Local and global history

- Local Behavior

  What is the predicted direction of Branch A given the outcomes of previous instances of Branch A ?

- Global Behavior

  What is the predicted direction of Branch Z given the outcomes of *all** previous branches A, B, …, X and Y?

  Number of previous branches tracked limited by the history length

Two Level Branch Predictors

First level: Global branch history register (N bits)

The direction of last N branches

Second level: Table of saturating counters for each history entry

The direction the branch took the last time the same history was seen

GHR
(global history register)

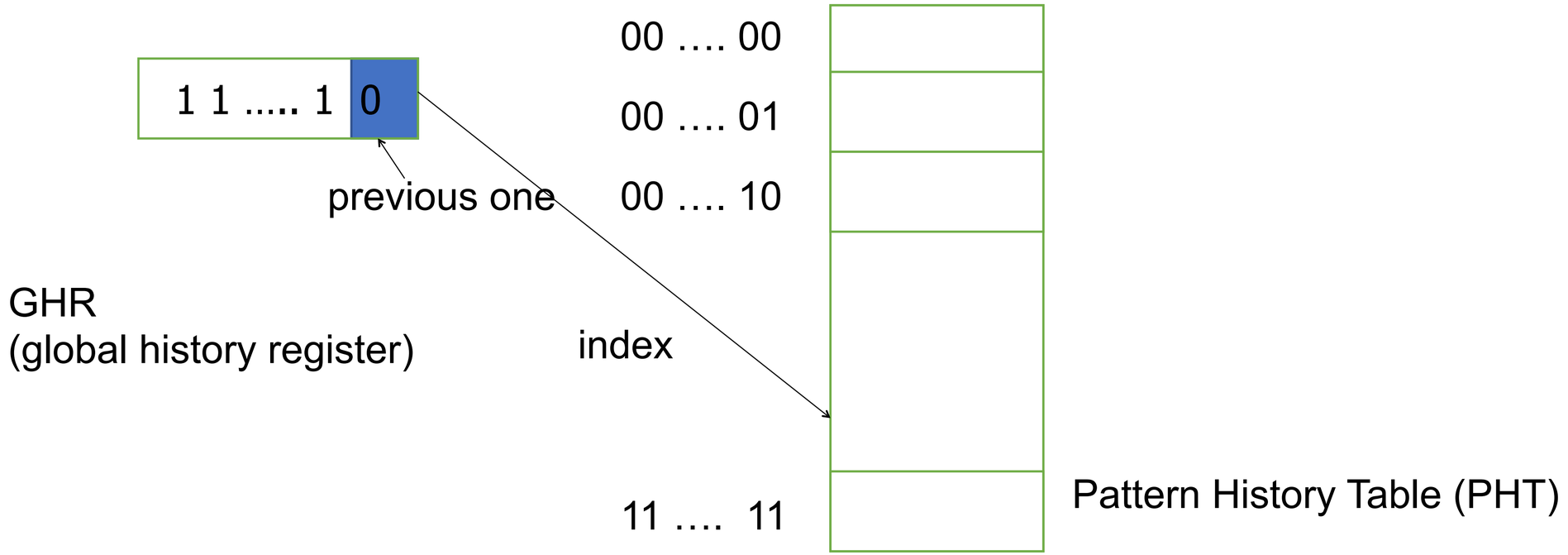
Two Level Branch Predictors

First level: Global branch history register (N bits)

The direction of last N branches

Second level: Table of saturating counters for each history entry

The direction the branch took the last time the same history was seen



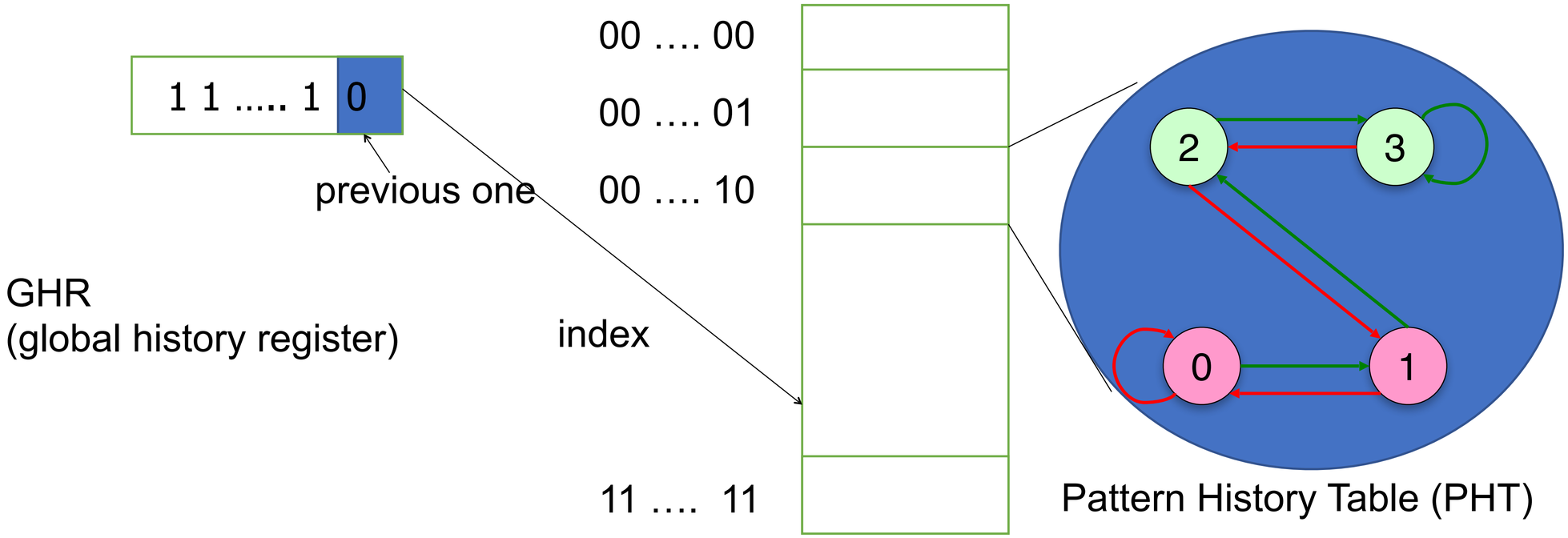
Two Level Branch Predictors

First level: Global branch history register (N bits)

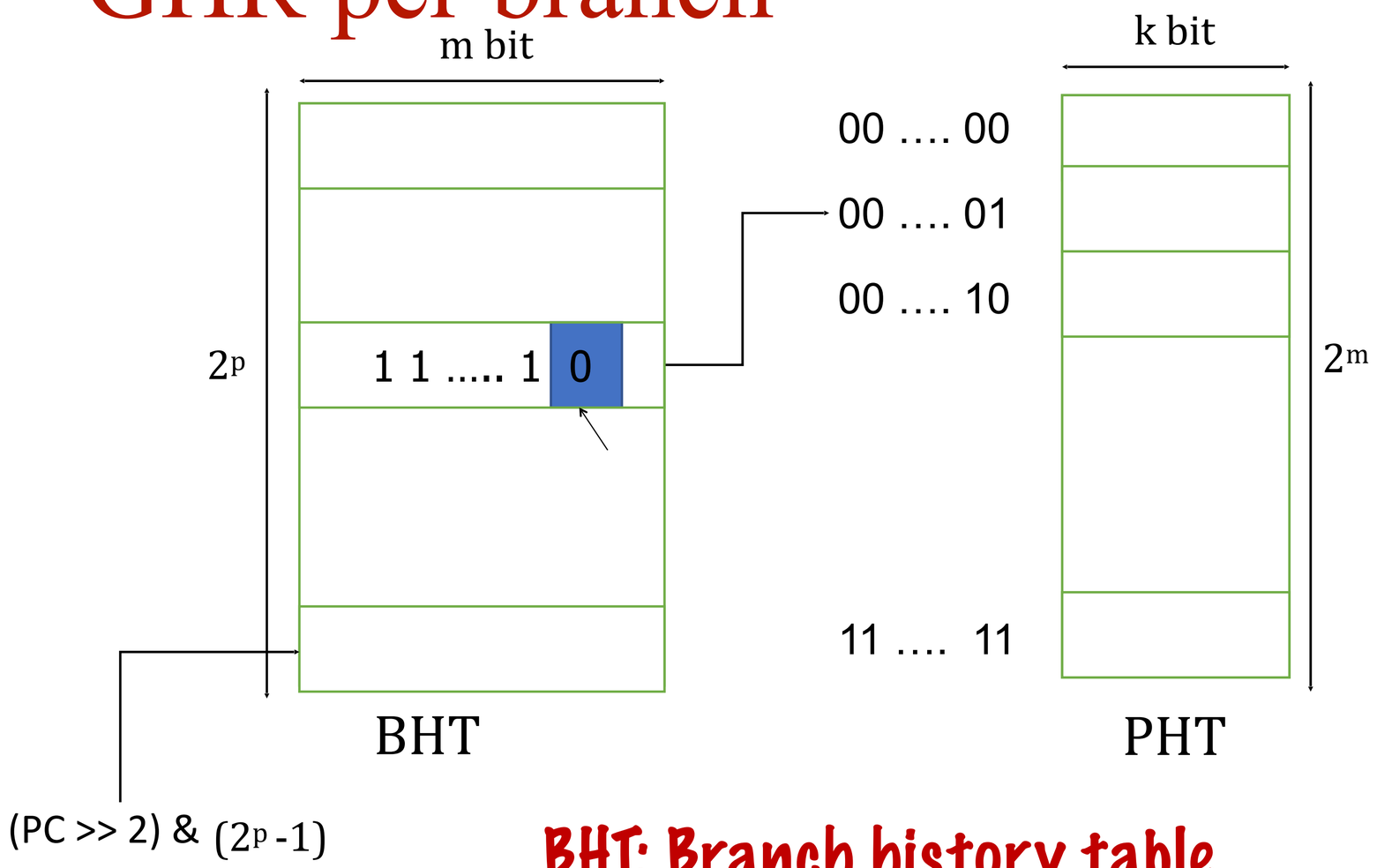
The direction of last N branches

Second level: Table of saturating counters for each history entry

The direction the branch took the last time the same history was seen



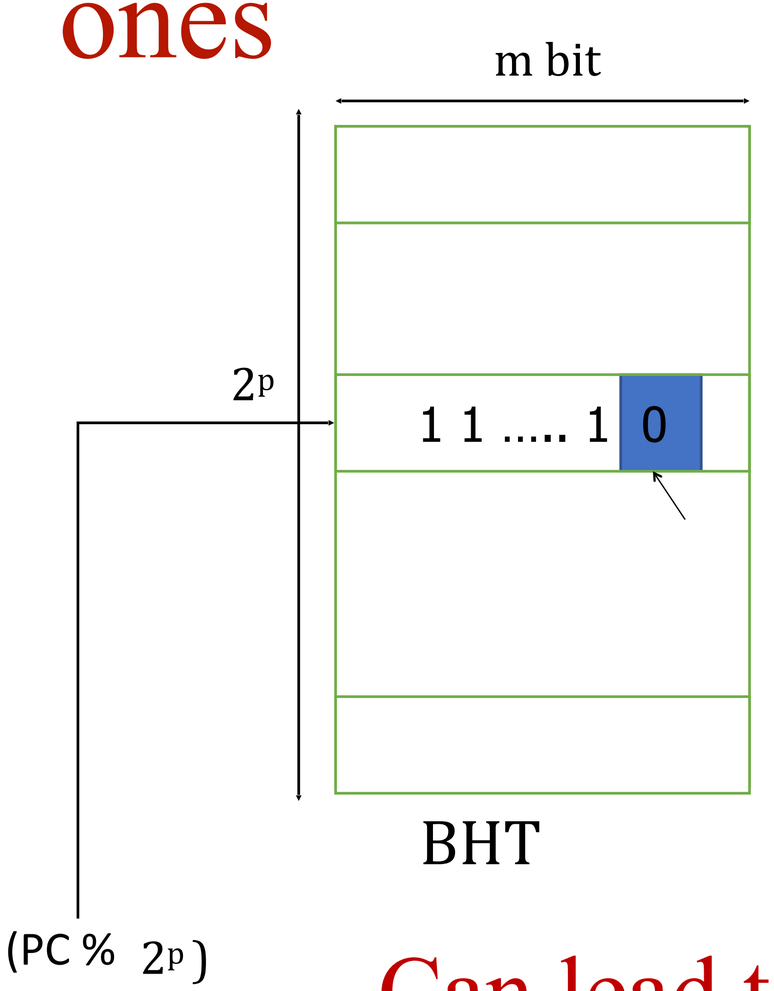
GHR per branch

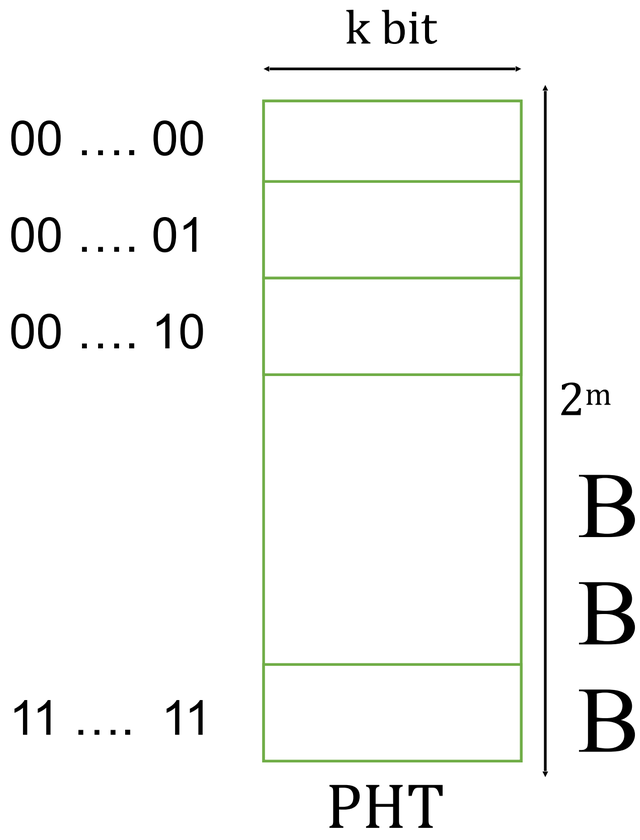


BHT: Branch history table

Mostly K=2, m =12 for example

Set of branches: One register for correlated ones

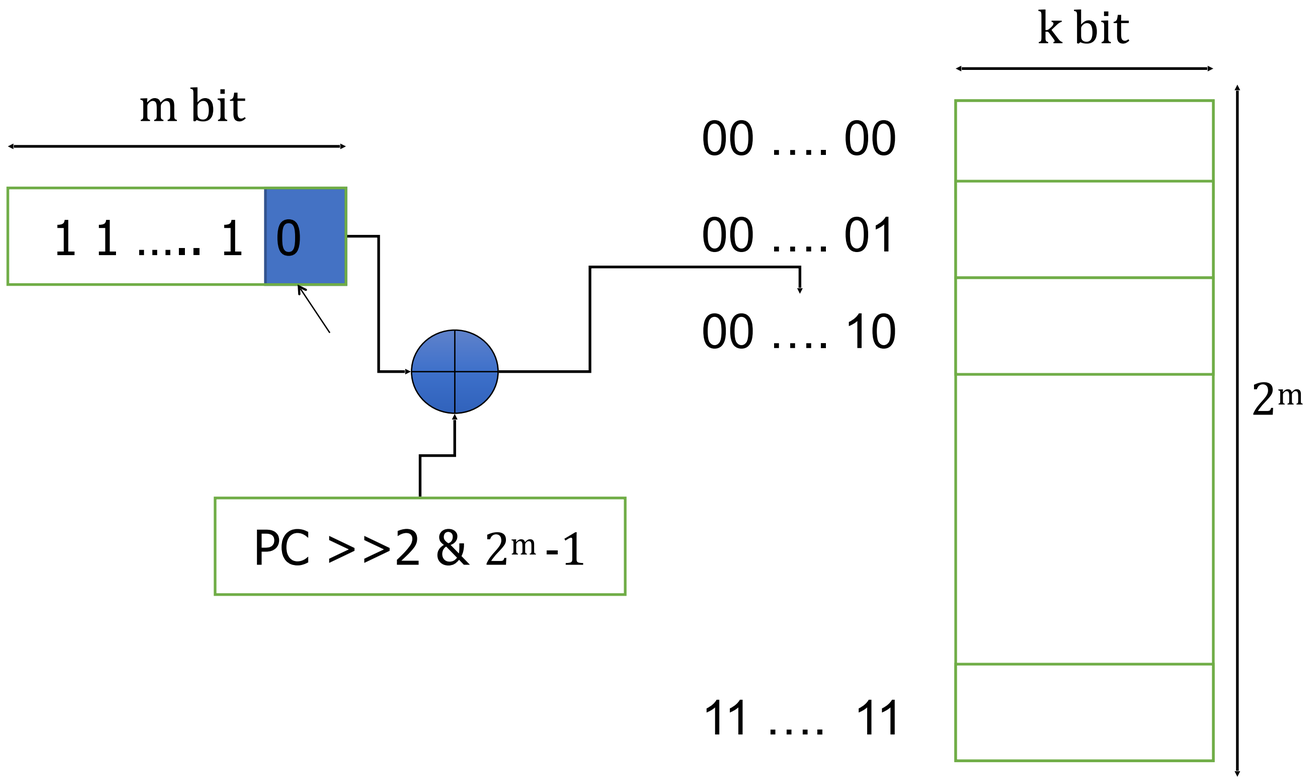




B1: if (aa == 2) aa = 0;
B2: if (bb == 2); bb=0;
B3: if (aa != bb) { … }

Can lead to positive/negative/neutral interference

Gshare is the answer



For a given history and for a given branch (PC) counters are trained

Few Important Points

Branch prediction happens at the IF stage.

We know the target outcome at the end of EX stage.

So BHT and PHT will be updated after EX stage for the corresponding PC. Any issues here?

Issue

I1 F D E
I2 F D E
I3 F D E

Lets assume I1 and I3 are branch instructions. I1 will update BHT and PHT in E stage, and I3 will probe BHT and PHT in F stage. To make sure PHT is updated correctly with the correct BHT entry, BHT entry is communicated till the E stage.

State-of-the-art

State of the art: Neural vs. TAGE

1970: Flynn
1972: Riseman/Foster
1979: Smith Predictor
1991: Two-level prediction
1993: gshare, tournament
1996: Confidence estimation
1996: Vary history length
1998: Cache exceptions
2001: Neural predictor
2004: PPM
2006: TAGE
2016: Still TAGE vs Neural

- Neural: AMD, Samsung
- TAGE: Intel?, ARM?
- Similarity
  - Many sources or “features”
- Key difference: how to combine them
  - TAGE: Override via partial match
  - Neural: integrate + threshold
- Every CBP is a cage match
  - Andre Seznec vs. Daniel Jimenez

BTB (Target Address Predictor)

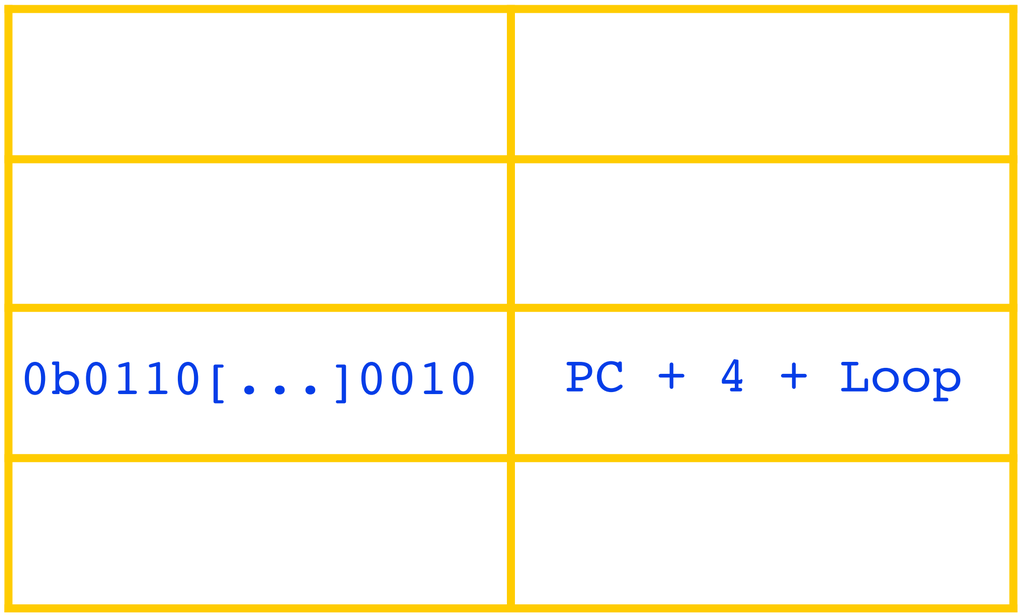
Address of branch instruction

`0b0110[...]01001000`

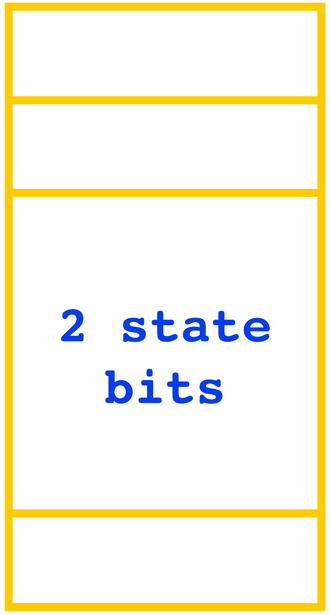
Branch instruction

`BNEZ R1 Loop`

Branch Target Buffer (BTB)



Branch History Table (BHT)



BTB is probed in the fetch stage along with the direction predictor. A hit in the BTB means the PC is a branch PC.

Branch Target Buffer

- **For BTB to make a correct prediction, we need:**
  - **BTB hit:** the branch instruction should be in the BTB
  - **Prediction hit:** the prediction should be correct
  - **Target match:** the target address must not be changed from the last time
- Example: BTB hit ratio of 96%, 97% prediction hit, 1.2% of target change,

  The overall prediction accuracy = **0.96 * 0.97 *0.988 = 92%**

Branch Instruction Address	Branch Prediction Statistics	Branch Target Address
. . .	. . .	. . .

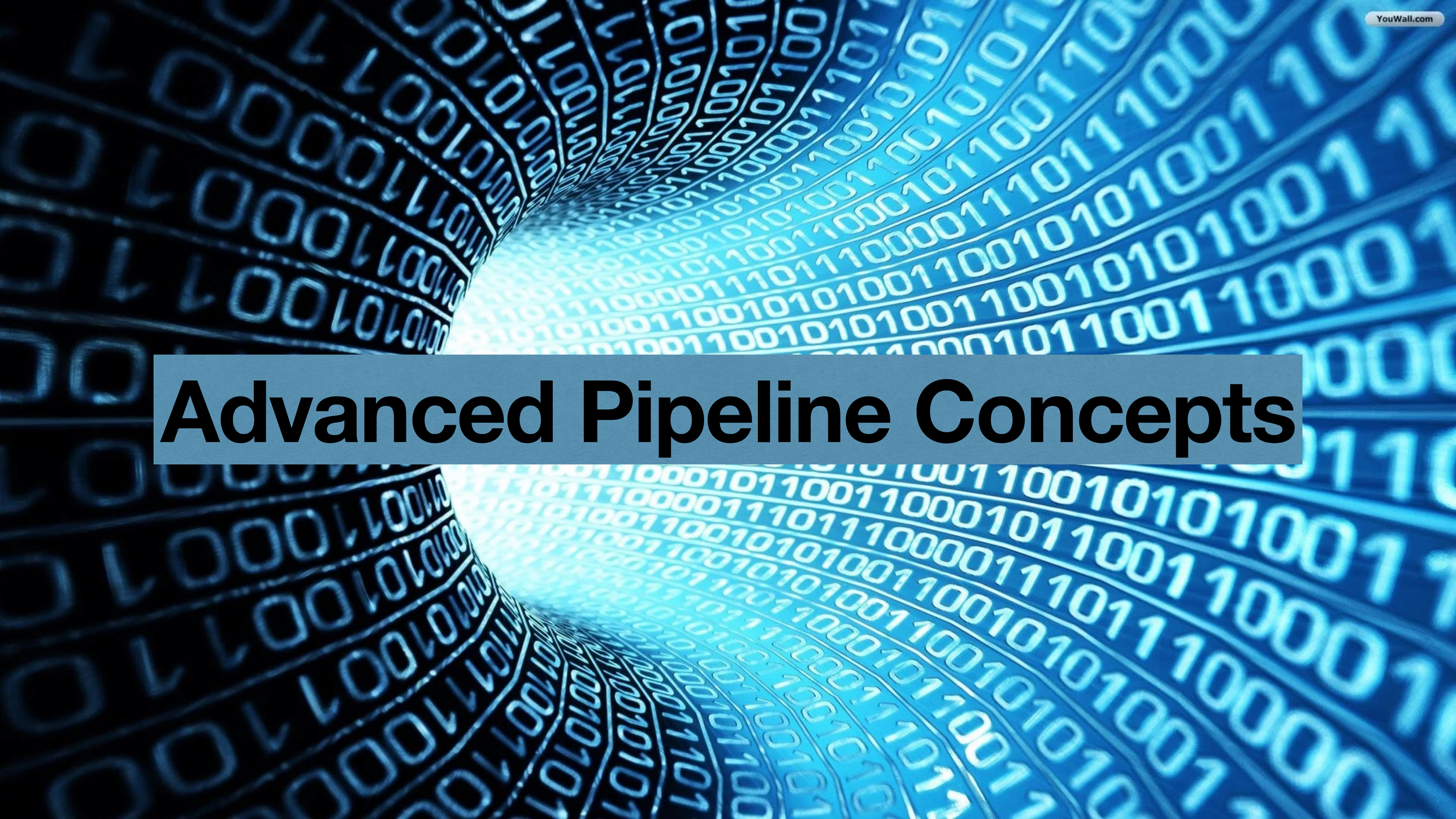
Book

Textbook reading: P & H, Chapter 4

Digital Logic Design + Computer Architecture

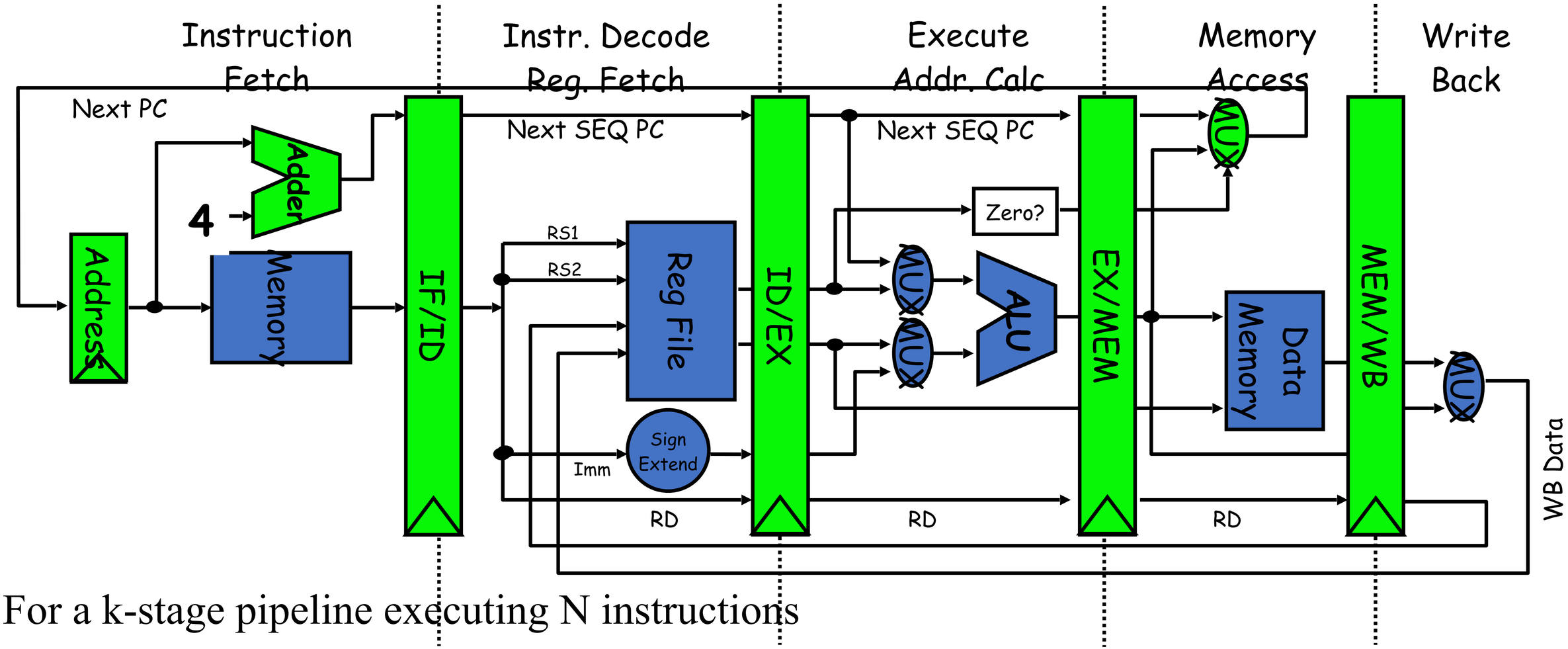
Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay



Advanced Pipeline Concepts

Pipeline Recap…



For a k-stage pipeline executing N instructions

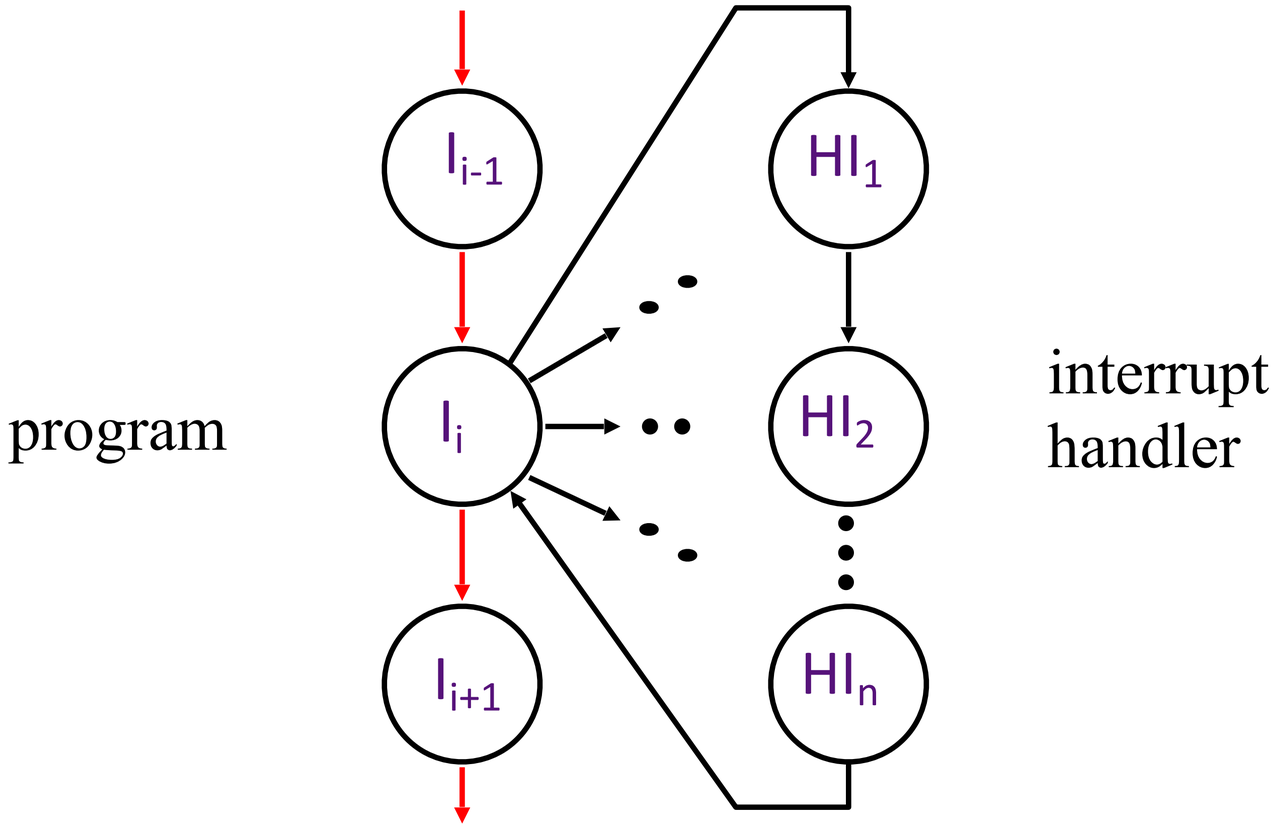
first instruction: k cycles

Next N-1 instructions: N-1 cycles, total = K + (N-1) cycles

Exception/Interrupt

An unscheduled event that disrupts program (instructions) in action.

Interrupt Handling

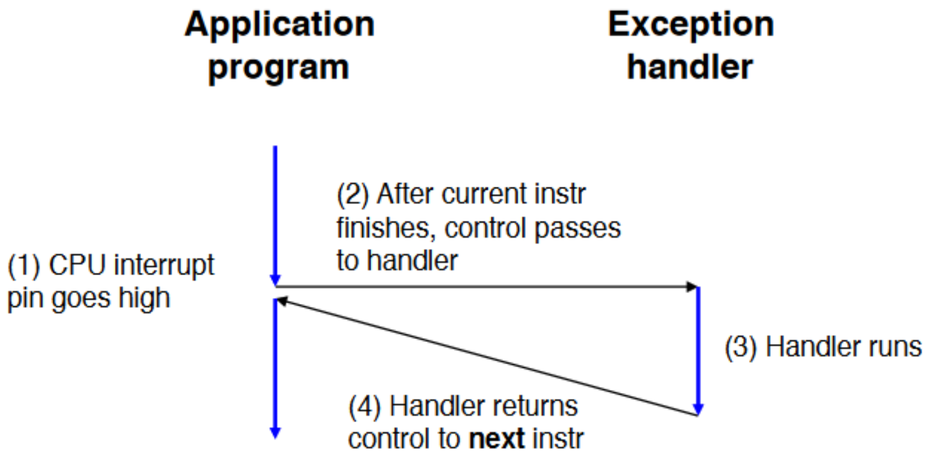


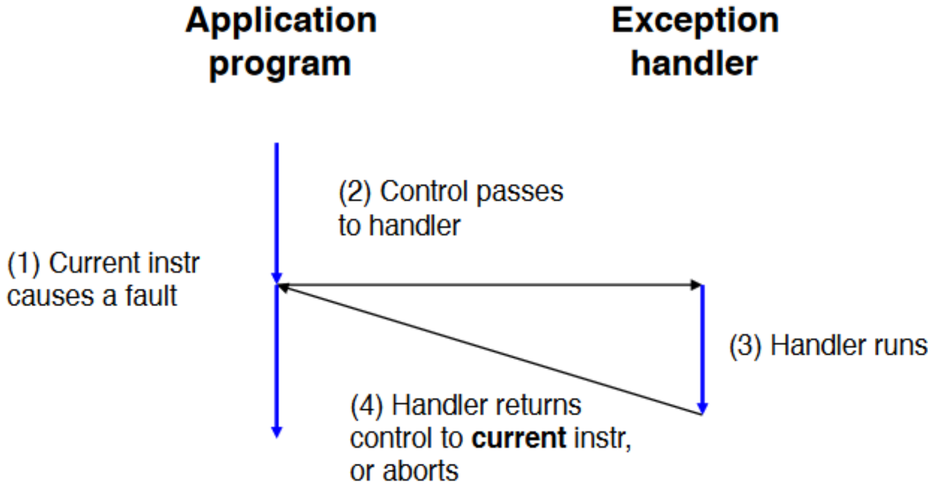
An *external or internal event* that needs to be processed. The event is usually unexpected or rare from program's point of view.

Types of Interrupts

- Asynchronous: an *external event*
  - input/output device service-request
  - timer expiration
  - power disruptions, hardware failure
- Synchronous: an *internal event (a.k.a. traps or exceptions)*
  - undefined opcode, privileged instruction
  - arithmetic overflow, FPU exception, misaligned memory access
  - *virtual memory exceptions:* page faults, TLB misses, protection violations
  - system calls, e.g., jumps into kernel

Interrupt and Exception





Interrupt Handler

Exception program counter (EPC): address of the offending instruction,

Saves EPC before enabling interrupts to allow nested interrupts

Need to mask further interrupts at least until EPC can be saved

Need to read a *status register* that indicates the cause of the interrupt

Handshake between processor and the OS

Processor:

stops the offending instruction,

makes sure all prior instructions complete,

flushes all the future instructions (in the pipeline)

Sets a register to show the cause

Saves EPC

Disables further interrupts

Jumps to pre-decided address (cause register or vectored)

Handshake between processor and the OS

OS:

Looks at the cause of the exception

Interrupt handler saves the GPRs

Handles the interrupt/exception

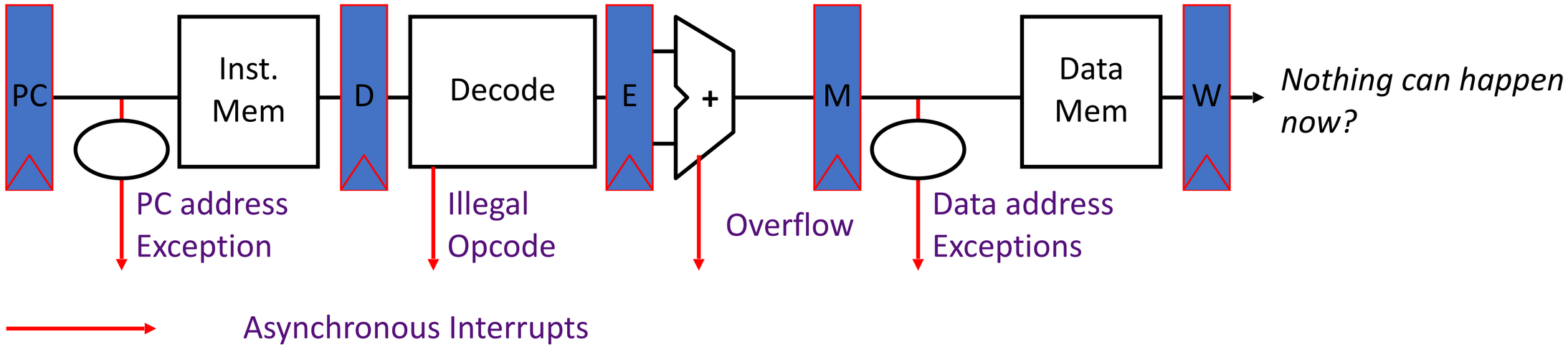
Calls RFE

Contd.

Uses a special indirect jump instruction RFE (*return-from-exception*) which

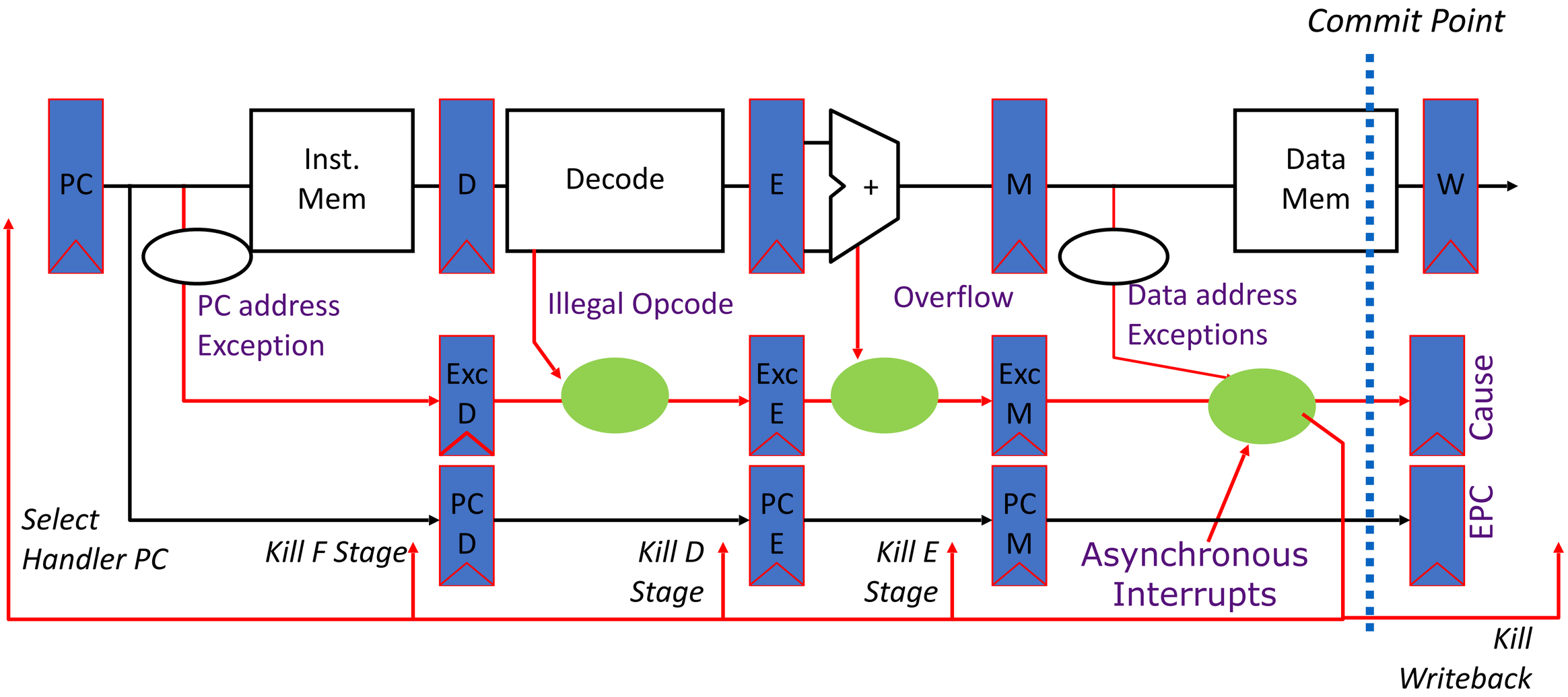
- enables interrupts
- restores the processor to the user mode

Exception handling and Pipelining



- When do we stop the pipeline for *precise* interrupts or exceptions?
- How to handle multiple simultaneous exceptions in different pipeline stages?
- How and where to handle external asynchronous interrupts?

Exception handling and Pipelining



Contd.

- Hold exception flags in pipeline until commit point for instructions that will be killed
- Exceptions in earlier pipe stages override later exceptions *for a given instruction*
- If exception at commit: update cause and EPC registers, kill all stages, inject handler PC into fetch stage

SuperScalar: More Instructions Per Cycle (IPC)

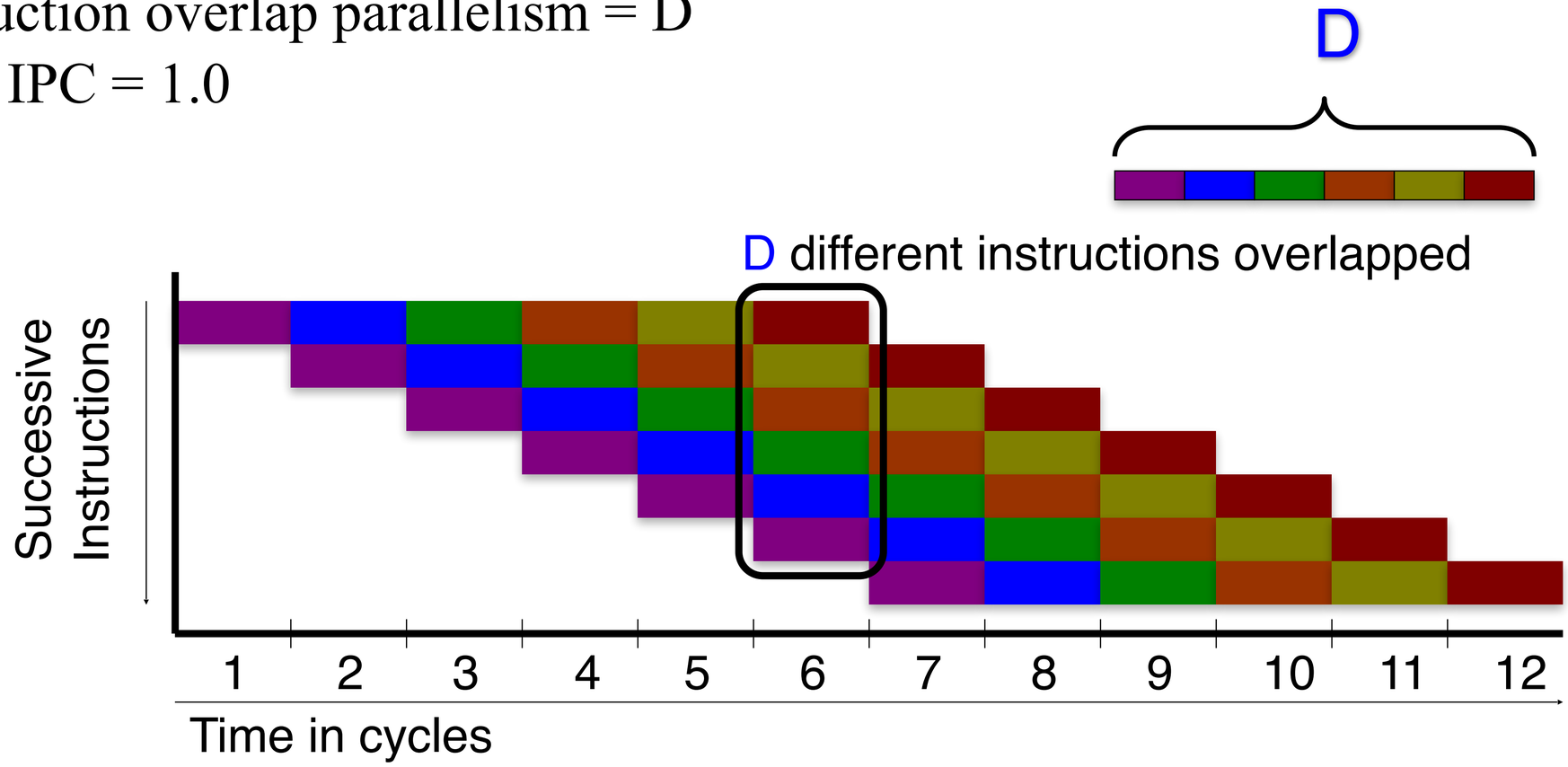
Beyond Scalar

- Scalar pipeline limited to CPI ≥ 1.0
  - Can never run more than 1 insn per cycle

- "Superscalar" can achieve CPI ≤ 1.0 (i.e., IPC ≥ 1.0)
  - *Superscalar* means executing multiple insns in parallel

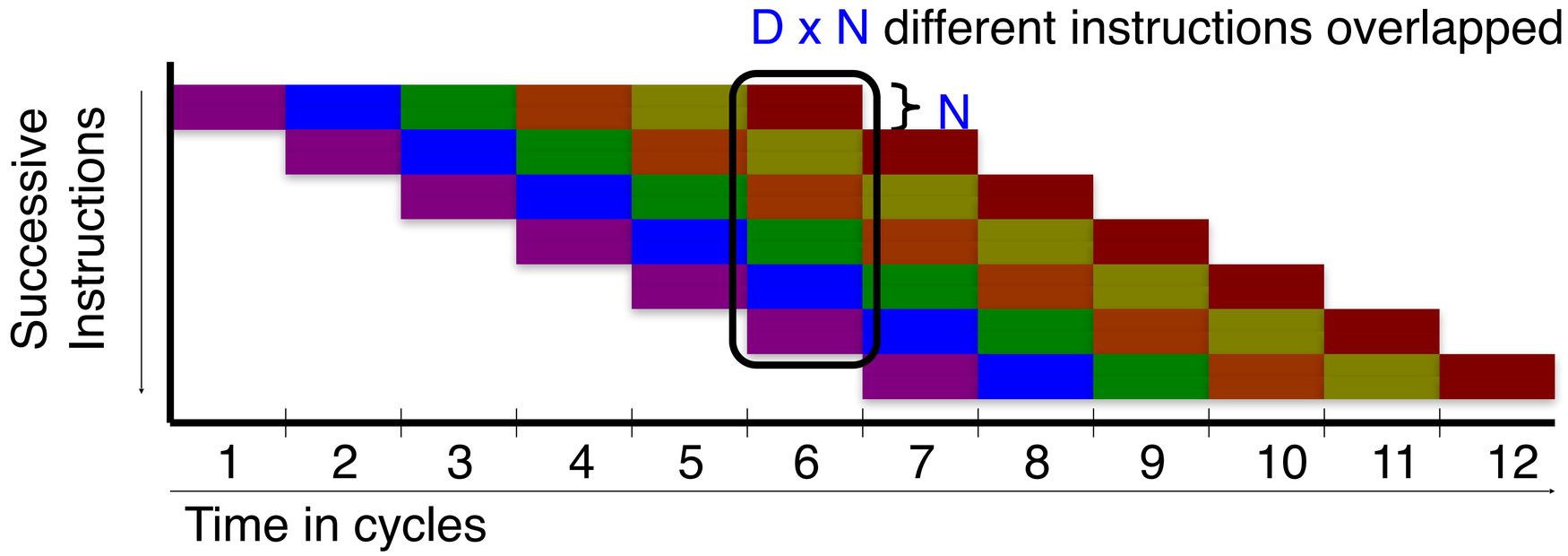
Instruction Level Parallelism (ILP)

- Scalar pipeline (baseline)
  - Instruction overlap parallelism = D
  - Peak IPC = 1.0



Superscalar Processor

- Superscalar (pipelined) Execution
  - Instruction parallelism = D x N
  - Peak IPC = N per cycle



What is the deal?

We get an IPC boost if the number of instructions fetched in one cycle are independent ☺

Complicates datapaths, multi-ported structures, complicates exception handling

Out of order (O3) processor: Pursuit of even higher IPC

Out-of-order follows data-flow order

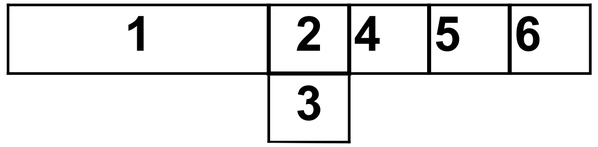
Example:

```
(1) r1 ← r4 / r7
(2) r8 ← r1 + r2
(3) r5 ← r5 + 1
(4) r6 ← r6 - r3
(5) r4 ← r5 + r6
(6) r7 ← r8 * r4
```

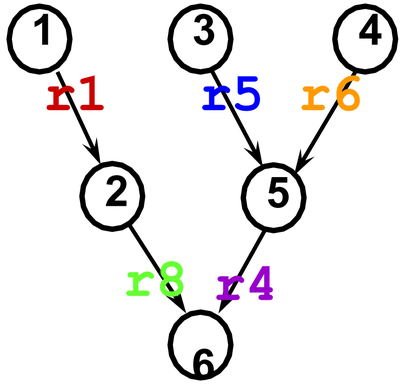
/* assume division takes 20 cycles */

In-order execution

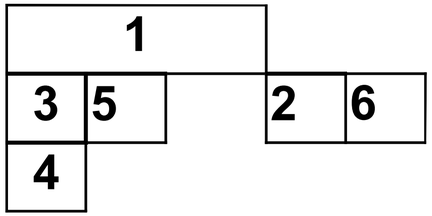
In-order (2-way superscalar)



Data Flow Graph



Out-of-order execution

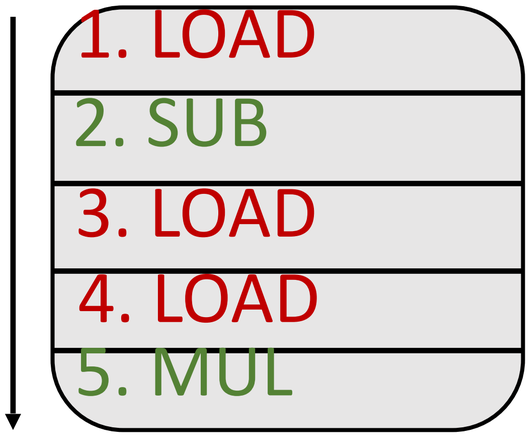


O3

Two or more instructions can execute in any order if they have no dependences (RAW, WAW, WAR)

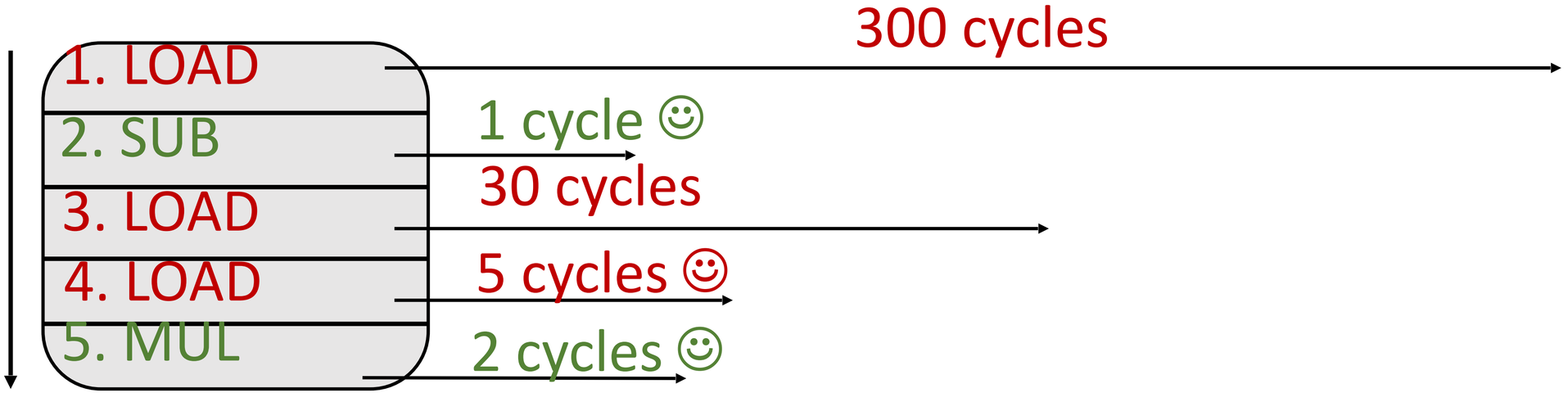
Completely orthogonal to superscalar/pipelining

O3 + Superscalar



In-order Instruction Fetch
(Multiple fetch in one cycle)

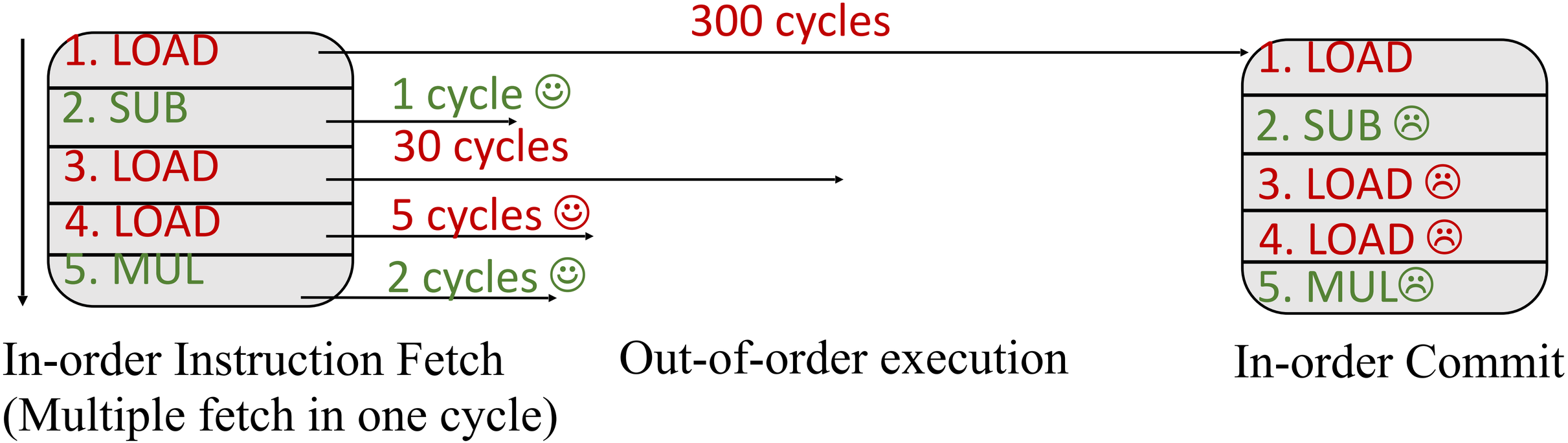
O3 + Superscalar



In-order Instruction Fetch
(Multiple fetch in one cycle)

Out-of-order execution

O3 + Superscalar



O3

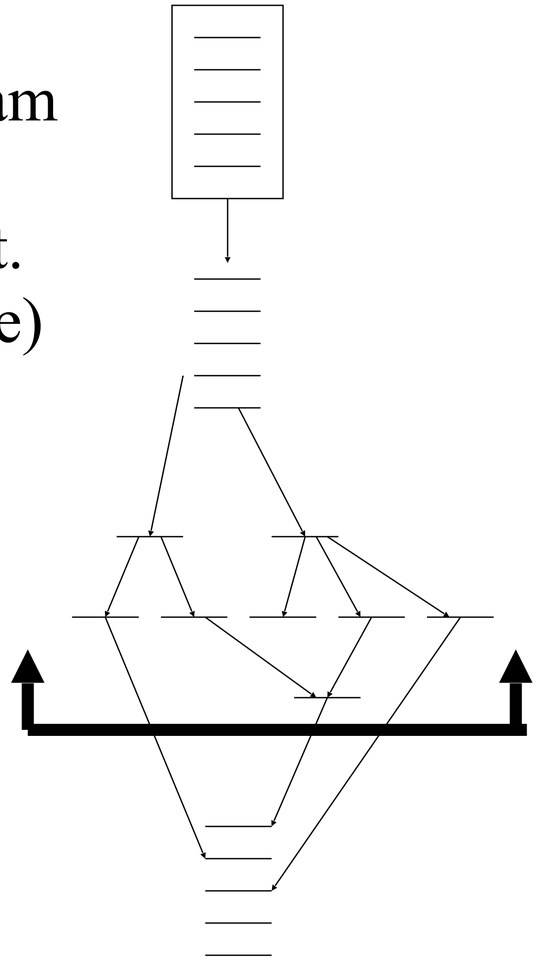
Program Form

Static program

dynamic inst.
Stream (trace)

execution
window

completed
instructions



Processing Phase

Dispatch/ dependences

inst. Issue

inst execution

inst. Reorder &
commit

The notion of Commit

After commit, the results of a committed instruction is visible to the programmer

and

the order at which instructions are fetched is also visible.

Why we need in-order commit?

Think about exceptions and precise exceptions

We should know till when we are done as per the programmer's view.

Performance: Time (Iron Law)

Time/Program =

Instructions/program X cycles/instruction X Time/cycle

Source code	ISA	microarch.
Compiler	microarch.	technology
ISA		

Example

Program p = one billion instructions

Processor takes one cycle per instruction

Processor clock is 4 GHz

CPU time = 10^{9} instructions X 1 cycle/instruction X 1/4 ns

= 0.25 second (4X faster)

Example

Program p = one billion instructions

Processor processes 10 instructions in one cycle

Processor clock is 4 GHz

CPU time = 10^{9} instructions X 0.10 cycle/instruction X 1/4 ns

= 0.025 second (40X faster)

Performance

- *Latency* (execution/response time): time to finish one task. It is additive (Performance = 1/latency)
- *Throughput* (bandwidth): number of tasks/unit time. It is not additive

Latency vs bandwidth

Latency vs Bandwidth, How they affect each other?

Latency helps bandwidth but not vice versa.

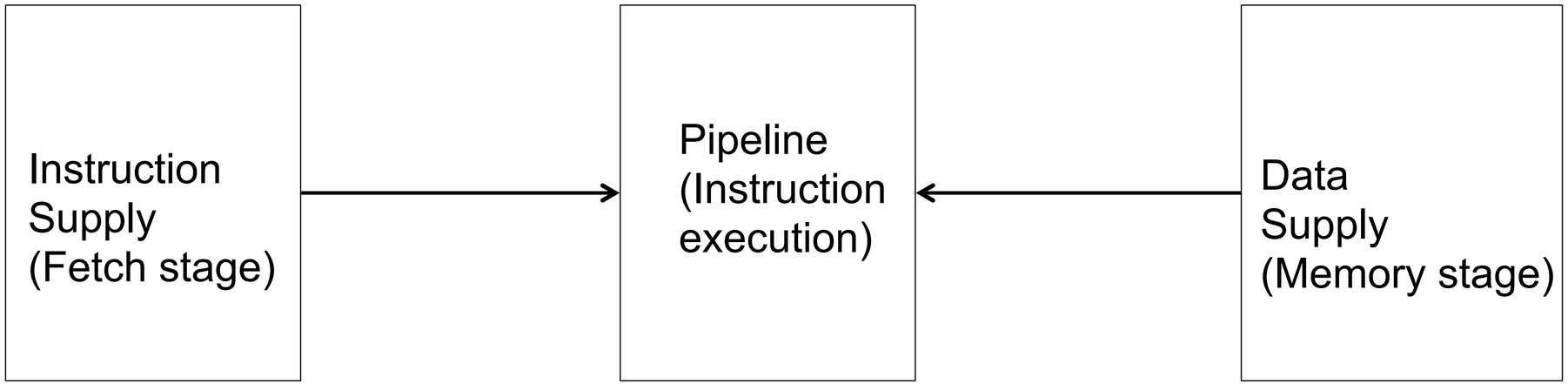
DRAM latency ↓ More # Accesses ~DRAM Bandwidth ↑

Bandwidth usually hurts latency

Queues - Bandwidth ↑ Increases latency ↓

Down the Memory Lane

The Ideal World

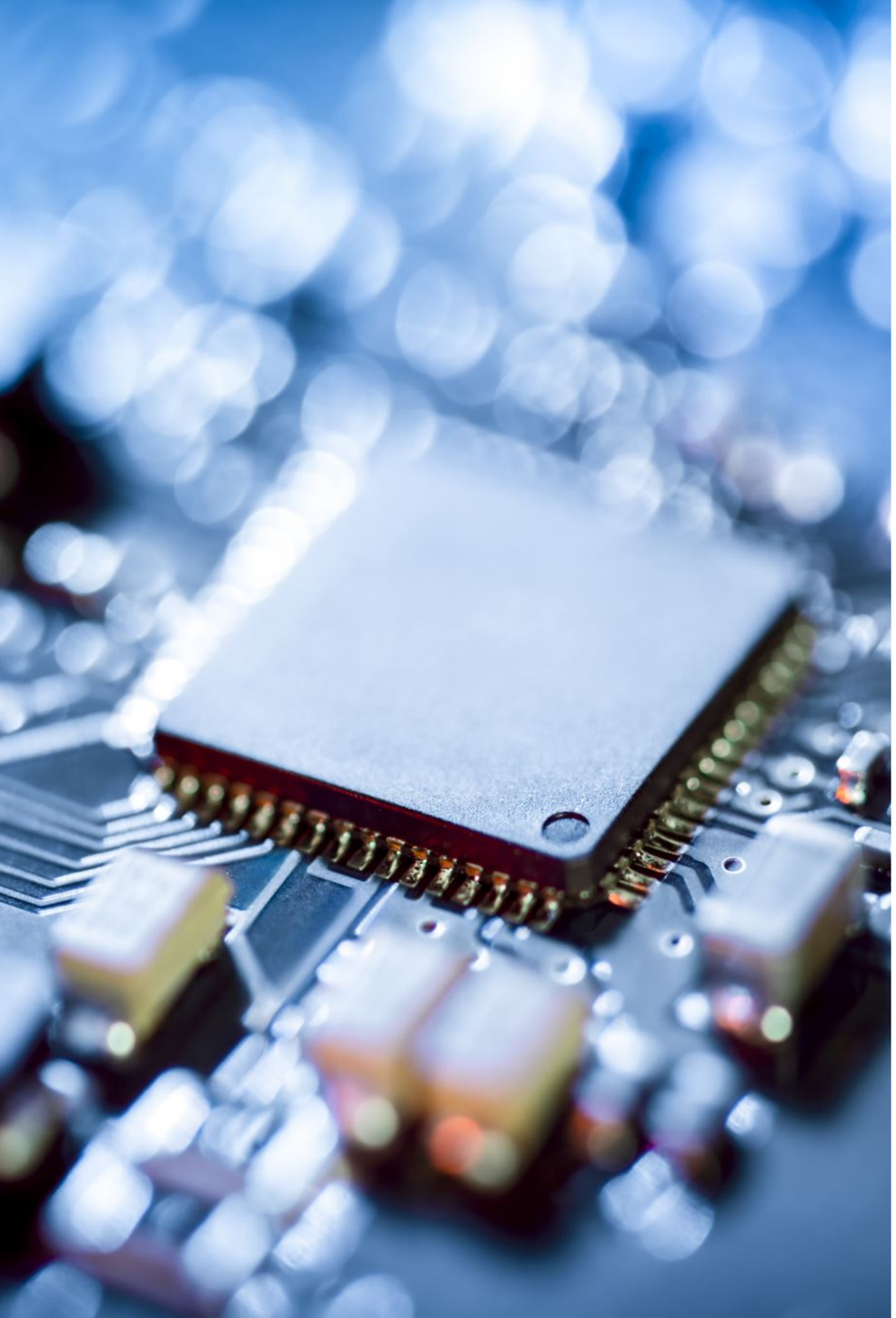


- Zero-cycle latency
- Infinite capacity
- Perfect control flow

- Zero-cycle latency
- Infinite capacity
- Infinite bandwidth

World of Memory Hierarchy: Why?

Before that What is memory?

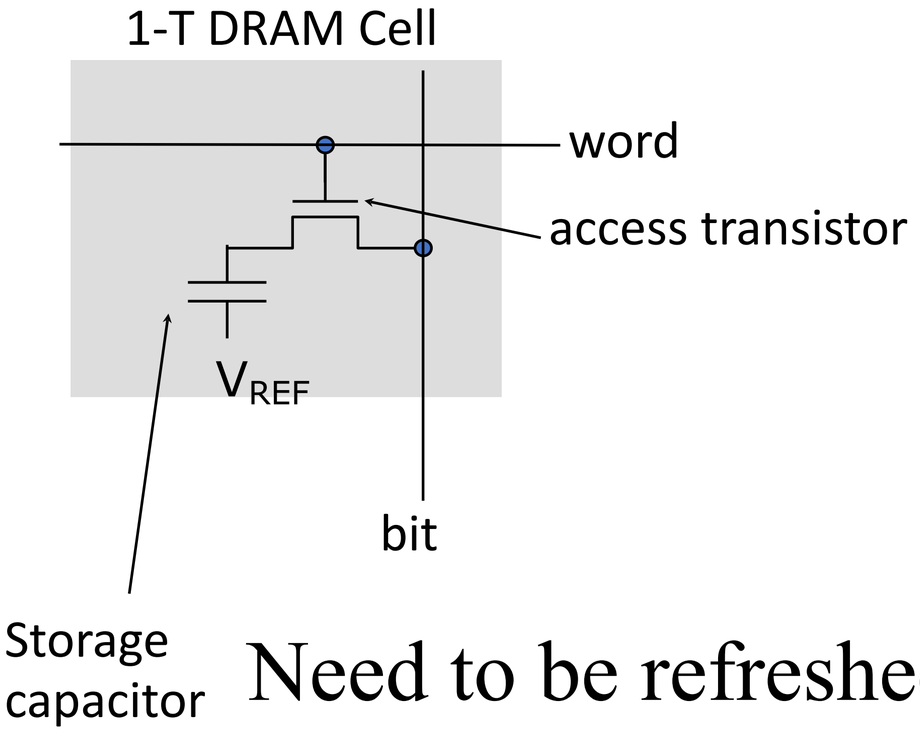


Semiconductor Memory

- Semiconductor memory began to be competitive in early 1970s
  - Intel formed to exploit market for semiconductor memory
  - Early semiconductor memory was Static RAM (SRAM). SRAM cell internals similar to a latch (cross-coupled inverters).

- First commercial Dynamic RAM (DRAM) was Intel 1103
  - 1Kbit of storage on single chip
  - charge on a capacitor used to hold value

One transistor DRAM



Denser

Value kept in one cell is as per the charge in the capacitor

Need to be refreshed periodically to maintain the charge

So dynamic RAM (DRAM)

DRAM

- Dynamic random-access memory
- Capacitor charge state indicates stored value
  - Whether the capacitor is charged or discharged indicates storage of 1 or 0
  - 1 capacitor
  - 1 access transistor

- Capacitor leaks
  - DRAM cell loses charge over time
  - DRAM cell needs to be refreshed

SRAMs (6T to 8T)

Static RAMs. No need of refresh as no capacitor.

Faster access (no capacitor)

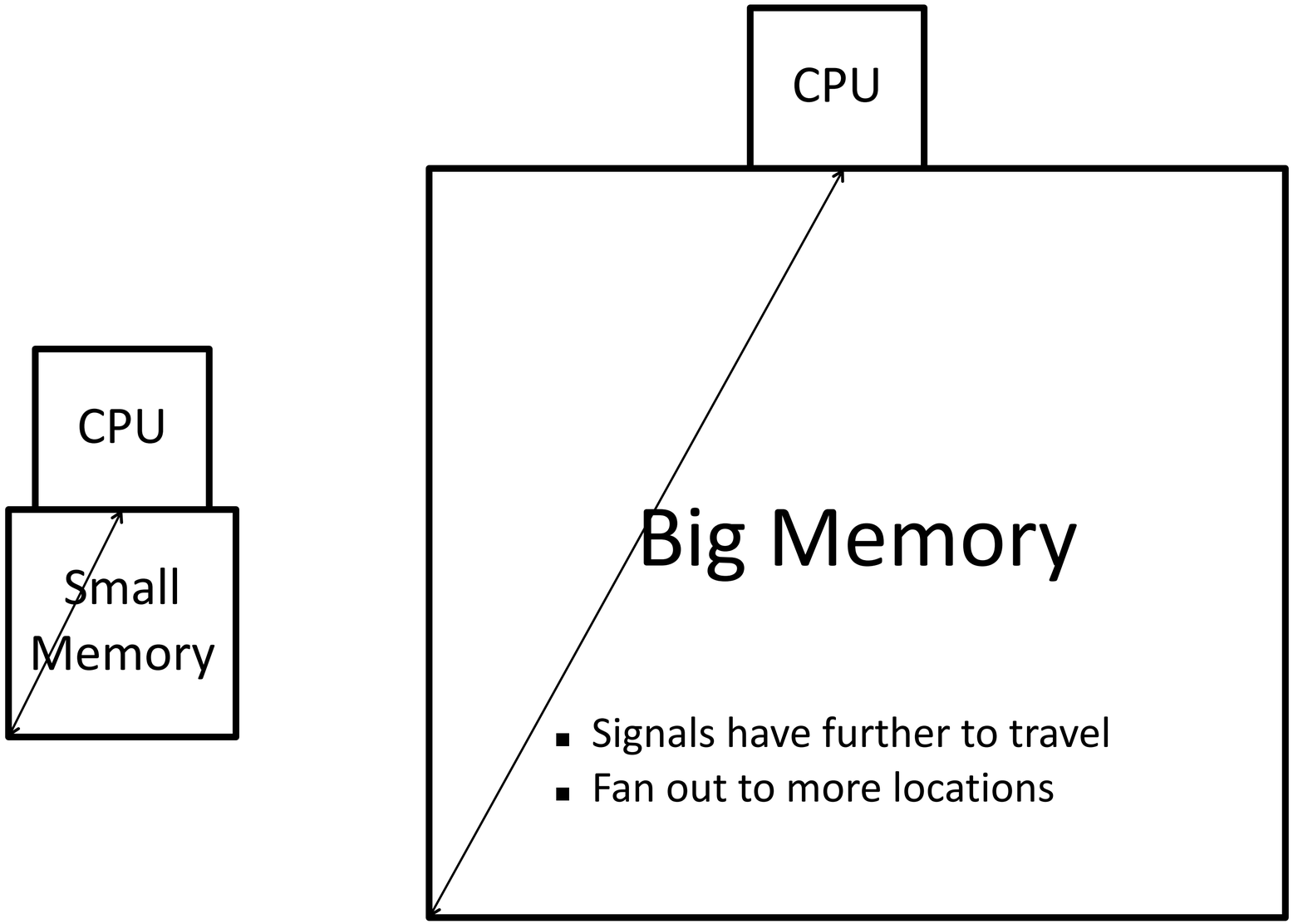
Density low as 6T: 1 bit compared to 1T: 1bit in DRAM

Costly than DRAM

The Problem

- Bigger is slower
  - SRAM, 512 Bytes, sub-nanosec
  - SRAM, KByte~MByte, ~nanosec
  - DRAM, Gigabyte, ~50 nanosec
  - Hard Disk, Terabyte, ~10 millisec
- Faster is more expensive (dollars and chip area)
  - SRAM, < 1000$ per GB
  - DRAM, < 20$ per GB
  - Hard Disk < 0.01$ per GB
  - These sample values scale with time
- Other technologies have their place as well
  - Flash memory, NVRAM, MRAM etc

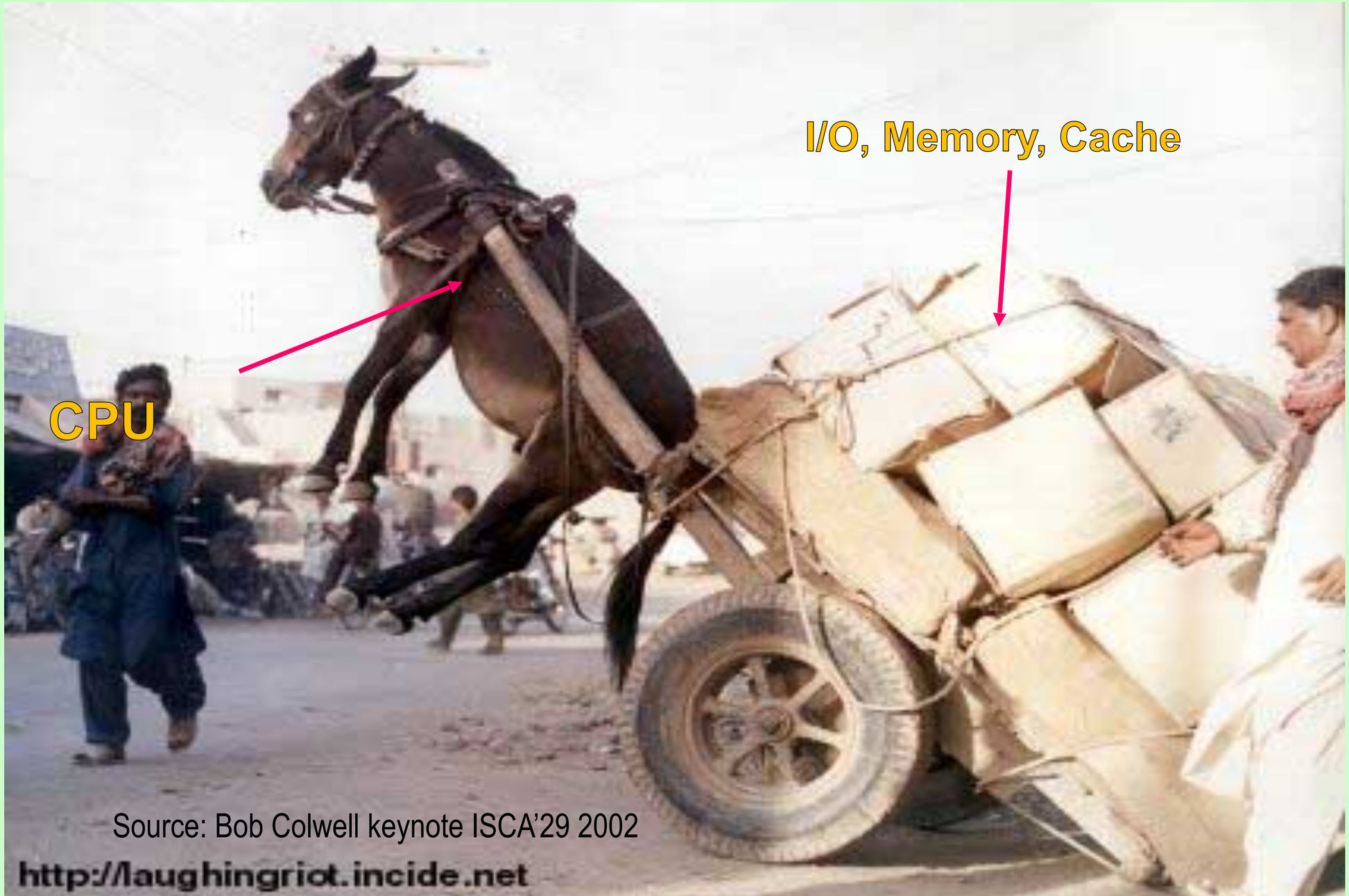
Size affects latency



An Unbalanced System

- Even a sophisticated processor may perform well below an ordinary processor:
  - Unless supported by matching performance by the memory system.
  - Unfortunately the gap is widening.
- Over the next few classes:
  - Study how memory system performance has been enhanced through various innovations and optimizations.

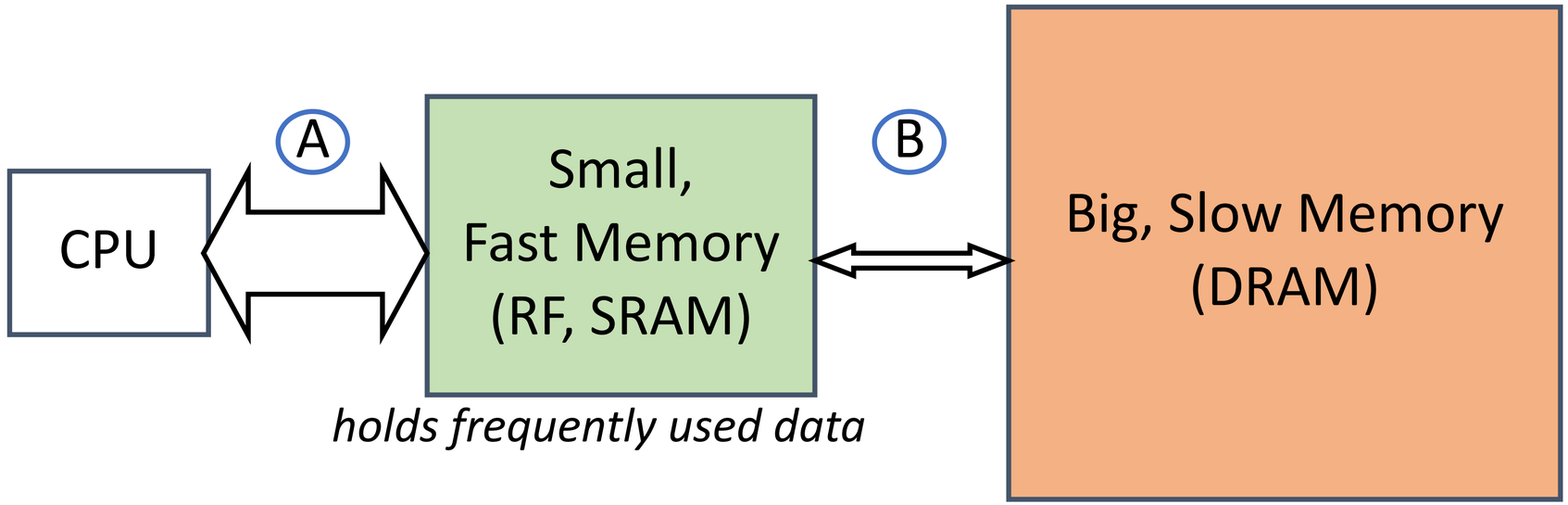
An Unbalanced System



Why Memory Hierarchy

- We want both fast and large
- But we cannot achieve both with a single level of memory
- Idea: Have multiple levels of storage (progressively bigger and slower as the levels are farther from the processor) and ensure most of the data the processor needs is kept in the fast(er) level(s)

Welcome to Memory Hierarchy



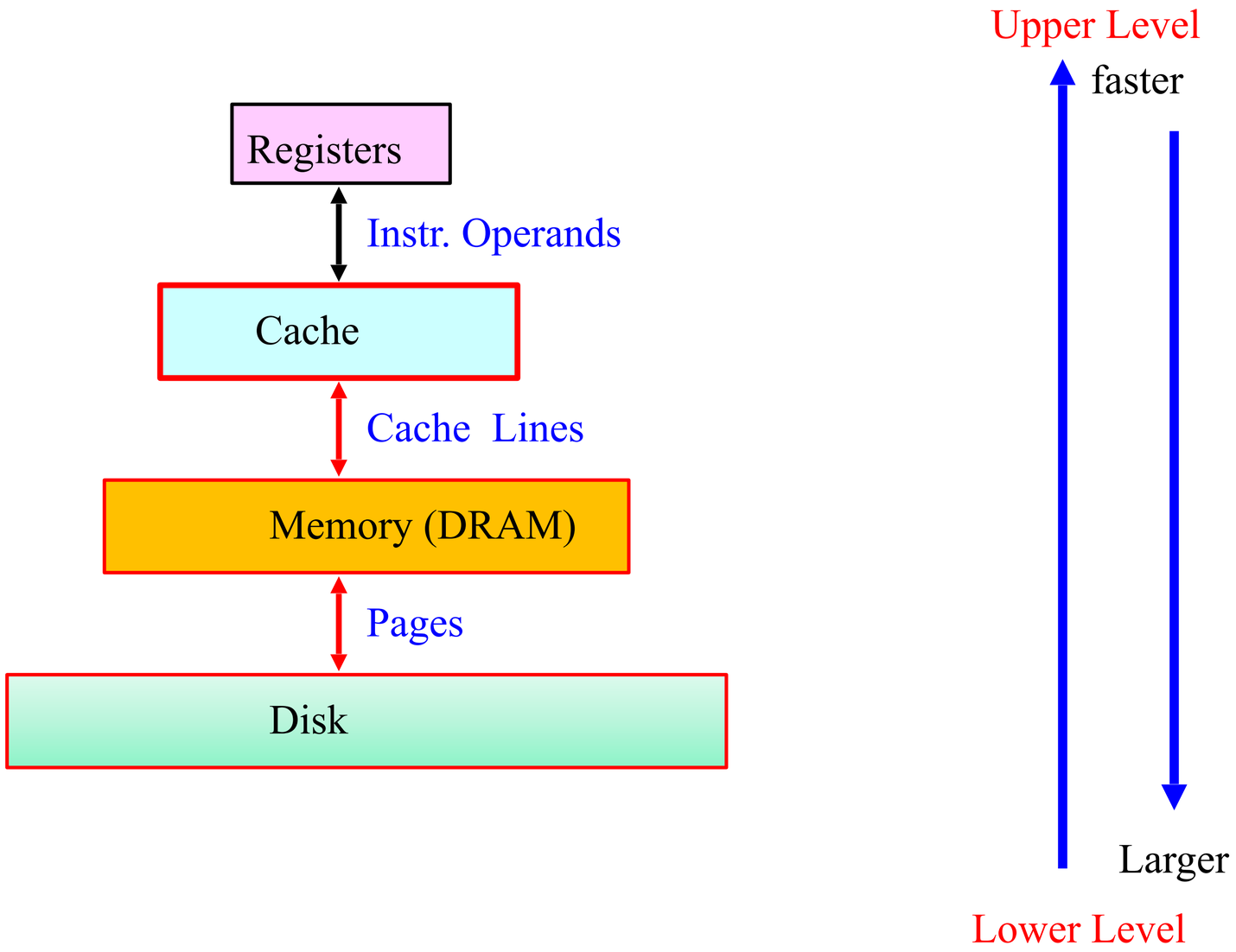
- *capacity*: Register << SRAM (Cache) << DRAM
- *latency*: Register << SRAM (Cache) << DRAM
- *bandwidth:* on-chip >> off-chip

On a data access:

if data $\in$ fast memory $\Rightarrow$ low latency access *Cache*

if data $\notin$ fast memory $\Rightarrow$ high latency access *DRAM*

Levels of the Memory Hierarchy



World with no caches

North pole ☹

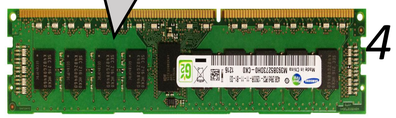
Core

32-bit Address

Data

200 to 300 cycles

Costly DRAM accesses☹



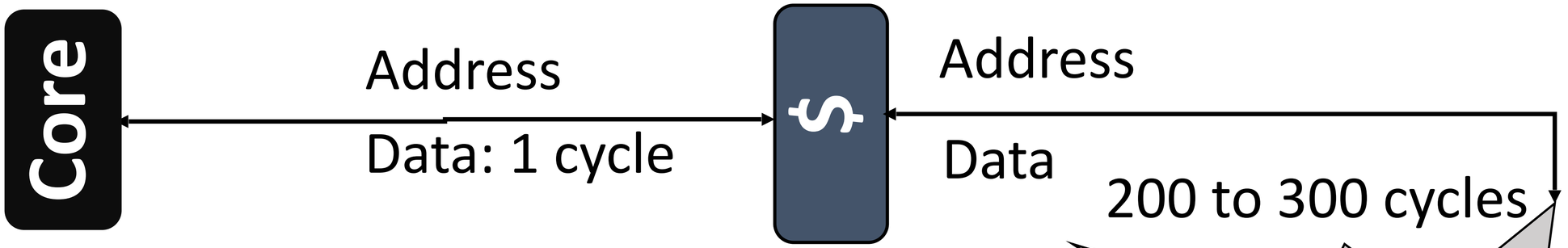
4 GB DRAM

South pole ☹

Minimizing costly DRAM accesses is critical for performance

Cache with latency

North pole ☺

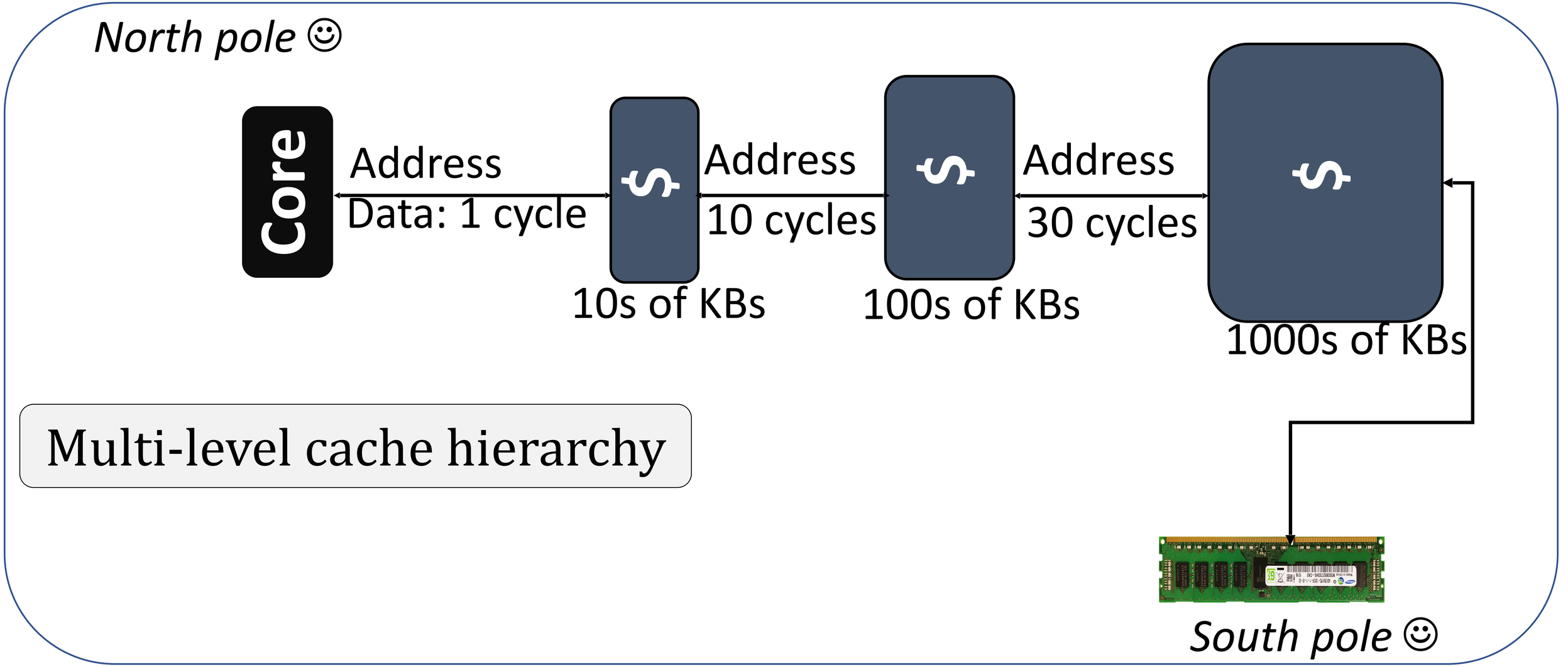


Costly DRAM accesses☹

32 to 64KB $ will be available in one to four cycles ☹

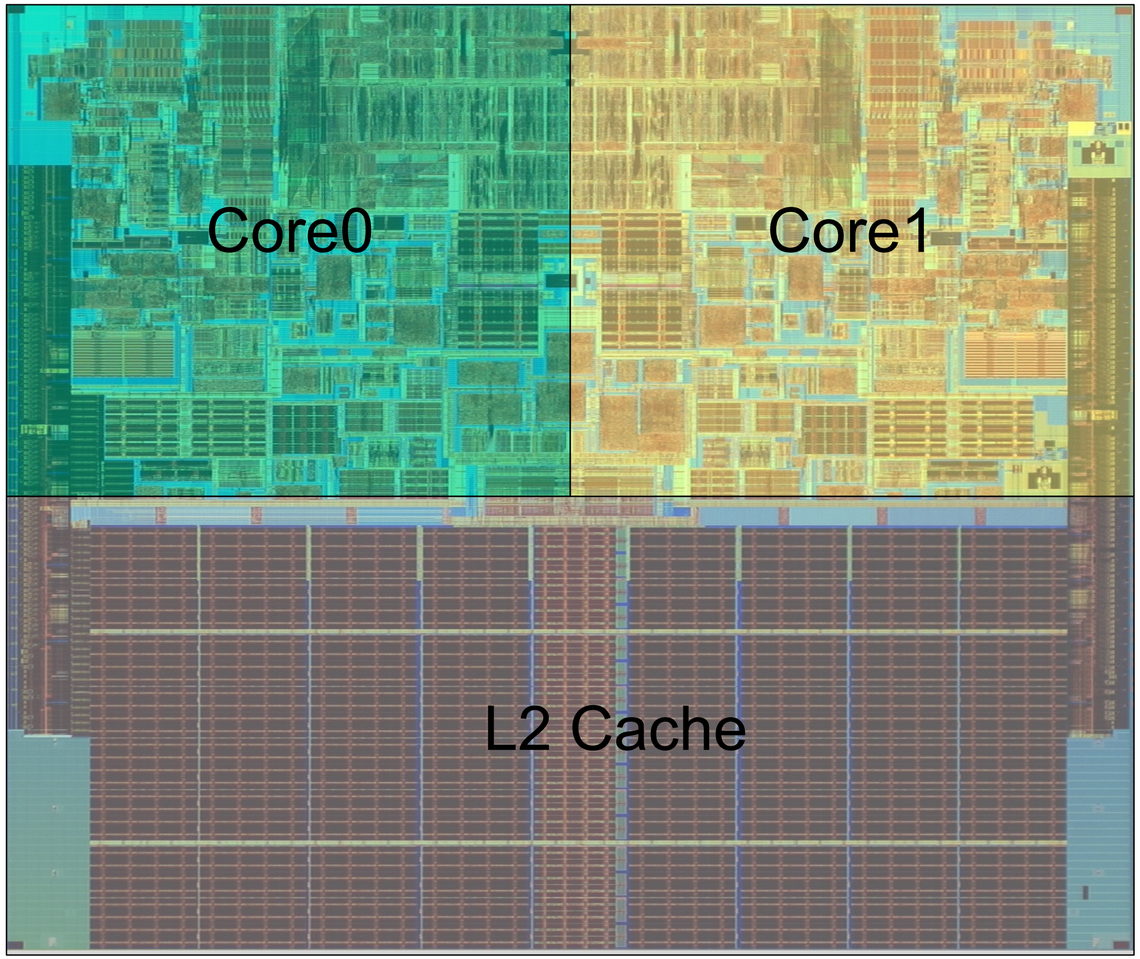
South pole ☺

Cache hierarchy with latency



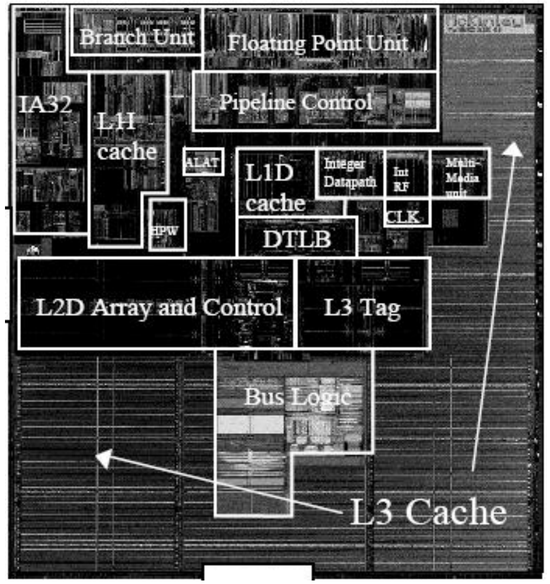
Case Study:Intel Core2 Duo

L1	32 KB, 8-Way, 64 Byte/ Line, LRU, WB 3 Cycle Latency
L2	4.0 MB, 16-Way, 64 Byte/Line, LRU, WB 14 Cycle Latency



Source: http://www.sandpile.org

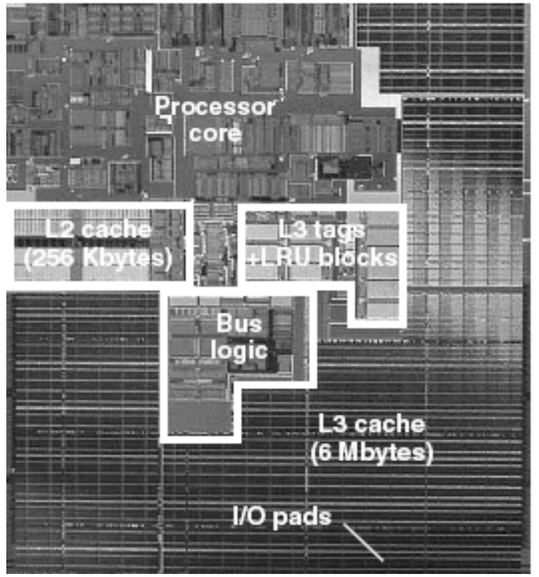
Case study: Intel Itanium 2



3MB
Version
180nm
421 mm^2

2002

6MB
Version
130nm
374 mm^2



Attribute	L1 instruction cache	L1 data cache	L2	L3
Size	16 Kbytes	16 Kbytes	256 Kbytes	6 Mbytes
Line size (bytes)	64	64	128	128
No. of ways	4	4	8	24
Replacement policy	Least recently used	Not recently used	Not recently used	Not recently used
Latency (no. of cycles)	1	Integer: 1 Floating-point: NA	Integer: 5 Floating-point: 6	14
Write policy	NA	Write through	Write back	Write back
Bandwidth (Gbytes/s)				
Read	48	24	48	48
Write	NA	24	48	48

Book

P & H, Chapter 4